

Distributional Semantics Tracing: A Framework for Explaining Hallucinations in Large Language Models

Gagan Bhatia

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science in Artificial Intelligence
of the
University of Aberdeen.



Department of Computing Science

2025

Declaration

No portion of the work contained in this document has been submitted in support of an application for a degree or qualification of this or any other university or other institution of learning. All verbatim extracts have been distinguished by quotation marks, and all sources of information have been specifically acknowledged.

Signed: Gagan Bhatia

Date: 2025

Abstract

Large Language Models (LLMs) are increasingly deployed in settings where factual reliability matters, yet they still produce fluent fabrications, “hallucinations”, that undermine trust and safety. This dissertation presents a mechanistic account of why such errors arise intrinsically from Transformer computation, rather than merely from data defects. It argues that hallucinations are predictable failures of internal reasoning caused by competition between a computationally fast, associative pathway (dominated by parametric co-occurrence knowledge in MLP layers) and a slower, contextual pathway (driven by self-attention to integrate prompt-specific evidence) (Geva et al., 2021; Meng et al., 2023; Cheng et al., 2025).

To diagnose these failures end-to-end, we introduce **Distributional Semantics Tracing (DST)**, a unifying explainability framework that integrates causal path tracing, activation patching and subsequence attribution into a single layer-wise semantic network. DST reconstructs the semantic network that the model instantiates at each layer, quantifying how specific associations amplify or suppress alternatives en route to the final token distribution. DST enables two central advances. First, it identifies the **onset**, **semantic inversion**, and **commitment** layers of a hallucination, localising the point of no return where internal representations irreversibly diverge from factuality. Building on DST, we propose **Distributional Semantics Strength (DSS)**, a quantitative measure of the coherence and stability of the slow, contextual pathway during inference. Across models, DSS strongly and negatively correlates with hallucination rates (Pearson $\rho = -0.863$, $R^2 = 0.746$), showing that weaker contextual coherence is a primary and **predictable** vulnerability that drives higher hallucination propensity.

Empirically, the study evaluates DST on two complementary benchmarks: HALoGEN, which induces diverse, verifiable factual errors across nine domains, and Racing Thoughts, which probes fine-grained contextualization (e.g., polysemy) with counterfactual pairs. DST consistently achieves the highest explanation faithfulness across both settings and model scales (e.g., on Gemma2-2B/9B, DST attains average faithfulness of 0.66/0.79 and is statistically superior to advanced baselines). Layer-wise case studies further illustrate DST’s diagnostic precision. In Qwen3-0.6B and OLMo-2-1B, DST pinpoints the emergence of spurious meanings (e.g., `bass`→`fish`) before inversion and final commitment, aligning the semantic network’s edge weights with observed token probabilities. Overall, this work reframes hallucination as an **architectural trade-off**, a byproduct of scaling choices that privilege fast associative retrieval alongside slower contextual reasoning, rather than as a mere knowledge deficit. By supplying faithful, causal explanations and actionable layer-level diagnostics, DST moves the field from post-hoc detection toward **predictive failure analysis** and targeted, real-time mitigation via representation-level control.

Acknowledgements

Hare Krishna

Firstly, I would like to express my deepest gratitude to my supervisor, Dr Yaji Sripada, for their steady guidance, sharp feedback, and the right mix of freedom and direction. Their insightful feedback and challenging questions were instrumental in shaping the core ideas of this dissertation and navigating the complexities of explainability. I am indebted to the professors who shaped my academic journey throughout this MSc AI program.

Secondly, I owe a mountain of gratitude to my family and friends, who were the unsung heroes of this thesis. To my parents, thank you for the endless encouragement and for providing the perfect environment for me to learn and succeed. Your belief in me never wavered, even when my own did, and your support was a constant comfort in dealing with my anxieties. To my incredible friends, thank you for being my sanity check. You tolerated me, entertained me, and made this degree truly memorable.

Finally, I offer my devotion to Lord Krishna and Radha Rani, whose spiritual guidance provided me with inner strength and clarity during the most challenging moments of this journey.

Contents

List of Figures	7
List of Tables	10
List of Abbreviations	12
1 Introduction	13
1.1 The Challenge of Hallucination	13
1.2 Motivation for a Mechanistic Approach	13
1.3 Research Questions and Objectives	15
1.4 Thesis Structure	15
2 Background	17
2.1 The Transformer Architecture	18
2.2 Distributional Semantics	18
2.3 The Hallucination Problem	19
2.4 Fast and Slow Reasoning Pathways	20
2.5 From Black Boxes to Interpretable Models	21
3 Literature Review	23
3.1 Situating the Problem: Research on LLM Hallucination	23
3.1.1 Characterisation and Benchmarking	24
3.1.2 Investigating Extrinsic and Behavioral Causes	24
3.1.3 A Survey of Mitigation Strategies	25
3.2 A New Paradigm for Diagnosis: Mechanistic Interpretability	25
3.3 Identifying the Research Gap and Thesis Contribution	28
4 Problem Statement and Solution Design	29
4.1 Problem Statement	29
4.2 Distributional Semantics Tracing (DST)	30
4.3 Evaluation Datasets	32
4.3.1 HALoGEN: A Benchmark for Breadth and Generality	32
4.3.2 Racing Thoughts: A Benchmark for Depth and Precision	33
4.4 Faithfulness Metric	34

5	Methodology and Experiments	37
5.1	Experimental Strategy	37
5.2	Models and Implementation Details	38
5.3	RQ1: Qualitative Comparison of Explainability Methods	38
5.4	RQ2: Quantitative Evaluation of Explanation Faithfulness	39
5.5	RQ3: Pinpointing the Onset of Hallucination	39
5.6	RQ4: Analyzing the Intrinsic Causes of Hallucination	40
6	Results and Discussion	42
6.1	RQ1: Understanding Explanations from Different Methods	42
6.1.1	Local Attribution Methods: SHAP and LIME	43
6.1.2	Token Evolution using Logit Lens	43
6.1.3	Causal Path Tracing	44
6.1.4	Patchscopes	45
6.1.5	Sparse Autoencoders	45
6.1.6	Subsequence Analysis	46
6.1.7	Distributional Semantics Tracing (Our Contribution)	46
6.2	RQ2: Evaluating Explainability Methods for Faithfulness	48
6.2.1	Performance on Contextual Reasoning (Racing Thoughts).	49
6.2.2	Performance on Diverse Hallucinations (HALoGEN).	50
6.3	RQ3: Pinpointing the Onset of Hallucinations in Model Layers	51
6.3.1	Case Study 1: Contextual Ambiguity	51
6.3.2	Case Study 2: Polysemy	52
6.4	RQ4: Explaining the Intrinsic Cause of Hallucination	53
6.4.1	Pathway Competition: Associative vs. Contextual Pathways	53
6.4.2	Failure Pattern 1: Reasoning Shortcut Hijack	54
6.4.3	Failure Pattern 2: Analogical Collapse	55
6.4.4	Hallucination as an Architectural Trade-Off	56
7	Conclusion	58
7.1	Key Findings and Contributions	58
7.2	Limitations	59
7.3	Future Work	59
	Bibliography	61
A	User Manual	73
A.1	Introduction	73
A.2	System Requirements and Installation	73
A.3	Interactive Analysis with Jupyter Notebooks	74
A.4	Core Experimental Framework	74
A.5	Advanced Interpretability Techniques	75
A.6	Causal Intervention and Control Experiments	76

A.7	Baseline Attribution and Comparative Analysis	76
A.8	Main Experimental Pipeline and Visualization	77
A.9	Model Configuration and Performance Optimization	77
A.10	Results Interpretation and Expected Outputs	77
B	Maintenance Manual	78
B.1	Project Architecture and Configuration	78
B.2	Core Library Structure and Organization	78
B.3	Experimental Implementation Framework	79
B.4	Example Script Architecture and Progression	79
B.5	Implementation Details and Key Algorithms	80
B.6	Configuration Management and Extension Patterns	80
B.7	Quality Assurance and Development Standards	80
C	Additional Information	82
C.1	LLM-as-Judge Prompt	82
C.2	Reproducibility Guide: Distributional Semantics Tracing (DST) Experiments . .	84
C.2.1	Setup & Installation	84
C.2.2	Qualitative Comparison of Explainability Methods	84
C.2.3	Pinpointing the Onset of Hallucination	86
C.2.4	Analyzing Intrinsic Causes	88
C.2.5	Quantitative Evaluation of Explanation Faithfulness	90

List of Figures

1.1	A visual example of an LLM hallucination. When asked for the capital of Mars, a planet with no official capital, the model confidently fabricates a plausible-sounding but entirely false answer, "Muskland." This illustrates how hallucinations can be grammatically correct and convincing, yet factually incorrect. Figure is taken from SuperAnnotate (2024).	14
2.1	Architectural Comparison of Transformer Decoders	17
2.2	Word Vectors in Semantic Space	19
2.3	An Overview of Hallucination Causes	20
2.4	Visual metaphor for the two reasoning pathways examined in this thesis. Left: Fast (associative) thinking —quick, pattern-matching, and statistically driven. Right: Slow (contextual) thinking —deliberate, compositional, and attention-driven. The figure anchors our hypothesis that conflicts between these pathways are a root cause of hallucination.	21
2.5	The Goal of Explainable AI	22
3.1	Contextualization Race Conditions	23
3.2	The HALOGEN benchmark pipeline flags hallucinations by decomposing a model's response into atomic claims and checking each span against a verifiable evidence source. This allows for the classification of errors, such as Type B (an incorrect fact that was likely present in the pre-training data) or Type C (a completely fabricated fact). Adapted from Ravichander et al. (2025).	24
3.3	Conceptual illustration of several methods used in this thesis: Logit Lens, Causal Path Tracing, and Patchscopes.	25
3.4	Illustration of Sparse Autoencoders, Subsequence Analysis Tracing, and our integrated Distributional Semantics Tracing.	26
3.5	Internal Mechanisms of Hallucination	26
3.6	Subsequence Association Model	27
4.1	The Distributional Semantics Tracing (DST) framework. It integrates signals from concept importance, patched representations, and subsequence analysis tracing to build a semantic network that reveals the conceptual relationships driving a prediction.	30
4.2	Distributional Semantics Tracing (DST) visualises the final-layer semantic network that drives the model's output.	31

5.1	The associative pathway makes a fast, computationally cheap inference based on simple word correlations. The contextual pathway performs slower, fact-based inference. A hallucination occurs when the high-probability token from the associative pathway is selected over the contextually appropriate token.	41
6.1	Side-by-side comparison of SHAP and LIME target–saliency heat-maps.	42
6.2	Side-by-side Token-Evolution visualisations. Left: hallucination example; Right: correct-reasoning example.	43
6.3	Causal Path Tracing explanations for (a) a correct response and (b) a hallucination. The lines trace the flow of information causally responsible for the final output token through specific components (highlighted in red on the x-axis) at different layers. (a) In the correct response, the path is semantically coherent. (b) In the hallucination, the path reveals that the model’s reasoning was corrupted by a spurious intermediate concept ("cars"), which causally influenced the final incorrect output.	44
6.4	Patchscopes analysis comparing entity probability trajectories during generation for (a) a hallucinated response and (b) a correct response. By tracking which entities the model considers, Patchscopes reveals how the model’s focus diverges from factual entities towards associated but incorrect concepts during a hallucination.	45
6.5	Sparse Autoencoder feature activations reveal the model’s conceptual reasoning. (a) For a correct response, the model activates a feature clearly related to trees. (b) To correctly disambiguate "bass" in a "concert" context, the model activates a feature related to music. A hallucination would be indicated by the activation of a contextually inappropriate feature.	46
6.6	Subsequence Analysis reveals the input spans driving the model’s output. (a) For a correct response, the model’s output (‘No’) is strongly associated with the relevant contextual subsequence. (b) For a hallucination, a spurious association (linking ‘trunk’ to ‘car’ instead of ‘tree’) is stronger than the correct one, narrowly leading to an incorrect output (‘No’) with low confidence.	47
6.7	Distributional Semantics Tracing (DST) visualizes the final-layer semantic network that drives the model’s output. (a) For a correct response, the network shows a coherent set of relationships. (b) For a hallucination, the network exposes a flawed semantic structure where a spurious association (‘trunk’–‘car’) competes with and overpowers the correct one (‘trunk’–‘tree’).	48
6.8	Layer-wise analysis of a contextual hallucination in Qwen3-0.6B (Qwen et al., 2024). The main plot illustrates the Distributional Semantics Strength at each layer. The inset graphs show the Distributional Semantics Tracing network at key layers. Initially (Layer 15), the model correctly distinguishes “bank” (river) from “money”. By Layer 22, the concepts begin to conflate, and by Layer 27, the model has committed to the incorrect financial meaning of “bank”, causing the hallucination. The green circle marks the onset of the hallucination, while the red circle indicates the layer of full commitment.	52

- 6.9 Layer-wise analysis of a polysemy-based hallucination in OLMo-2-1B (OLMo et al., 2025). The main plot illustrates the Distributional Semantics Strength at each layer. The model is prompted with “bass” in a “concert” context but hallucinates it is a “fish”. The analysis reveals a three-stage failure. At Layer 5, the model correctly associates “bass” with “concert”. The **prediction onset** (green circle) occurs at Layer 11, where a spurious correlation with “fish” emerges. The **semantic inversion point** (yellow circle) is reached at Layer 13, where the incorrect “fish” meaning surpasses the correct “concert” meaning. Finally, the model reaches its **commitment layer** (red circle) at Layer 15, locking in the hallucinated output. 53
- 6.10 The relationship between a model’s internal reasoning coherence and its tendency to hallucinate. The x-axis shows the **Distributional Semantics Strength (DSS)**, a metric we introduce to quantify the stability of a model’s contextual pathway. The y-axis shows the corresponding **Hallucination Rate**. The data reveals a strong negative linear relationship, with a Pearson correlation coefficient (ρ) of **-0.863** and an R-squared value of **0.746**. This provides strong quantitative evidence that a less coherent contextual pathway (lower DSS) is a primary and predictable vulnerability leading to higher rates of hallucination. 54
- 6.11 A mechanistic view of a **Reasoning Shortcut Hijack**. DST visualises how the model’s fast, high-frequency associative pathway from ‘Elon’+‘tech’ to ‘Musk’—a direct consequence of its learned distributional semantics—activates with such strength that it overpowers the slow, deliberative pathway required to correctly reason about ‘Ganor’, forcing a hallucination. 55
- 6.12 Visualization of an **Analogical Collapse**. The model’s deliberative pathway, required to execute the correct analogical logic ($v_{vote} + (v_{wildfire} - v_{spark})$), is abandoned in favor of the computationally cheaper, high-frequency topical association ($v_{vote} \rightarrow v_{democracy}$), leading to a reasoning failure. 56

List of Tables

4.1	Illustrative examples of prompts from the two evaluation benchmarks. The HALO-GEN example demonstrates a test of factual knowledge in an open-ended generation task, where a model might plausibly but incorrectly link two historical figures. The Racing Thoughts example demonstrates a targeted test of contextual understanding, probing whether the model can correctly disambiguate the word "bank." The hallucinated output is highlighted in red.	33
4.2	Representative examples from our LLM-as-Judge validation study, comparing our faithfulness metric against the average of five LLM judges. A small-sample Human Score is included for supplementary comparison.	34
4.3	Sensitivity analysis of the faithfulness metric. We start with a high-quality "Base Case" explanation and introduce targeted perturbations to test each component of our metric. The resulting drop in the metric score demonstrates its sensitivity to specific types of unfaithfulness.	35
5.1	Prompts used for qualitative analysis. Here, V. stands for Verbalisation.	39
6.1	Performance comparison of eleven explainability techniques, ranging from traditional attention analysis to advanced causal-tracing and ensemble methods, measured by faithfulness scores on the Racing Thoughts benchmark across four compact language models. Our method, Distributional Semantics Tracing , achieves the highest average faithfulness score (0.71), substantially outperforming all baseline, advanced, and ensemble methods.	49
6.2	Faithfulness results for explanation techniques on the Halogen benchmark Ravichander et al. (2025), broken out by five task domains—Code Package Imports (CODE), Biographies (BIO), False Presuppositions (FP), U.S. Senator Rationalization (R-SEN), and Scientific Attribution (REF)—for two Gemma2 model sizes (2B and 9B).	50

List of Abbreviations

t AI	Artificial Intelligence
LLM	Large Language Model
DST	Distributional Semantics Tracing (framework)
DSS	Distributional Semantics Strength (metric)
FFN	Feed-Forward Network
MLP	Multi-Layer Perceptron
MHA	Multi-Head Attention
Attn	(Self-)Attention mechanism
RMSNorm	Root Mean Square Normalization
SwiGLU	Swish-Gated Linear Unit
GQA	Grouped-Query Attention
RoPE	Rotary Positional Embeddings
XAI	eXplainable Artificial Intelligence
MI	Mechanistic Interpretability
RAG	Retrieval-Augmented Generation
DPR	Dense Passage Retrieval
SAE	Sparse Autoencoder
SHAP	SHapley Additive exPlanations
LIME	Local Interpretable Model-agnostic Explanations
KV cache	Key-Value cache (for attention)
GPT	Generative Pre-trained Transformer

Chapter 1

Introduction

Large Language Models (LLMs) have become a central focus of research and development in artificial intelligence, representing a significant advance in the field's ability to process and generate human language (Vaswani et al., 2017; Brown et al., 2020). A rapid succession of increasingly powerful models, including GPT-4, Gemini, Llama, and Qwen, have demonstrated state-of-the-art performance on a wide array of complex linguistic tasks (OpenAI et al., 2023; Team, 2023; Grattafiori et al., 2024; Qwen et al., 2024; DeepSeek-AI et al., 2024). The application of these models has expanded into numerous specialised domains, from scientific research to financial technology, where they offer substantial gains in efficiency and analytical capability (Taylor et al., 2022; Bhatia et al., 2024; Tatsat and Shater, 2025). This progress is mainly attributable to established scaling laws, which have consistently shown that performance improves with increases in model size, dataset volume, and computational resources (Kaplan et al., 2020; Hoffmann et al., 2022).

1.1 The Challenge of Hallucination

Despite their advanced capabilities, the practical utility of LLMs is significantly impeded by a critical failure mode known as *hallucination* (Ji et al., 2023). A hallucination is formally defined as the generation of content that is fluent and plausible but is factually incorrect, nonsensical, or inconsistent with a provided source context (Huang et al., 2023; Zhang et al., 2023b). This phenomenon is a primary obstacle to the reliable deployment of LLMs in high-stakes domains, including but not limited to medicine, law, and finance, where factual accuracy is a fundamental requirement for safe and effective operation (Tonmoy et al., 2024; Bai et al., 2024; Hu et al., 2025b; Tatsat and Shater, 2025). The problem is exacerbated by the tendency of LLMs to generate such fabrications with a high degree of confidence, making them difficult for users to identify and posing a significant risk to trust and safety (Venkit et al., 2024). Figure 1.1 provides a classic illustration of this behaviour, where the model invents a "fact" in a confident, human-like manner.

1.2 Motivation for a Mechanistic Approach

Effectively addressing the challenge of hallucination requires moving beyond surface-level evaluations of model outputs, as the internal mechanisms of these models remain largely opaque (Cambria et al., 2024; Zhao et al., 2024). Treating LLMs as "black boxes" is insufficient, as a model may produce a correct output via flawed reasoning by exploiting statistical shortcuts, or it may fail to generate an accurate output despite possessing the necessary factual knowledge within its

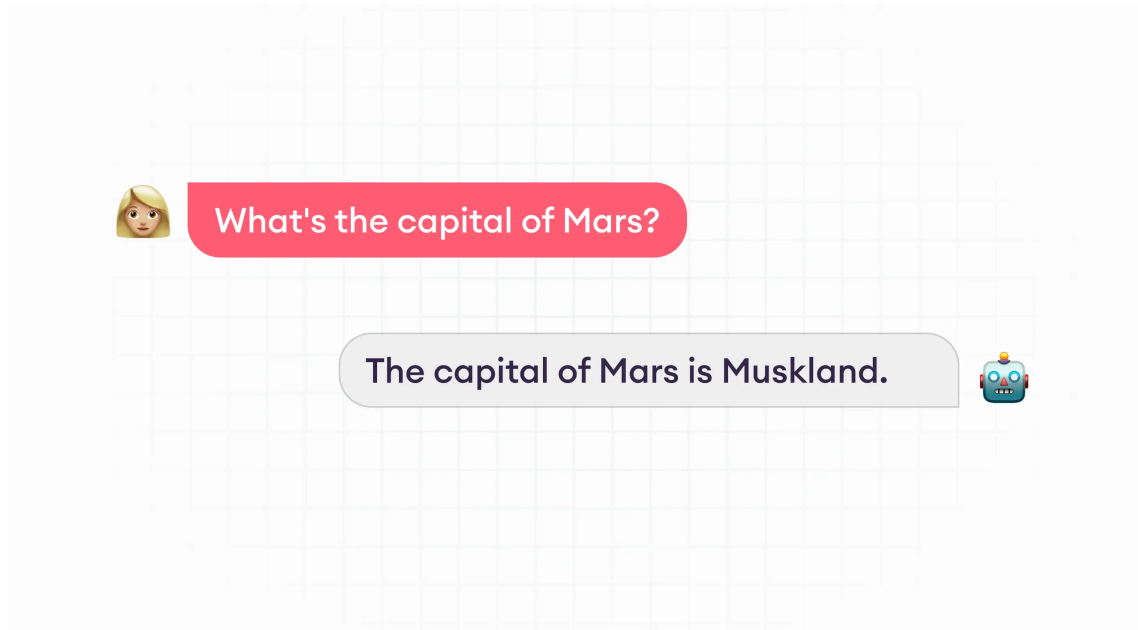


Figure 1.1: A visual example of an LLM hallucination. When asked for the capital of Mars, a planet with no official capital, the model confidently fabricates a plausible-sounding but entirely false answer, "Muskland." This illustrates how hallucinations can be grammatically correct and convincing, yet factually incorrect. Figure is taken from SuperAnnotate (2024).

parameters (McCoy, 2019; Geirhos et al., 2020). This discrepancy indicates that many hallucinations are not random errors or simple artefacts of the training data (Dziri et al., 2022). Instead, they are often symptoms of deeper, *intrinsic* failures within the model's computational architecture—systematic breakdowns in its internal processing pathways (Yu et al., 2024; Sun et al., 2024; Li et al., 2024).

This thesis is motivated by the need for a causally-grounded understanding of these internal failure modes, a research area known as mechanistic interpretability (Bereska and Gavves, 2024). We propose a conceptual framework wherein hallucinations arise from a conflict between two distinct computational pathways within the Transformer architecture, a model inspired by dual-process theories of cognition (Kahneman, 2011; Cheng et al., 2025). We hypothesise the existence of:

- A **"fast," associative pathway**, which relies on computationally efficient, high-frequency statistical patterns stored in the model's parameters Geva et al. (2021); Meng et al. (2023).
- A **"slow," contextual pathway**, which engages in more computationally intensive, deliberate reasoning required for logical and compositional tasks Olsson et al. (2022); Wei et al. (2023b).

Within this framework, a hallucination is theorised to occur when the associative pathway, activated by a strong but contextually inappropriate stimulus, overrides the contextual pathway. To investigate this hypothesis, it is necessary to develop and apply diagnostic tools capable of tracing information flow and identifying the specific mechanisms of failure. The approach of this thesis is grounded in the theory of **distributional semantics**, a foundational concept in linguistics and computational linguistics. The distributional hypothesis posits that the meaning of a word is

determined by the contexts in which it typically appears (Firth, 1957; Harris, 1954). Words that frequently occur in similar linguistic environments are assumed to share aspects of their meaning. In modern LLMs, this principle is instantiated computationally through high-dimensional vector representations, or *embeddings* (Peters et al., 2018). Each word or token is mapped to a vector in a continuous geometric space, where the model’s training process adjusts these vectors such that words with similar distributions are positioned closer to one another. Consequently, semantic relationships correspond to geometric relationships; for example, vectors for "king" and "queen" will be closer than vectors for "king" and "cabbage". From this perspective, a model’s reasoning process can be viewed as a trajectory through this semantic space (Kornblith et al., 2019). A factually correct output corresponds to a trajectory that remains in a semantically valid region of the space. We hypothesise that a hallucination can be understood as a form of **semantic drift**: a process where a concept’s internal representation strays from its factually-grounded meaning during the forward pass, causing the reasoning trajectory to veer into an incorrect region of the embedding space.

1.3 Research Questions and Objectives

The primary objective of this thesis is to explain the intrinsic causes of hallucination in LLMs mechanistically. The research aims to trace these factual errors to specific, predictable failures in the model’s internal semantic and computational pathways. To achieve this, the investigation is structured around four central research questions (RQs):

- RQ1:** *What are the comparative strengths and weaknesses of existing explainability methods for revealing a model’s reasoning, and can they be integrated into a more comprehensive framework?* This question is addressed through the development and evaluation of **Distributional Semantics Tracing (DST)**. (See Section 5.3 for methodology and Section 6.1 for results)
- RQ2:** *How can the faithfulness of an explanation be measured quantitatively and reliably?* This is answered by proposing and validating a composite **faithfulness metric** designed to assess the degree to which an explanation reflects a model’s internal decision-making process. (See Section 5.4 for methodology and Section 6.2 for results)
- RQ3:** *At what stage of the forward pass does a model’s internal state commit to a hallucination?* This is investigated by using layer-wise probing and DST to pinpoint the specific **commitment layer** where representations irreversibly diverge from factuality. (See Section 5.5 for methodology and Section 6.3 for results)
- RQ4:** *What are the underlying intrinsic mechanisms of these reasoning failures?* This question is addressed by applying the DST framework to provide a mechanistic account of how hallucinations arise from the conflict between associative and deliberative pathways, leading to failure patterns such as **Reasoning Shortcut Hijacks**. (See Section 5.6 for methodology and Section 6.4 for results)

1.4 Thesis Structure

This thesis is organised to present a systematic investigation of LLM hallucinations. **Chapter 2** provides the necessary technical background on the Transformer architecture, the phenomenon

of hallucination, and the tools of mechanistic interpretability. **Chapter 3** reviews the relevant academic literature, situating this work within the current state of the field. **Chapter 4** describes the research methodology, including the novel Distributional Semantics Tracing framework, the evaluation benchmarks, and the faithfulness metric. **Chapter 5** explains all of our experimental setups. **Chapter 6** presents the empirical results of the experiments conducted to answer the four research questions. Finally, **Chapter 7** summarises the contributions of the thesis, acknowledges its limitations, and proposes directions for future research.

Chapter 2

Background

This chapter provides the foundational technical and conceptual knowledge required to understand the central arguments of this thesis. Section 2.1 begins by detailing the Transformer architecture, the technological underpinning of modern Large Language Models (LLMs). Section 2.2 introduces the theory of distributional semantics, which explains how these models represent meaning. Section 2.3 then formally defines the problem of hallucination, presenting a taxonomy of its causes. Section 2.4 introduces the core conceptual framework for this thesis: the distinction between fast associative and slow contextual reasoning pathways within LLMs. Finally, Section 2.5 clarifies the concepts of explainability and interpretability, establishing the principles for analysing the internal workings of these complex models.

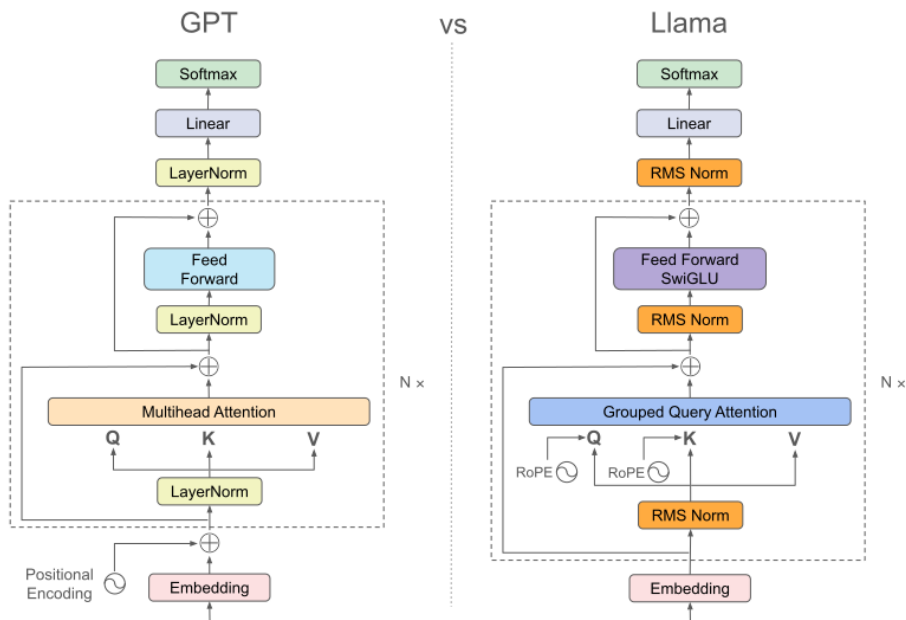


Figure 2.1: A comparison of the Transformer decoder block architecture in traditional GPT models versus modern Llama models. Llama introduces several key modifications for improved performance and efficiency: replacing LayerNorm with RMS Norm; using a SwiGLU activation function in the feed-forward network; substituting Multi-Head Attention with Grouped-Query Attention (GQA); and employing Rotary Positional Embeddings (RoPE) instead of learned positional encodings. Figures adapted from Touvron et al. (2023); Vaswani et al. (2017).

2.1 The Transformer Architecture

The recent and rapid progress in artificial intelligence has been largely driven by the development of massive, general-purpose **foundation models** trained on vast quantities of text data. This approach allows a single, pre-trained model to acquire a broad understanding of language, facts, and reasoning, which can then be adapted to a multitude of specific tasks (Brown et al., 2020).

This paradigm was enabled by the **Transformer architecture**, a breakthrough introduced by Vaswani et al. (2017). Unlike its predecessors, which processed text sequentially, the Transformer’s core innovation—the self-attention mechanism—enables the parallel processing of all tokens in a sequence. This architectural efficiency made it computationally feasible to scale models to billions of parameters trained on trillions of tokens, leading to the emergence of highly capable models from both industrial labs (e.g., Google’s Gemini, OpenAI’s GPT-4) and the open-source community (e.g., Llama, Mistral, Qwen) (Team, 2023; OpenAI et al., 2023; Touvron et al., 2023; Jiang et al., 2023; Yang et al., 2024a).

A large language model processes information through a series of stacked, identical layers, each containing two primary sub-components: a self-attention mechanism and a feed-forward network.

Self-Attention: The Context Engine. The self-attention mechanism is responsible for integrating contextual information. For each token in a sequence, it dynamically computes "attention scores" that weigh the relevance of all other tokens. It then produces a new, context-aware representation for that token by creating a weighted average of all token representations in the sequence.

Feed-Forward Networks: The Knowledge Stores. After the attention mechanism has aggregated contextual information, the updated representation for each token is processed independently by a **Feed-Forward Network (FFN)**. FFNs are where much of the model’s parametric knowledge—facts and learned associations—is believed to be stored and applied. Research suggests that these layers function as distributed key-value memories, activating stored information relevant to the input token’s representation (Geva et al., 2021; Chughtai et al., 2024).

These layers are interconnected with **residual connections** and **layer normalization** to ensure stable training and effective information flow. As shown in Figure 2.1, modern LLMs like the Llama family have introduced architectural refinements such as RMS Normalization, SwiGLU activations, Grouped-Query Attention (GQA), and Rotary Positional Embeddings (RoPE) to improve computational efficiency and performance (Touvron et al., 2023).

2.2 Distributional Semantics

Underpinning the Transformer’s ability to understand language is the principle of **distributional semantics**, famously summarised by the linguist J.R. Firth as: "You shall know a word by the company it keeps" (Firth, 1957). This theory asserts that the meaning of a word is not an intrinsic property but is defined by the contexts in which it appears. Words that frequently occur in similar linguistic environments are assumed to have similar meanings.

LLMs operationalise this theory by learning high-dimensional vector representations, or **embeddings**, for every token. During training, the model adjusts these embeddings so that tokens appearing in similar contexts are located close to one another in the vector space. As illustrated in Figure 2.2, this process captures complex semantic relationships as geometric relationships.

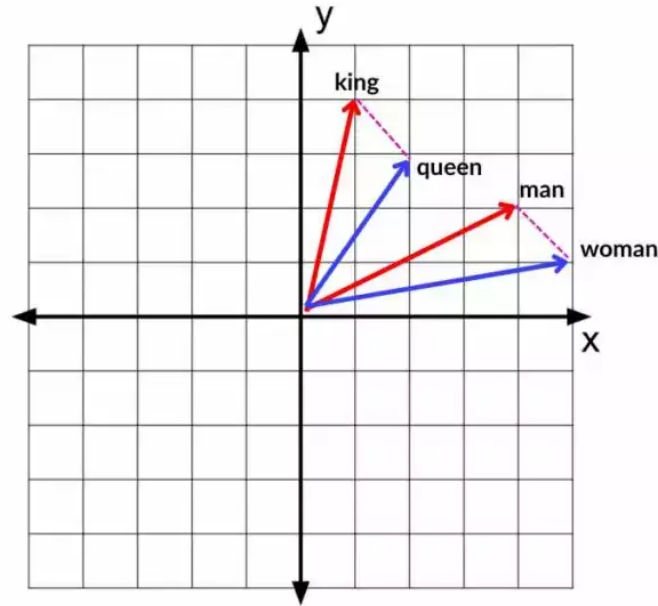


Figure 2.2: A visualization of the distributional hypothesis. Words are represented as vectors in a 2D space for simplicity. The spatial relationships between vectors capture semantic relationships. For example, the vector from "man" to "king" is similar to the vector from "woman" to "queen," encoding the concept of gender and royalty.

For instance, the vector offset between "man" and "woman" is remarkably similar to the offset between "king" and "queen," allowing the model to perform analogical reasoning through simple vector arithmetic (e.g., $\text{vector}('king') - \text{vector}('man') + \text{vector}('woman') \approx \text{vector}('queen')$). This geometric encoding of meaning is fundamental to how LLMs process and generate language.

2.3 The Hallucination Problem

Despite their impressive capabilities, LLMs are prone to a critical failure mode known as **hallucination**: the generation of text that is plausible and grammatically correct but is factually wrong, nonsensical, or untethered to provided source material (Huang et al., 2023). As illustrated in Figure 2.3, these failures are not random glitches but systematic errors that undermine model reliability. The origins of hallucinations are complex and can be broadly categorized as either extrinsic or intrinsic.

Extrinsic Causes (Data-Driven Failures). These failures originate from the data and methods used to train the model. The vast, web-scale corpora used for pre-training contain factual errors, biases, and contradictions, which the model inevitably learns (Dziri et al., 2022; Penedo et al., 2024). Furthermore, a model's knowledge is static and becomes outdated past its **knowledge cutoff**.

Intrinsic Causes (Model-Driven Failures). More central to this thesis are intrinsic failures, where a model hallucinates despite possessing the correct knowledge internally. These are algorithmic failures rooted in the model's architecture. They include **compositional errors** (combining known facts incorrectly), **attentional drift** (losing focus on relevant context in long prompts), and **heuristic over-reliance** (defaulting to a strong statistical association instead of performing context-specific reasoning).

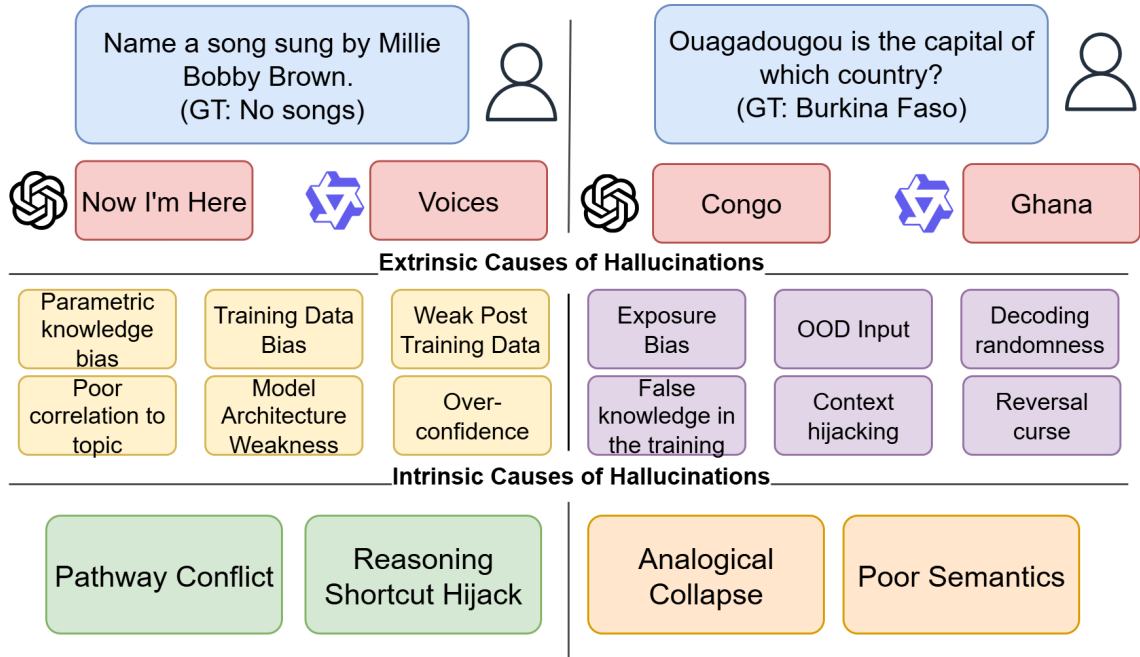


Figure 2.3: An overview of hallucination in LLMs. The top panel shows two real-world prompts that elicit incorrect, high-confidence responses from a capable model. The bottom panel presents a taxonomy of hallucination causes, divided into two main categories. **Extrinsic causes** are rooted in the data and training process, including issues like factual errors in the training corpus, knowledge cutoffs, and decoding randomness. **Intrinsic causes**, the focus of this thesis, are architectural failures within the Transformer itself, such as a conflict between reasoning pathways, the use of faulty reasoning shortcuts, and the collapse of analogical reasoning.

2.4 Fast and Slow Reasoning Pathways

To analyze the intrinsic causes of hallucination, this thesis adopts a conceptual framework inspired by dual-process theories of human cognition, which distinguish between a fast, intuitive "System 1" and a slow, deliberate "System 2" (Kahneman, 2011). We hypothesize that a similar duality emerges in LLMs as a consequence of their architecture and training objective (Cheng et al., 2025).

The "Fast" Associative Pathway. This pathway represents the model's default, computationally efficient mode of operation. It functions by leveraging strong, high-frequency statistical correlations learned from the training data. This pathway is primarily actualized by the MLP layers, which act as key-value memories that can quickly retrieve stored facts and common associations (Geva et al., 2021; Meng et al., 2023; Elhage et al., 2022). It excels at pattern completion and is responsible for the model's fluency, but it is context-agnostic and prone to activating the most statistically likely, rather than the most logically correct, information.

The "Slow" Contextual Pathway. This pathway is more computationally intensive and is required for tasks that demand compositional reasoning, logical inference, and sensitivity to specific context. It is primarily driven by the self-attention mechanism, which must perform complex operations to dynamically build representations by integrating information from across the input sequence (Vaswani et al., 2017; Olsson et al., 2022; Wei et al., 2023b). This pathway is essential for resolving ambiguity and handling novel prompts but is computationally "slower" and can be overridden by the stronger signals from the associative pathway.

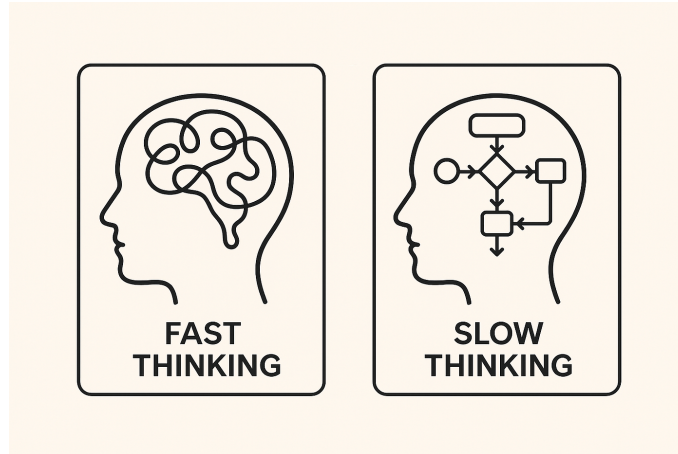


Figure 2.4: Visual metaphor for the two reasoning pathways examined in this thesis. **Left: Fast (associative) thinking**—quick, pattern-matching, and statistically driven. **Right: Slow (contextual) thinking**—deliberate, compositional, and attention-driven. The figure anchors our hypothesis that conflicts between these pathways are a root cause of hallucination.

Our central hypothesis is that many hallucinations are the result of a **pathway conflict**, where the fast associative pathway produces a strong, immediate, but contextually inappropriate response that hijacks the reasoning process before the slow contextual pathway can complete its more nuanced analysis.

2.5 From Black Boxes to Interpretable Models

Due to the vast number of parameters and complex, non-linear interactions within their architecture, LLMs are often characterized as "black boxes." Their internal decision-making processes are not immediately transparent to human developers and users. The field of **eXplainable AI (XAI)** is dedicated to developing methods that render these opaque models more understandable (Cambria et al., 2024). Within this field, it is useful to distinguish between interpretability and explainability.

Interpretability refers to the degree to which a human can understand the cause of a model's decision. Some models, like linear regression or decision trees, are intrinsically interpretable because their structure directly maps to their logic. Deep neural networks, conversely, have very low intrinsic interpretability. **Explainability**, on the other hand, is the process of generating a post-hoc artefact, an **explanation**, that provides a human-understandable justification for a specific prediction made by an otherwise uninterpretable model (Figure 2.5).

An explanation is the output of an interpretability method, but not all explanations provide the same level of insight. For the purposes of this thesis, the most critical distinction is between correlational and causal explanations.

Correlational Explanations. Most traditional XAI techniques provide correlational explanations. These methods, which include LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017), typically generate *feature attributions* or *saliency maps*. These outputs highlight which parts of an input (e.g., which words in a sentence) were most influential in leading to a particular decision. For example, a saliency map might show that the model assigned high importance to the word "not" in a sentiment classification task. While useful for understanding the input-output relationship, these explanations are fundamentally correlational. They identify *what* the model focused on but do not

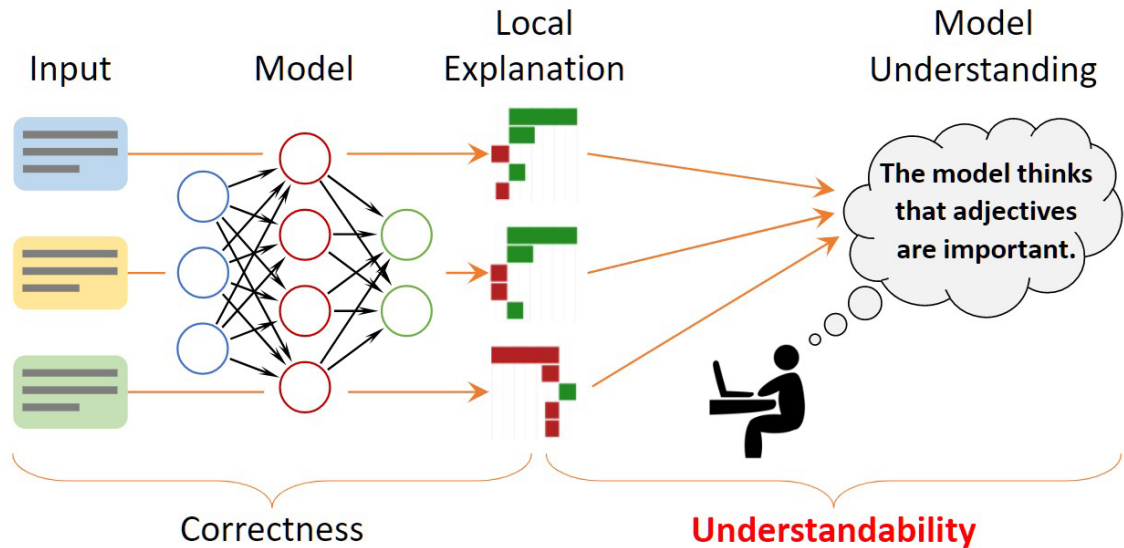


Figure 2.5: The distinction between correctness and understandability, the core goal of explainable AI (XAI). While a model's performance is measured by its correctness, XAI aims to achieve understandability by providing methods to reveal a model's internal logic. Image adapted from Cambria et al. (2024).

reveal the internal computational steps or the algorithm the model used to process that information.

Causal and Mechanistic Explanations. This thesis focuses on a more rigorous standard of explanation derived from the field of **Mechanistic Interpretability (MI)** (Olah et al., 2020; Bereska and Gavves, 2024). The objective of MI is to move beyond correlation and reverse-engineer the actual algorithm a model has learned to perform a task. A mechanistic explanation seeks to provide a causal account of a model's behaviour by identifying the specific components (e.g., attention heads, MLP neurons) and the pathways of information flow between them that are responsible for a given output. This approach treats the model as a complex but knowable circuit. Just as an engineer can trace signals through a physical circuit board to diagnose a fault, an MI researcher can trace information flow through a model's internal "circuits" to understand a failure like a hallucination.

Because our central hypothesis concerns a failure in the model's *internal reasoning process*, the conflict between fast and slow pathways, correlational methods are insufficient for our analysis. A mechanistic approach is therefore required to diagnose the precise architectural failures that lead to hallucinations.

Chapter 3

Literature Review

Research into the fallibility of Large Language Models (LLMs) has rapidly matured, progressing from the initial characterisation of hallucination as a behavioural problem to a more recent focus on developing a mechanistic understanding of its architectural origins. This evolution informs the present work, which aims to bridge these two areas by employing internal model analysis to provide a causal explanation for behavioural failures. This chapter surveys the key literature that forms the foundation for our investigation. Section 3.1 reviews the extensive body of work on the causes and mitigation of hallucination, highlighting the limitations of existing approaches that primarily treat the model as a black box. Section 3.2 then delves into the field of mechanistic interpretability, detailing the advanced tools that enable a causal analysis of a model’s internal computations. Finally, Section 3.3 synthesizes these two research streams to identify the critical gap in the literature that this thesis aims to address.

3.1 Situating the Problem: Research on LLM Hallucination

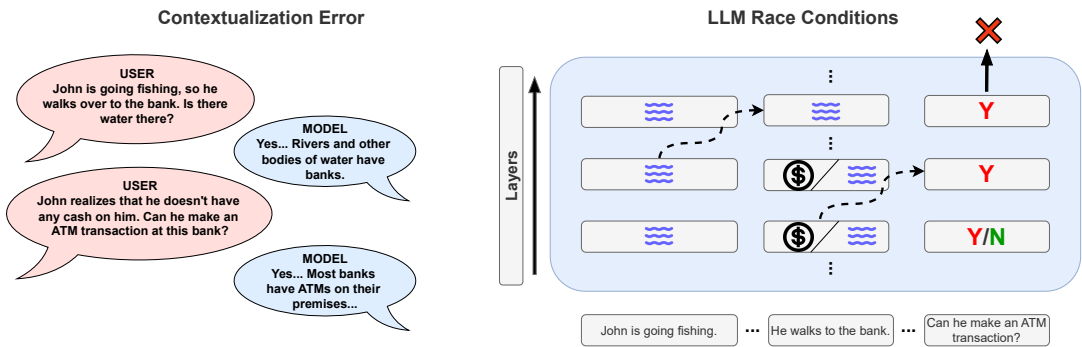


Figure 3.1: An illustration of contextualization error and “race conditions” inside an LLM. Two senses of the word *bank* (waterfront vs. financial institution) propagate along different representational paths. The faster, more statistically common path (financial) can reach the output headfirst, producing a contextually inconsistent answer. Adapted from Lepori et al. (2024).

The challenge of hallucination—the generation of fluent yet factually inaccurate content—was identified early in the scaling of LLMs and is now widely recognised as a primary barrier to their reliable deployment in high-stakes applications (Ji et al., 2023; Zhang et al., 2023b; Huang et al., 2023; Tonmoy et al., 2024; Bai et al., 2024; Venkit et al., 2024; Cleti and Jano, 2024). The initial wave of research in this area focused on characterising and benchmarking the phenomenon from an external, black-box perspective.

3.1.1 Characterisation and Benchmarking

Initial research efforts were dedicated to developing a clear taxonomy of the different types of factual errors that LLMs produce. This foundational work distinguished between various failure modes, such as fabricating non-existent entities, misstating relationships between real entities, and generating text that contradicts a provided source document (Zhang et al., 2023a; Nan et al., 2021; Hao et al., 2025; Walters and Wilder, 2023). To systematically measure these failures, the research community developed a suite of evaluation benchmarks. Prominent examples such as TruthfulQA, HaluEval, and HALoGEN provide standardized testbeds for quantifying a model’s propensity to hallucinate across diverse domains and question types, establishing a rigorous empirical basis for studying the problem (Goodrich et al., 2019; DeYoung et al., 2020; Min et al., 2023; Li et al., 2023; Ravichander et al., 2025; Zhang et al., 2024b). As shown in Figure 3.2, advanced benchmarks like HALoGEN employ automated verifiers to check each atomic claim in a model’s output against a knowledge source, providing granular error analysis.

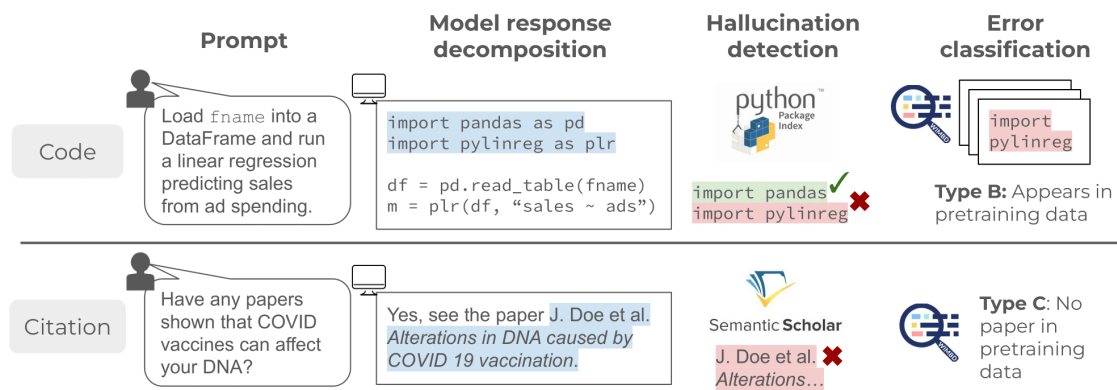


Figure 3.2: The HALoGEN benchmark pipeline flags hallucinations by decomposing a model’s response into atomic claims and checking each span against a verifiable evidence source. This allows for the classification of errors, such as Type B (an incorrect fact that was likely present in the pre-training data) or Type C (a completely fabricated fact). Adapted from Ravichander et al. (2025).

3.1.2 Investigating Extrinsic and Behavioral Causes

This foundational work on characterization identified several high-level root causes of hallucination. A significant body of research has traced these errors to flaws in the vast, web-scale training corpora used to pre-train LLMs. These datasets inevitably contain noise, factual inaccuracies, and societal biases, which are subsequently learned and reproduced by the model (Penedo et al., 2023, 2024; Soldaini et al., 2024; Dziri et al., 2022). Another line of inquiry has focused on the nature of knowledge storage in LLMs, revealing that their memorization of factual information is often brittle and superficial. Models may store facts in a way that is not robust to paraphrasing or that fails to generalize to new contexts (Dankers and Titov, 2024; Huang et al., 2024; Stoeck et al., 2024; Haviv et al., 2023; Lu et al., 2024; Zhu et al., 2024). A third major cause is the tendency for models to engage in *shortcut learning*, where they learn to rely on simple, spurious correlations in the training data instead of performing robust, multi-step reasoning. As Lepori et al. (2024) describe, this can lead to “race conditions” where an incorrect, high-frequency association outpaces

the correct, context-dependent interpretation, as illustrated in Figure 3.1.

3.1.3 A Survey of Mitigation Strategies

In response to these identified causes, a diverse ecosystem of mitigation strategies has been developed. These can be broadly categorized into two groups. First, **inference-time interventions** aim to improve factuality without altering the model’s parameters. The most prominent of these is Retrieval-Augmented Generation (RAG), which grounds the model’s output in external, verifiable documents retrieved from a knowledge base (Lee et al., 2022; Ren et al., 2023; Huo et al., 2023; Sun et al., 2024; Su et al., 2024; Liang et al., 2024). Other inference-time techniques include prompting the model to perform self-correction or to generate a chain of verification steps to check its own claims (Dhuliawala et al., 2023; Zhang et al., 2024a; Manakul et al., 2023; Li et al., 2025; Sanwal, 2025; Chu et al., 2025; Cheng et al., 2025; Lin et al., 2024).

Second, **model-centric approaches** involve altering the model itself through continued training. This includes fine-tuning on high-quality, curated datasets to improve factuality or using alignment procedures like Reinforcement Learning from Human Feedback (RLHF) to penalize the model for generating hallucinations (Hu et al., 2025b; Wei et al., 2023a).

While these strategies have proven effective at reducing the symptoms of hallucination, they share a common limitation: they largely operate on the model’s inputs and outputs. They do not, for the most part, address the underlying internal mechanisms within the model that produce the hallucinations in the first place.

3.2 A New Paradigm for Diagnosis: Mechanistic Interpretability

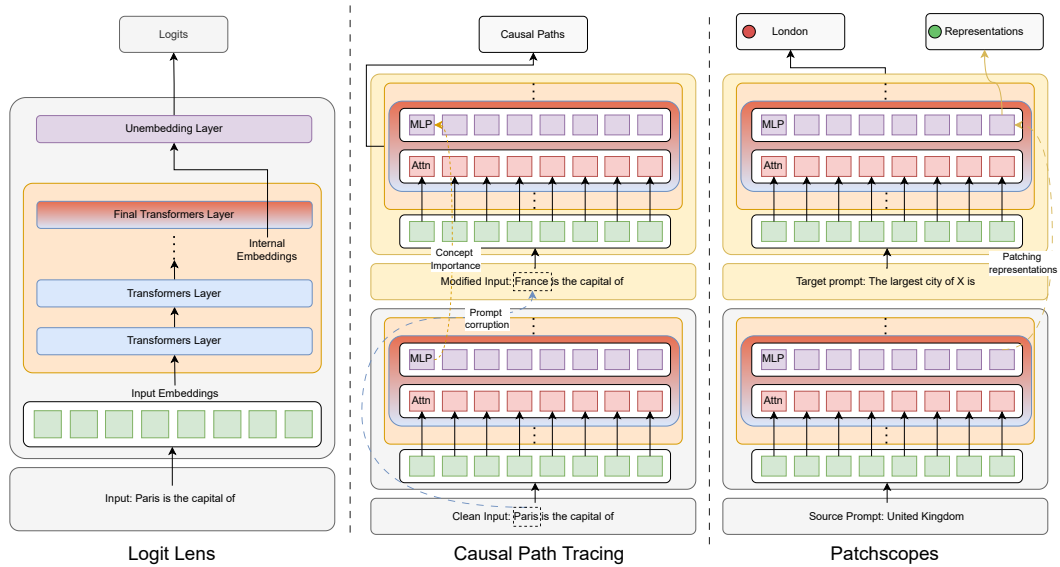


Figure 3.3: Conceptual illustration of several methods used in this thesis: Logit Lens, Causal Path Tracing, and Patchscopes.

To move beyond black-box fixes for hallucinations, this thesis adopts **mechanistic interpretability (MI)**—a causal, reverse-engineering approach to understanding the internal algorithms and computational structures that neural networks learn during training (Olah et al., 2020; Bereska and Gavves, 2024). Unlike traditional eXplainable AI (XAI), which often offers correlational,

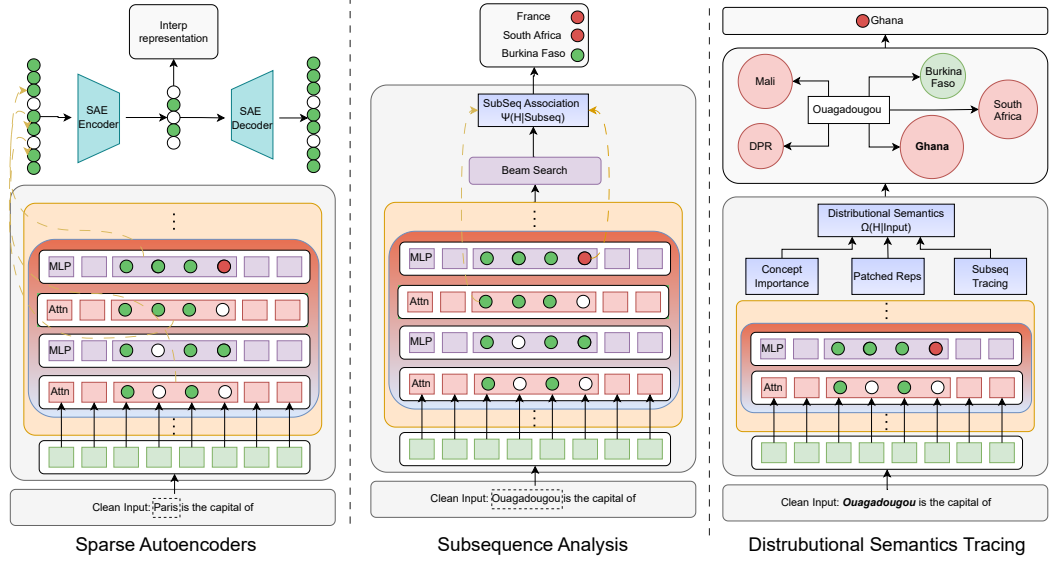


Figure 3.4: Illustration of Sparse Autoencoders, Subsequence Analysis Tracing, and our integrated Distributional Semantics Tracing.

post-hoc justifications (Ribeiro et al., 2016; Lundberg and Lee, 2017), MI aims to build human-understandable models of network computation, much like analyzing a complex circuit (Zhao et al., 2024; Lin et al., 2025; Palikhe et al., 2025; Singh et al., 2024). Central to this approach are *causal interventions* that probe how specific components (e.g., neurons, attention heads) implement distinct, scrutable functions.

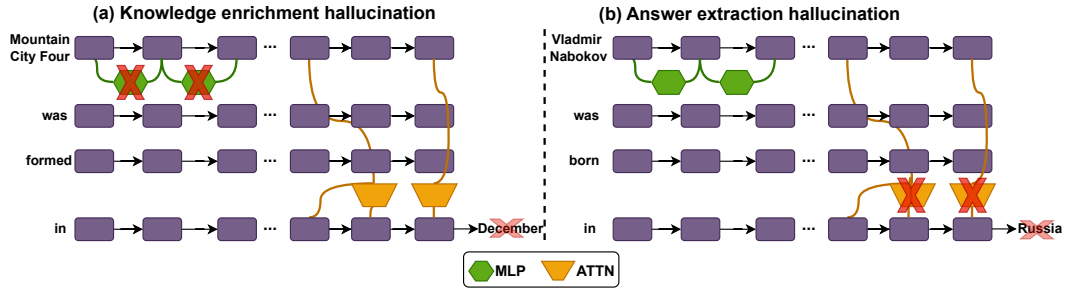


Figure 3.5: Two non-factual hallucination mechanisms identified in related work. **(a) Knowledge enrichment hallucinations** arise from insufficient general knowledge retrieved from early/middle-layer MLPs. **(b) Answer extraction hallucinations** stem from middle/late self-attention heads failing to identify the most relevant object for a given subject and relation. Figure from Yu et al. (2024).

Causal tracing and activation patching. A cornerstone of MI is to localize the *circuits* responsible for behaviors by contrasting a “clean” and a “corrupted” run. **Causal tracing** copies activations for specific components from the clean run into the corrupted run; if the correct output is restored, those components are inferred to be causally necessary (Meng et al., 2023). Iterating this procedure reveals minimal sets of interacting components for capabilities such as factual recall and multi-step reasoning (Wang et al., 2022; Ameisen et al., 2025; Zhang et al., 2025a), spanning applications from text-to-SQL to the emergence of implicit finite-state automata in chain-of-thought

settings (Harrasse et al., 2025; Zhang et al., 2025b).

Feature decomposition with Sparse Autoencoders (SAEs). Because neurons are often *poly-semantic*, MI employs **Sparse Autoencoders**—a dictionary-learning approach that reconstructs internal activations from a larger, sparse set of features. Enforcing sparsity promotes “monosemantic” units aligned with human-interpretable concepts (e.g., syntactic constraints or specific entities), enabling concept-level analysis of network states (Bricken et al., 2023; Cunningham et al., 2023; Minegishi et al., 2025).

Internal state inspection (Logit Lens and variants). The **Logit Lens** projects intermediate hidden states through the final unembedding to yield layer-wise “pseudo-predictions,” exposing when the model commits to candidate outputs or diverges from correct reasoning (Wang, 2025; Belrose et al., 2023).

Patchscopes. Extending causal tracing, **Patchscopes** search for the smallest subspace of a component’s activation that suffices to restore correct behavior when patched, thereby identifying low-dimensional, semantically meaningful directions whose misalignment can drive errors (Ghandeharioun et al., 2024).

Subsequence analysis and the subsequence association model. To link internal failures to specific prompt spans, **Subsequence Analysis** attributes predictions to contiguous input spans via a subsequence association $\Psi(y, |, [x_{i:j}])$ —the log-probability assigned to token y given span $[x_{i:j}]$ (Sun et al., 2025). This pinpoints textual triggers for spurious associations that can overwhelm correct knowledge.

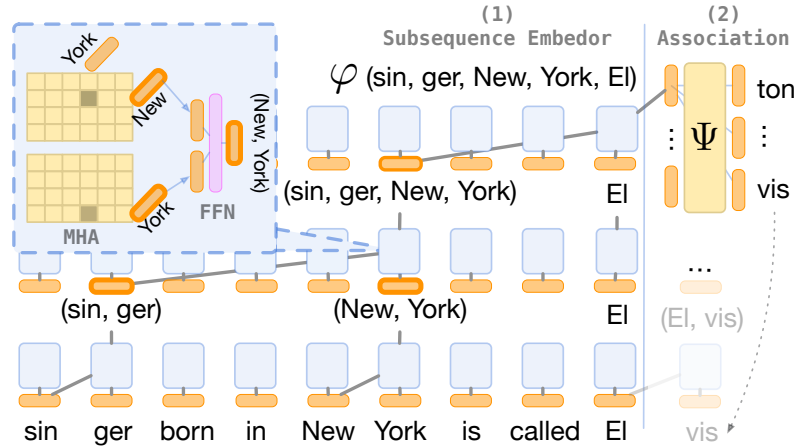


Figure 3.6: A two-stage view of language model computation: (1) a **Subsequence Embedder** (ϕ) maps text spans to embeddings; (2) an **Association** module (Ψ) maps those embeddings to a vocabulary distribution. Figure from Sun et al. (2025).

Correlational and attribution-based methods (XAI). We also integrate established, model-agnostic attribution tools that provide complementary, correlational signals. **LIME** fits a locally faithful, interpretable surrogate by perturbing inputs and weighting proximity to the original instance (Ribeiro et al., 2016). **SHAP** computes game-theoretic Shapley values to allocate contribution scores that sum to the prediction’s deviation from a baseline (Lundberg and Lee, 2017). **ReAGent** adapts attribution to generative LLMs by recursively replacing tokens with plausible alternatives (e.g., from RoBERTa) and measuring shifts in next-token distributions (Zhao and Shan,

2024).

Discoveries and applications. MI has illuminated how MLP layers act as distributed key-value memories for factual associations (Geva et al., 2021) and how recall composes additively across mechanisms (Chughtai et al., 2024). These tools have clarified behaviors such as multi-hop reasoning (Yang et al., 2024b) and multilingual concept structure (Schut et al., 2025; Wendler et al., 2024; Saji et al., 2025). For hallucinations specifically, MI distinguishes *knowledge-enrichment* failures from *answer-extraction* failures (Figure 3.5) (Yu et al., 2024) and explains how spurious subsequence associations can dominate correct information (Sun et al., 2025). While MLP layers store the primary associative knowledge, this knowledge can be inappropriately retrieved or misused by later-layer self-attention during answer extraction, reconciling reports (Figure 3.5) that implicate MLPs in enrichment-type errors and self-attention heads in extraction-type errors. Building on these diagnostics, **representation engineering** intervenes in internal states at inference time to steer models toward desired concepts (e.g., honesty), marking a shift from passive analysis to active control (Zou et al., 2025; Bartoszcze et al., 2025; Hu et al., 2025a; Cywiński et al., 2025).

3.3 Identifying the Research Gap and Thesis Contribution

The literature reviewed above reveals a critical gap. On one hand, the hallucination research community has developed a rich understanding of the external causes and behavioral patterns of factual errors, along with a suite of effective, but largely black-box, mitigation strategies. On the other hand, the mechanistic interpretability community has developed a powerful toolkit for analyzing the internal workings of LLMs, but its application to the specific, complex problem of hallucination is still in its early stages.

What is currently missing is a unified framework that connects these two worlds—one that uses the causal tools of MI to provide a comprehensive, end-to-end explanation of how a behavioral failure like hallucination arises from its specific architectural origins. We posit that many of the failures MI uncovers are predictable consequences of architectural trade-offs made to achieve efficient scaling, a subject of extensive research covering everything from Mixture-of-Experts/Depths (Fedus et al., 2022; Jiang et al., 2024; Raposo et al., 2024; Elhoushi et al., 2024) and KV cache compression (Xiao et al., 2023; Ge et al., 2023; Zhang et al., 2023c; Liu et al., 2023) to looped, recurrent computation (Dehghani et al., 2018; Giannou et al., 2023; Saunshi et al., 2025).

Our contribution is the **Distributional Semantics Tracing (DST)** framework, which integrates the MI toolkit to provide a unified, causal account of hallucination. We use it to demonstrate that hallucinations arise from a predictable conflict between a model’s computationally efficient "fast" associative pathway and its more deliberate "slow" contextual pathway, thereby directly linking a behavioural failure to its mechanistic and architectural origins.

Chapter 4

Problem Statement and Solution Design

This chapter details the methodological foundation of our investigation into the intrinsic causes of LLM hallucinations. The central aim of our methodology is to move beyond correlational observations and provide a more causally grounded explanation of why these factual errors occur within the Transformer architecture. We begin with explaining our Problem Statement (Section 4.1) that delineates the core gaps—namely, the absence of an explanation for hallucinations and the lack of a unified framework. Section 4.2 then formally introduces our primary contribution, the **Distributional Semantics Tracing (DST)** framework, a novel approach that synthesises these techniques. Section 4.3 details the evaluation benchmarks. Finally, Section 4.4 presents the composite metric developed to rigorously evaluate the faithfulness of the generated explanations, along with its validation.

4.1 Problem Statement

Despite rapid progress in large language models (LLMs), they continue to produce *hallucinations*, fluent but factually unsupported outputs, without a principled, mechanistic account of *why* these errors arise inside the Transformer architecture. Existing mitigations (e.g., retrieval augmentation, decoding constraints, post-hoc verification) can reduce incidence but do not explain the underlying failure process, leaving users unable to predict *when* a hallucination will occur, *which* internal computations cause it, or *how* to intervene at the right point in the forward pass. As a result, reliability remains brittle across domains, inputs, and model scales.

Compounding this gap, today’s interpretability tools provide only fragmented views. Token- or span-level attribution methods offer correlational importance over inputs but no causal link to hidden-state dynamics; per-layer decoding (e.g., logit lenses) reveals *when* beliefs drift but not the semantic factors driving that drift; component-level interventions (e.g., head/neuron patching) can identify responsible modules but do not explain the *conceptual* relationships they carry; representation analyses (e.g., sparse features) surface candidate concepts without connecting them to specific failures on specific prompts. There is, therefore, no unified, model-agnostic framework that integrates *input triggers*, *representation subspaces*, and *causal component interactions* into a single, falsifiable account that both explains and *quantifies* a hallucination’s emergence. This thesis addresses that methodological void by formulating precise requirements for such a framework: (i) layer-wise causal traceability from inputs to outputs, (ii) semantic grounding of internal features, (iii) local, prompt-specific explanations that scale to large benchmarks, and (iv) a quantitative signal that predicts failure and suggests actionable repair.

4.2 Distributional Semantics Tracing (DST)

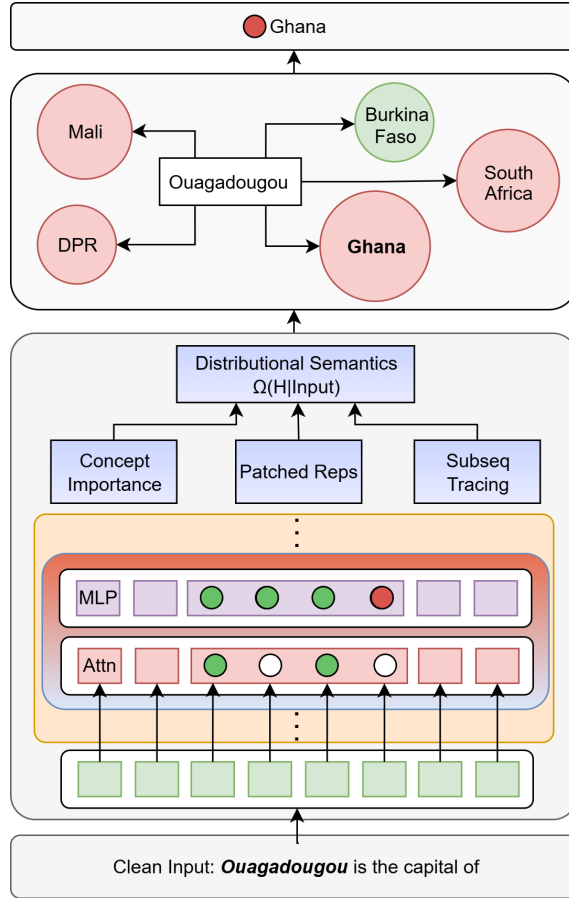


Figure 4.1: The Distributional Semantics Tracing (DST) framework. It integrates signals from concept importance, patched representations, and subsequence analysis tracing to build a semantic network that reveals the conceptual relationships driving a prediction.

The motivation for our framework, Distributional Semantics Tracing (DST), stems from the fragmented insights of existing interpretability techniques. While methods like the Logit Lens show **when** a prediction drifts (Wang, 2025), Causal Path Tracing identifies **which** components are responsible (Ameisen et al., 2025), and Sparse Autoencoders (SAEs) reveal **what** concepts are active (Bricken et al., 2023), each provides only a partial view. DST overcomes this by integrating these signals into a unified pipeline.

The DST Pipeline. As illustrated in Figure 4.1, the pipeline constructs a causal, layer-wise map of the model’s reasoning process through a multi-stage synthesis. The process begins by establishing the network’s nodes, which represent key concepts. These are identified from the prompt’s main entities and the most highly activated, interpretable features discovered by Sparse Autoencoders (SAEs) Cunningham et al. (2023). The strength of each concept node is then quantified using Subsequence Analysis Tracing Sun et al. (2025), which measures the influence of the corresponding input text. Subsequently, the edges connecting these nodes are drawn to represent the flow of causal information. The direction and strength of these edges are determined using Causal Path Tracing Bhatia et al. (2025) and Patchscopes Ghandeharioun et al. (2024), which measure the causal influence between the concepts. This integrated process assembles findings from each

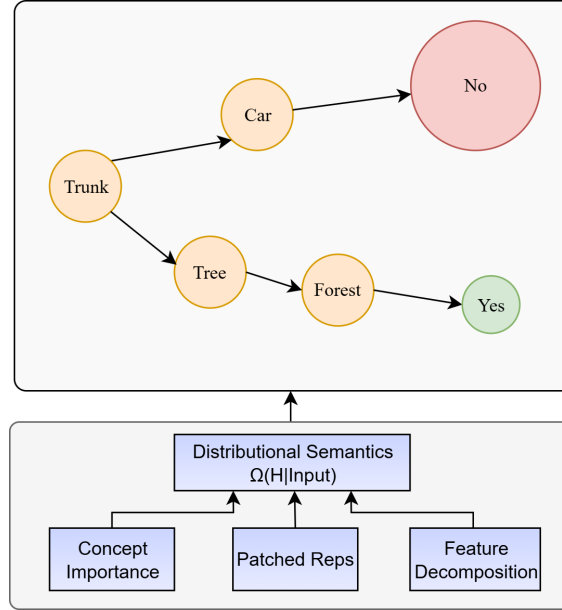


Figure 4.2: Distributional Semantics Tracing (DST) visualises the final-layer semantic network that drives the model’s output.

technique into a single, holistic semantic map that reveals the model’s reasoning.

The Semantic Network. The final output of the DST pipeline is a layer-wise **distributional semantic network**. As illustrated in Figure 4.2, this network serves as an interpretable map of the model’s reasoning. It shows how internal semantic relationships determine the final output probability, which we can formalise as $\Omega(H|\text{Input})$, the strength of a given hypothesis H based on the input. In this network, the nodes represent key concepts, and the weighted edges represent the strength of the links between them. The key concepts (nodes) are identified from the prompt’s entities and the most highly activated, interpretable features discovered by the Sparse Autoencoders (SAEs). The strength of each concept is then determined by Subsequence Analysis Tracing, which measures the influence of the corresponding input text. The connections between these nodes (edges), which represent the flow of information, are determined by Causal Path Tracing and Patchscopes. Together, these methods measure the causal influence between concepts, determining both the direction and strength of the arrows in the network. We can denote a specific internal link’s strength as $\Omega(A \Rightarrow B)$. The final output strength, $\Omega(H|\text{Input})$, is determined by the interplay of these various internal links that are activated during the reasoning process. Figure 4.2 shows this network for a polysemy task. In the hallucination Figure 4.2, the model outputs the incorrect hypothesis, H_{No} . This occurs because the internal associative link, represented by a high value for $\Omega(\text{Trunk} \Rightarrow \text{Car})$, is inappropriately strong and overrides the weaker contextual link, $\Omega(\text{Trunk}|\text{Forest} \Rightarrow \text{Tree})$. The strength of this faulty internal link directly leads to a high final output probability for the wrong answer. In essence, the generated network serves as a powerful, interpretable proxy for the underlying distributional semantics of each layer.

Distributional Semantics Strength (DSS) To empirically validate our hypothesis at scale, we introduce a novel metric called Distributional Semantics Strength (DSS). This metric is designed to provide a quantitative measure of the coherence and stability of a model’s “slow” contextual

pathway during a given reasoning task. The fundamental purpose of DSS is to move beyond anecdotal case studies and test whether a measurable weakness in a model’s ability to form correct, context-dependent semantic relationships is a reliable predictor of hallucination. The DSS for a given prompt is derived directly from the layer-wise semantic networks generated by our Distributional Semantics Tracing (DST) framework. The calculation involves two main steps. First, we use DST to trace the reasoning process and generate the final-layer semantic network. Second, we quantify the "strength" of this network by analysing the relationships between the key concepts. Specifically, we measure the edge weight (representing semantic and causal strength) of the contextually correct pathway (e.g., the link between trunk and tree in a forest context) and compare it to the strength of any competing, contextually incorrect associative pathways (e.g., the link between trunk and car).

Let $\mathcal{C}$ be the set of correct contextual pathways and $\mathcal{A}$ be the set of all active pathways (contextual $\cup$ associative). If s_p is the strength of a given pathway at a given layer p , then DSS is formally defined in Equation 4.1. The metric calculates the ratio of the summed strengths of all correct contextual pathways to the summed strengths of all active pathways. Here, ϵ is a small constant to avoid division by zero.

$$\text{DSS} = \frac{\sum_{p \in \mathcal{C}} s_p}{\sum_{p \in \mathcal{A}} s_p + \epsilon} \quad (4.1)$$

A high DSS score indicates that the correct contextual path is strong, stable, and dominant within the network. Conversely, a low DSS score signifies a weak or ambiguous contextual pathway, where spurious associations compete with or overpower the correct interpretation. By computing the DSS across thousands of examples, we can test the direct correlation between the strength of the slow pathway and the model’s rate of hallucination, thereby providing large-scale evidence for our hypothesis.

4.3 Evaluation Datasets

The selection of appropriate benchmarks is critical for a comprehensive and rigorous assessment of our proposed framework. Different benchmarks are designed to probe distinct types of model failures, and a thorough evaluation requires testing across a spectrum of challenges. While numerous benchmarks for factuality and hallucinations exist, such as TruthfulQA for common misconceptions (Lin et al., 2022) and HaluEval for general-purpose hallucination detection (Li et al., 2023), we deliberately selected two complementary benchmarks to test our framework for both **breadth** and **depth**. This dual approach allows us to assess the general applicability of our methods while also verifying their precision on nuanced semantic failures.

4.3.1 HALoGEN: A Benchmark for Breadth and Generality

To evaluate the general robustness and broad applicability of our diagnostic framework, we use **HALoGEN** (Ravichander et al., 2025). HALoGEN is a large-scale and comprehensive benchmark consisting of nearly 11,000 prompts that span nine diverse and knowledge-intensive domains, ranging from structured tasks like code generation and scientific attribution to more open-ended tasks like biography writing and question answering.

Prompt	Correct Answer	Hallucination
HALoGEN (Biography Domain)		
"Write a short biography of the mathematician Ada Lovelace."	"Ada Lovelace was an English mathematician... known for her work on Charles Babbage's proposed Analytical Engine. She is often regarded as the first computer programmer."	"Ada Lovelace was an English mathematician... known for her work on Charles Babbage's proposed Analytical Engine. She later went on to work with Alan Turing at Bletchley Park during WWII. "
Racing Thoughts (Contextualization Error)		
"John is going fishing, so he walks over to the river bank. Can he withdraw money at the bank?"	"No."	" Yes , he can go to the ATM located at the bank."

Table 4.1: Illustrative examples of prompts from the two evaluation benchmarks. The HALoGEN example demonstrates a test of factual knowledge in an open-ended generation task, where a model might plausibly but incorrectly link two historical figures. The Racing Thoughts example demonstrates a targeted test of contextual understanding, probing whether the model can correctly disambiguate the word "bank." The hallucinated output is highlighted in red.

A key strength of HALoGEN is its design, which systematically induces different categories of hallucinations. Its prompts are constructed to test a model's ability to recall facts correctly, avoid fabricating information, and remain faithful to provided context. Furthermore, it includes high-precision, automated verifiers for each domain that decompose a model's generated response into atomic claims and check each claim against a reliable knowledge source. This rigorous setup provides a granular and scalable way to measure hallucination rates. We chose HALoGEN to test the **generality** of our framework across a wide variety of hallucination types and to ensure our findings are not limited to a narrow set of reasoning failures.

4.3.2 Racing Thoughts: A Benchmark for Depth and Precision

To evaluate the precision and diagnostic power of our framework on subtle semantic failures, we use the **Racing Thoughts** benchmark (Lepori et al., 2024). Unlike large-scale, broad-coverage benchmarks, Racing Thoughts is specifically designed to probe for a critical and complex type of reasoning failure: **contextualization errors**. These errors occur when a model fails to correctly interpret a word with multiple meanings (a polysemous word) based on its surrounding context.

The benchmark employs a counterfactual design, presenting models with minimally different prompts where a change in a single contextual cue alters the correct interpretation of a key term. For example, it tests whether a model understands that a "bank" in the context of a "river" is different from a "bank" in the context of "money." By analysing the model's performance on these carefully controlled pairs, the benchmark can isolate failures of contextual grounding from simple factual recall. We chose this benchmark to test the **diagnostic precision** of our framework

on nuanced, reasoning-based errors that go beyond the retrieval of memorised facts. Table 4.1 provides illustrative examples from both benchmarks to highlight their distinct characteristics. The HALoGEN example tests for factual fabrication in an open-ended generation task, while the Racing Thoughts example tests for a specific contextualization failure using a targeted question.

4.4 Faithfulness Metric

We evaluate the faithfulness of explanations using a formal, composite metric. First, **Strength of Evidence** confirms that the explanation cites input tokens with high model-internal saliency, as measured by gradient-based attribution. Second, **Agreement with Truth** checks whether the explanation’s internal logic, when simulated, arrives at the correct conclusion.

In addition to this formal metric, our validation process includes a **Relevance to Verbalisation** analysis. This step measures the semantic alignment between our mechanistic explanation and the model’s own Chain-of-Thought (CoT) output, where available. The purpose is not to treat the CoT as infallible ground truth, but rather to ensure our explanation does not fundamentally contradict the model’s own stated reasoning. This check serves as a guardrail against significant discrepancies; if our mechanistic explanation points to one reasoning path, but the model’s own stated logic describes an entirely different one, it suggests a potential flaw in either our explanation or the model’s self-awareness that warrants further investigation.

Our overall evaluation framework, combining the two-part metric with the Relevance to Verbalisation analysis, was validated to ensure its reliability, as detailed below. This ensures our findings are based on explanations that reliably reflect the model’s computational process. By combining evidence alignment, logical correctness, and verbalisation fidelity into a single composite score, we obtain a metric that ranges from completely unfaithful (when none of the criteria are met) to fully faithful (when all are satisfied). As a result, our metric provides a rigorous, interpretable measure for comparing explanation techniques and tracking progress toward genuinely transparent and trustworthy language models.

LLM Prompt (Context)	LLM Output	Explanation from Algorithm	LLM Judge Score	Human Score	Metric Score
"Please answer succinctly. I am in a forest. I see a trunk. Is it an automobile part?"	"Yes."	"The model hallucinated by ignoring the context 'in a forest'. It defaulted to the automotive meaning of 'trunk' instead of the biological one."	9.55	10.0	9.41
"Please answer succinctly. I am at a baseball game. I see a pitcher. Is it a container of liquid?"	"Yes."	"The model hallucinated because 'pitcher' can also mean a person who throws a ball."	6.90	7.24	6.75
"Answer based on the information provided here...The capital city of Chad was just renamed...to Canberra. Is the capital city of Chad named Canberra?"	"No."	"The model answered based on real-world geography."	2.15	3.30	1.85

Table 4.2: Representative examples from our LLM-as-Judge validation study, comparing our faithfulness metric against the average of five LLM judges. A small-sample Human Score is included for supplementary comparison.

Metric Validation To validate our metric, we performed three analyses: an "LLM-as-Judge" evaluation to proxy human assessment, a correlation analysis against the model’s verbalisations, and a sensitivity analysis to test its robustness.

First, to approximate expert human evaluation, we employed an "LLM-as-Judge" framework

LLM Prompt (Context)	LLM Output	Explanation from Algorithm	Metric Score
Base Case: A fully faithful explanation			
"Please answer succinctly. I am in a forest. I see a trunk. Is it an automobile part?"	"Yes."	"The model hallucinated by ignoring the context 'in a forest'. It defaulted to the automotive meaning of 'trunk' instead of the biological one."	10.0
Perturbation 1: Degrade Strength of Evidence			
"Please answer succinctly. I am in a forest. I see a trunk. Is it an automobile part?"	"Yes."	"The model hallucinated because it saw the word 'automobile' and focused on that."	3.33
Perturbation 2: Degrade Agreement with Truth			
"Please answer succinctly. I am in a forest. I see a trunk. Is it an automobile part?"	"Yes."	"The model did not hallucinate. It correctly identified 'trunk' as an automobile part, ignoring the context."	0.0
Perturbation 3: Degrade Relevance to Verbalization			
"Please answer succinctly. I am in a forest. I see a trunk. Is it an automobile part?"	"Yes."	"The model hallucinated because tree trunks are not typically found in cars."	4.35

Table 4.3: Sensitivity analysis of the faithfulness metric. We start with a high-quality "Base Case" explanation and introduce targeted perturbations to test each component of our metric. The resulting drop in the metric score demonstrates its sensitivity to specific types of unfaithfulness.

using detailed judge prompts to guide the assessment, as shown in Appendix C.1. We used a panel of five diverse, state-of-the-art models (Gemini 2.5 Pro, Gemini 2.5 Flash, Qwen/Qwen3-235B-A22B-Instruct-2507, GPT-4o, and Claude 4 Sonnet), prompted to act as expert human annotators. Each LLM judge scored the faithfulness of 100 explanations on a 0-10 scale, with any tie-breaking resolved by averaging the scores of the tied judges. The reliability of the LLM annotations was high, confirmed by a Fleiss' Kappa score of **0.85**. We then compared the average LLM Judge Scores to our metric's scores, finding a Pearson correlation coefficient of **$r = 0.9245$** ($p < 0.0001$). This very strong, statistically significant correlation demonstrates that our metric accurately captures what a panel of expert models perceives as explanation faithfulness. Table 4.2 provides representative examples from this study; for additional validation, we include scores from a small, informal human study, which align with both the LLM judges and our metric.

Second, we leveraged the **Relevance to Verbalisation** component as a direct validation signal. We found a strong, positive semantic correlation (Pearson correlation coefficient of **$r = 0.8942$** ($p < 0.001$)) between the explanations generated by our method and the model's own Chain-of-Thought reasoning. This alignment confirms that our metric rewards explanations that are not only mechanistically sound but also consistent with the model's expressed rationale.

Finally, we conducted a sensitivity analysis to confirm that our metric responds appropriately to specific types of unfaithfulness. As shown in Table 4.3, we began with a high-quality, faithful explanation and then systematically introduced perturbations designed to degrade one facet of the metric. The resulting drop in the metric score demonstrates that our metric is robustly sensitive to the individual components that constitute a faithful explanation.

Taken together, the high correlation with LLM-as-Judge evaluations, the strong alignment with model verbalisations, and the successful sensitivity analysis provide strong evidence that our

metric is a valid and reliable instrument for measuring explanation faithfulness.

Chapter 5

Methodology and Experiments

This chapter details the complete experimental methodology designed to address the four central research questions of this thesis. Our goal is to move from observing hallucinations to explaining their intrinsic, mechanistic origins within the Transformer architecture. To achieve this, we devise a multi-stage experimental plan that progressively deepens our analysis: first, we establish *what* different explainability methods can reveal (RQ1); second, we rigorously measure *how well* they explain (RQ2); third, we pinpoint *when* in the computation a failure occurs (RQ3); and finally, we investigate *why* the failure happens (RQ4).

5.1 Experimental Strategy

Our research strategy is structured as a cumulative investigation, where each experiment builds upon the insights of the previous one to construct a comprehensive, causally-grounded account of LLM hallucinations. The overarching goal is to answer the fundamental **what, how, when, and why** of this failure mode.

What Explanations Reveal (RQ1): We begin with a qualitative exploration to understand the landscape of existing interpretability tools. The purpose of this initial stage is to establish a baseline by comparing the explanatory outputs of various methods on a controlled set of hallucination cases. This allows us to identify their individual strengths (e.g., Causal Path Tracing for component identification) and weaknesses (e.g., SHAP’s locality), motivating the need for an integrated framework like DST.

How Faithful are Explanations? (RQ2): A qualitative understanding is insufficient without quantitative validation. The second experiment addresses this by systematically evaluating the faithfulness of our proposed DST framework against a suite of baseline methods. By applying our validated faithfulness metric across large-scale, diverse benchmarks (HALoGEN and Racing Thoughts), we aim to empirically demonstrate which methods produce the most reliable and accurate explanations, thereby validating DST as a superior diagnostic tool.

When do Hallucinations Begin? (RQ3): With a faithful diagnostic tool established, we shift our focus from the final output to the internal process. This experiment aims to trace the temporal origin of a hallucination within the model’s forward pass. By applying DST on a layer-by-layer basis, we seek to identify the precise "commitment layer" where a model’s internal representation irreversibly diverges towards a factual error. This answers the crucial question of *when* a failure becomes inevitable.

Why do Hallucinations Occur? (RQ4): The final and most critical stage of our strategy is to

explain the root cause of these failures. We hypothesise that hallucinations arise from a fundamental conflict between two competing reasoning pathways. This final experiment tests this theory at scale. We use our DST framework to derive the Distributional Semantics Strength (DSS) metric, a quantitative proxy for the coherence of a model’s contextual reasoning. By correlating this metric with hallucination rates, we seek to provide large-scale evidence that a measurable weakness in this pathway is a primary intrinsic cause of factual errors.

This four-stage progression, from qualitative comparison to quantitative validation, temporal localisation, and finally, causal explanation, forms a cohesive and rigorous methodology for deconstructing the phenomenon of LLM hallucination.

5.2 Models and Implementation Details

All experiments were conducted using publicly available, decoder-only Transformer models, including the **Gemma2** family (2B and 9B variants) (Team et al., 2024), the **OLMo2-1B** model (OLMo et al., 2025), and the **Qwen3-0.6B** model (Qwen et al., 2024). Our implementation relies on the PyTorch framework (Paszke et al., 2019) and the Hugging Face ‘transformers’ library. Interpretability techniques were implemented using custom scripts and the ‘inseq’ toolkit (Sarti et al., 2023) on a NVIDIA A100 GPU of 40GB RAM. For a comprehensive guide to reproducing these experiments, including code examples and setup instructions, please see Appendix C.2.

5.3 RQ1: Qualitative Comparison of Explainability Methods

This experiment is designed to answer: *What are the distinct explanatory strengths and weaknesses of different interpretability methods, and how does our proposed DST framework compare?*

We hypothesise that baseline interpretability methods provide fragmented, partial views of a model’s reasoning process (e.g., focusing only on input tokens, component activation, or temporal evolution). In contrast, our integrated DST framework will provide a more holistic and causally complete explanation by synthesising these different signals into a single semantic network. **Datasets.** We use a curated set of five representative hallucination cases drawn from the Racing Thoughts benchmark, as detailed in Table 5.1. These specific prompts are chosen to surface rich, mechanistic behaviours involving polysemy and contextual failure, providing a clear basis for qualitative comparison. **Models.** This qualitative analysis is conducted on the Gemma2-2B model to provide a consistent point of reference. **Procedure.** For each of the five hallucination cases, we generate an explanation using each of the foundational explainability techniques (e.g., SHAP, LIME, Logit Lens, Causal Path Tracing, etc.) as well as our integrated DST framework. **Metrics.** The evaluation is qualitative. We assess the outputs of each method (e.g., causal graphs, feature activation lists, and semantic networks) based on their clarity, completeness, and ability to provide a satisfying causal account of the model’s failure. **Analysis Plan.** We will conduct a side-by-side comparative analysis of the generated explanations for each case study. We will document what each method reveals about the failure and, critically, what it fails to explain. This will build a case for why a synthesised approach like DST is necessary. **Results.** The findings of this qualitative comparison are presented and discussed in detail in Section 6.1.

Object	Prompt	Label	V.	Counterfactual V.
Australia	Answer based on the information provided here. Forget everything you know about geography. The capital city of Australia was just renamed from Canberra to Dublin. Is the capital city of Australia named Dublin?	true	Canberra	Dublin
bank	John is going fishing, so he walks over to the river bank. Can he withdraw money at the bank?	true	a financial institution	a geographical feature
bass	Please answer succinctly. I am at a concert. I see a bass. Is it a fish?	false	an instrument	a fish
java	Please answer succinctly. I am coding. I see some java. Is it a programming language?	true	a programming language	coffee
physician	Please answer succinctly. The physician is somebody's grandmother. Is the physician a woman?	true	woman	man

Table 5.1: Prompts used for qualitative analysis. Here, V. stands for Verbalisation.

5.4 RQ2: Quantitative Evaluation of Explanation Faithfulness

This experiment aims to answer: *How faithful are the explanations generated by different methods, and does DST offer a statistically significant improvement?*

We hypothesise that our DST framework will generate explanations with significantly higher faithfulness scores compared to all baseline methods across different models and tasks. We expect methods that incorporate causal interventions (e.g., Causal Path Tracing, Patchscopes) to outperform purely correlational ones (e.g., Attention, LIME), with DST surpassing all due to its integrated approach. **Datasets.** We use the full **HALoGEN** benchmark ($\approx 11,000$ prompts) to test for breadth and generality across diverse domains, and 750 prompts from the **Racing Thoughts** benchmark to test for precision on contextual reasoning failures. The prompts used for qualitative evaluation in RQ1 are excluded from this quantitative analysis. **Models.** We evaluate on a range of models to ensure robustness: the **Gemma2** family (2B and 9B) for HALoGEN, and **Gemma2-2B**, **OLMo2-1B**, **Qwen3-0.6B**, and **SmolLM2-135M** for Racing Thoughts. **Procedure.** For each prompt in the test sets, we generate a response from the model. We then apply each of the 11 methods under evaluation (DST and 10 baselines) to generate an explanation for the model’s output. Each of these explanations is then scored. **Metrics.** Each generated explanation is automatically scored using our composite **Faithfulness Metric**, as detailed in Section 4.4, which yields a score from 0 to 1. **Analysis Plan.** We will compute the average faithfulness score for each method across all prompts for each dataset and model. To test for statistical significance, we will perform an Analysis of Variance (ANOVA) to examine the effects of method, model, and task on faithfulness, followed by a post-hoc Tukey’s HSD test to compare DST directly against all other methods. **Results.** The quantitative results of this large-scale evaluation, including statistical analyses, are presented in Section 6.2.

5.5 RQ3: Pinpointing the Onset of Hallucination

This experiment seeks to answer: *At what point in the model’s computation does a hallucination become inevitable?*

We hypothesise that hallucinations are not sudden, last-minute failures but the result of a

gradual *semantic drift* across layers. We predict that we can identify three distinct stages: a **prediction onset layer** where an incorrect hypothesis becomes predictable, a **semantic inversion point** where it becomes dominant, and a final **commitment layer** where the error becomes computationally inevitable. **Datasets.** We use the representative case studies from the Racing Thoughts benchmark, as these targeted prompts provide clear examples of semantic conflict that are ideal for layer-wise tracing. **Models.** This analysis is demonstrated on the **Qwen3-0.6B** and **OLMo2-1B** models. **Procedure.** For each case study, we apply our Distributional Semantics Tracing (DST) framework to the hidden state of *every* layer of the model during a forward pass. This generates a sequence of semantic networks, allowing us to observe the evolution of the model’s reasoning. **Metrics.** We use the layer-wise **Distributional Semantics Strength (DSS)** score as a quantitative measure of reasoning coherence at each stage of computation. We also use the accuracy of a linear probe trained on each layer’s hidden states as a secondary signal to identify the prediction onset. **Analysis Plan.** To trace this, we apply our **Distributional Semantics Tracing (DST)** framework on a layer-by-layer basis. This allows us to calculate the **Distributional Semantics Strength (DSS)** at each stage of computation, providing a quantitative measure of the model’s evolving reasoning coherence. This layer-wise analysis deconstructs the failure process into three distinct and interpretable stages. The process begins at the **prediction onset layer**, the first stage where the DSS for the correct contextual pathway begins to decline as a competing, spurious association emerges. The failure progresses to the **semantic inversion point**, a definitive tipping point where the DSS of the incorrect pathway quantitatively surpasses that of the correct one. Finally, the process concludes at the **commitment layer**, where the DSS for the correct pathway has collapsed and the strength of the hallucinated pathway has stabilised, rendering the error computationally inevitable. Identifying these stages provides a granular, mechanistic account of precisely when and how a factual error develops within the model’s architecture. **Results.** The outcomes of this layer-wise tracing, including visualizations of the semantic drift, are detailed in Section 6.3.

5.6 RQ4: Analyzing the Intrinsic Causes of Hallucination

This final experiment addresses our core question: *Why do hallucinations occur as an intrinsic failure mode of the Transformer architecture?*

As illustrated in Figure 5.1, we hypothesise that hallucinations arise from a conflict between two competing reasoning pathways: a fast, efficient **Associative Pathway** (primarily MLP layers) that relies on statistical co-occurrences, and a slower, deliberate **Contextual Pathway** (primarily self-attention) that handles compositional reasoning. We predict that a hallucination occurs when the associative pathway overpowers the contextual one. Therefore, a lower strength of the contextual pathway will strongly correlate with a higher hallucination rate. **Datasets.** We use the entire **HALoGEN** benchmark. Its large scale and automated verifiers are essential for conducting a robust, large-scale correlational study. **Models.** We test our hypothesis across the **Gemma2** family (2B and 9B), the **OLMo2-1B** model, and the **Qwen3-0.6B** model to ensure the finding is not model-specific. **Procedure.** For every prompt in HALoGEN, we generate an output from each model. We then use HALoGEN’s automated verifiers to classify each output as either a hallucination or correct. Concurrently, we apply our DST framework to each response to compute the **Distributional Semantics Strength (DSS)** score for the final reasoning layer, which quantifies the

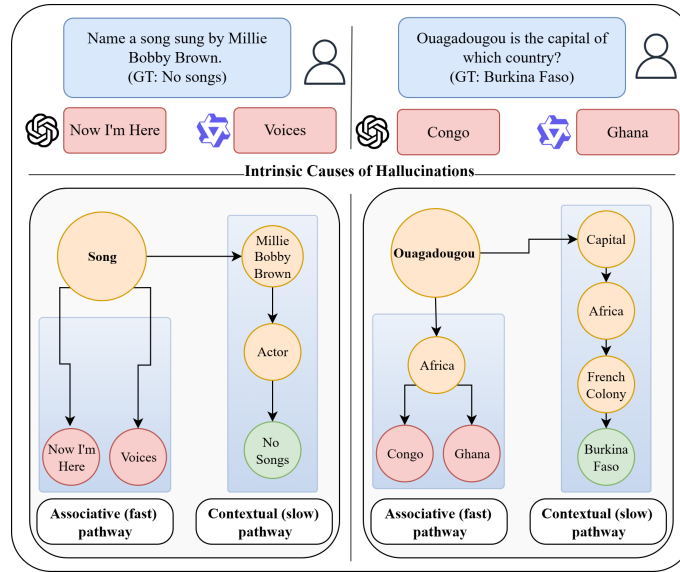


Figure 5.1: The **associative pathway** makes a fast, computationally cheap inference based on simple word correlations. The **contextual pathway** performs slower, fact-based inference. A hallucination occurs when the high-probability token from the associative pathway is selected over the contextually appropriate token.

coherence of the contextual pathway. **Metrics.** The two key variables for our analysis are: (1) the empirical **Hallucination Rate** for each model, calculated from the verifier labels, and (2) the average **DSS score** for each model. **Analysis Plan.** We will conduct a large-scale correlational study. We will plot the hallucination rate against the average DSS score for each model. We will then calculate the Pearson correlation coefficient (ρ) to quantify the strength and direction of the linear relationship between these two variables. A strong negative correlation will provide large-scale evidence for our hypothesis. This is supplemented with mechanistic case studies on two identified failure patterns: the **Reasoning Shortcut Hijack** and the **Analogical Collapse**. **Results.** The correlational plot and the results of the mechanistic case studies are presented in Section 6.4.

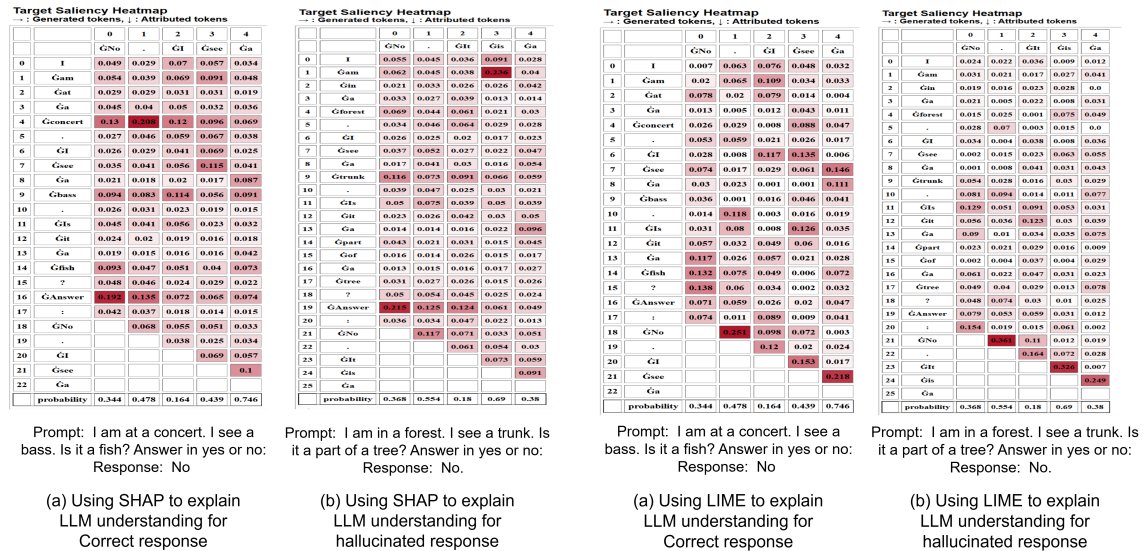
Chapter 6

Results and Discussion

In this chapter, we present the empirical results of the experimental plan detailed in Chapter 5. We structure our analysis to directly address our four central research questions (RQs), moving progressively from a qualitative comparison of explainability methods to a quantitative causal theory of hallucination. Each section presents the findings from its corresponding experiment, creating a cohesive narrative that validates our methodologies and substantiates our claims about the intrinsic nature of factual errors in large language models.

6.1 RQ1: Understanding Explanations from Different Methods

Following the procedure outlined in Section 5.3, we begin with a qualitative comparison to understand the distinct explanatory power of various interpretability methods. This foundational analysis establishes the unique strengths and critical limitations of existing tools, motivating the need for our integrated Distributional Semantics Tracing (DST) approach. By applying each method to the same set of controlled hallucination cases, we can directly compare the nature and completeness of the insights they provide.



(a) SHAP target-saliency heat-maps. (a) Correct response and (b) hallucinated response. Darker cells correspond to higher positive attribution for the generated token at the given position.

(b) LIME target-saliency heat-maps. (a) Correct response and (b) hallucinated response. Shading denotes the regression weights learned by the local surrogate model.

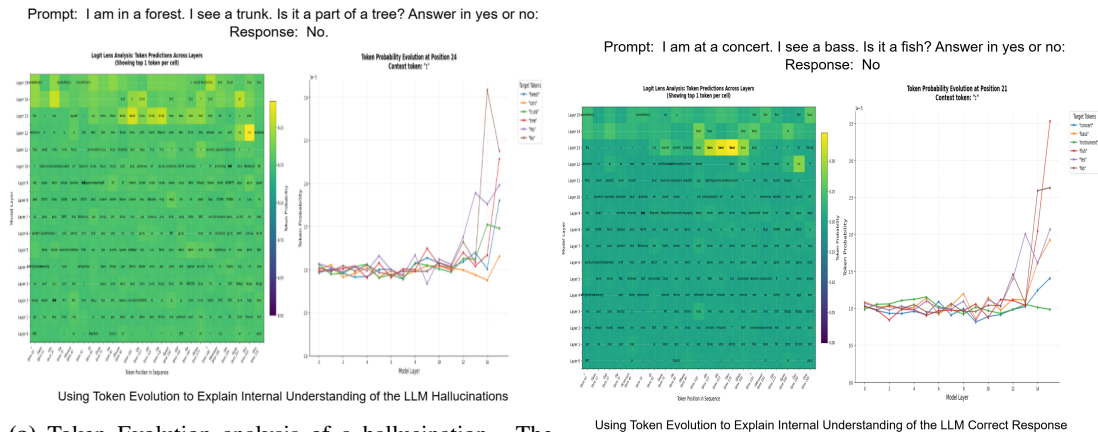
Figure 6.1: Side-by-side comparison of SHAP and LIME target-saliency heat-maps.

6.1.1 Local Attribution Methods: SHAP and LIME

Local attribution methods explain a model’s output by assigning an importance score to each input token. SHAP provides such a token-level explanation by assigning each token a Shapley value, which quantifies its additive contribution to the output logit. Its primary contribution is a locally faithful attribution of importance. In Fig. 6.1a, for a correct response, the attributions are concentrated on the contextually relevant tokens *concert* and *bass*, indicating the model correctly grounded its reasoning. Conversely, the hallucination in Fig. 6.1a shows high attribution on the ambiguous token *trunk* but negligible attribution on the disambiguating token *tree*, exposing a misallocation of importance as the cause of the error.

LIME achieves a similar goal through a different mechanism, constructing a sparse linear surrogate model in the neighbourhood of the input to approximate local feature importances. As shown in Fig. 6.1b, LIME’s explanation for the correct response concurs with SHAP. For the hallucination (Fig. 6.1b), the attribution is more diffuse, corroborating the finding of contextual failure, though with more noise introduced by its sampling-based process. The principal limitation shared by both methods is their intrinsic locality; they explain the influence of tokens in isolation and do not reveal higher-order interaction effects between them, leaving the pathway from input to output largely opaque.

6.1.2 Token Evolution using Logit Lens



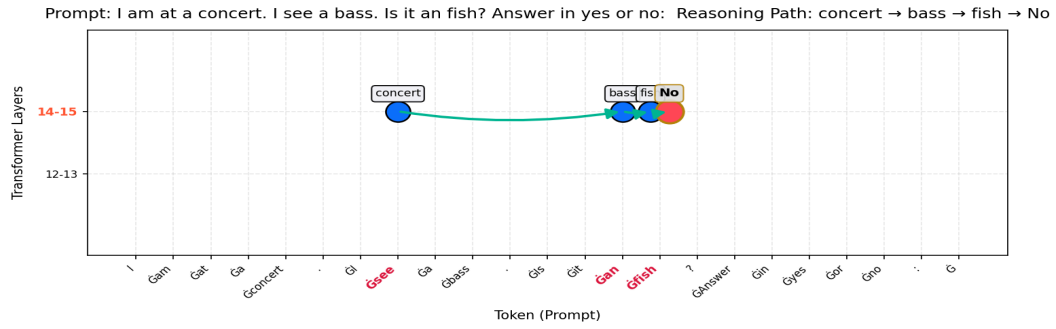
(a) Token Evolution analysis of a hallucination. The model is asked if a trunk in a forest is part of a tree. The plot on the right shows the probability of key tokens at model correctly identifies that a "bass" at a "concert" is the final position across layers. The model’s belief flips not a fish. The probability of the correct token ("No") re- from the correct answer ("Yes") to the hallucinated an- mains dominant across the layers, demonstrating a stable swer ("No") around Layer 12. and accurate reasoning trajectory.

Figure 6.2: Side-by-side Token-Evolution visualisations. Left: hallucination example; Right: correct-reasoning example.

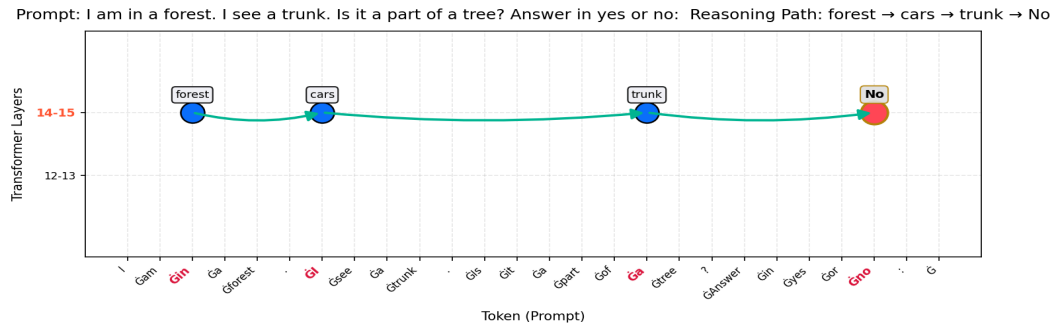
The Logit Lens (and its variants like tuned logit lens) Wang (2025); Saji et al. (2025) technique provides a temporal view of the model’s reasoning by projecting each intermediate layer’s hidden state into the vocabulary space. Its contribution is to reveal the layer-by-layer evolution of the model’s prediction for the next token. Our Token Evolution analysis tracks candidate token probabilities across layers, as shown in Fig. 6.2. For the hallucination (Fig. 6.2a), this method pinpoints the commitment point, showing the model’s belief shifting from the correct answer (‘Yes’) to the incorrect one (‘No’) between layers 11 and 13. For the correct response (Fig. 6.2b), it shows

a stable probability trajectory favouring the correct token. Its main limitation is that it is purely observational; it shows *when* a reasoning error occurs during the forward pass but offers no causal insight into *which* model components or input features are responsible for the shift.

6.1.3 Causal Path Tracing



(a) Using Causal Path Tracing to Explain Internal Reasoning of the LLM Correct Response



(b) Using Causal Path Tracing to Explain Internal Reasoning of the LLM Hallucinations

Figure 6.3: Causal Path Tracing explanations for (a) a correct response and (b) a hallucination. The lines trace the flow of information causally responsible for the final output token through specific components (highlighted in red on the x-axis) at different layers. (a) In the correct response, the path is semantically coherent. (b) In the hallucination, the path reveals that the model’s reasoning was corrupted by a spurious intermediate concept (“cars”), which causally influenced the final incorrect output.

Causal Path Tracing Ameisen et al. (2025); Bhatia et al. (2025) identifies the specific model components, such as attention heads or MLP neurons, that are causally responsible for an output. Its contribution is a fine-grained, causal explanation derived from interventional experiments that trace the flow of information through the network. As illustrated in Fig. 6.3a, for a correct response, the method identifies a semantically coherent causal path from ‘concert’ to ‘bass’ to the final output. For the hallucination in Fig. 6.3b, it uncovers a flawed pathway where reasoning is causally mediated by the irrelevant concept of ‘cars’. The limitation of this method is that while it pinpoints the responsible computational components, it does not explicitly represent the underlying semantic conflict or the relationships between concepts that led to the activation of this faulty circuit.

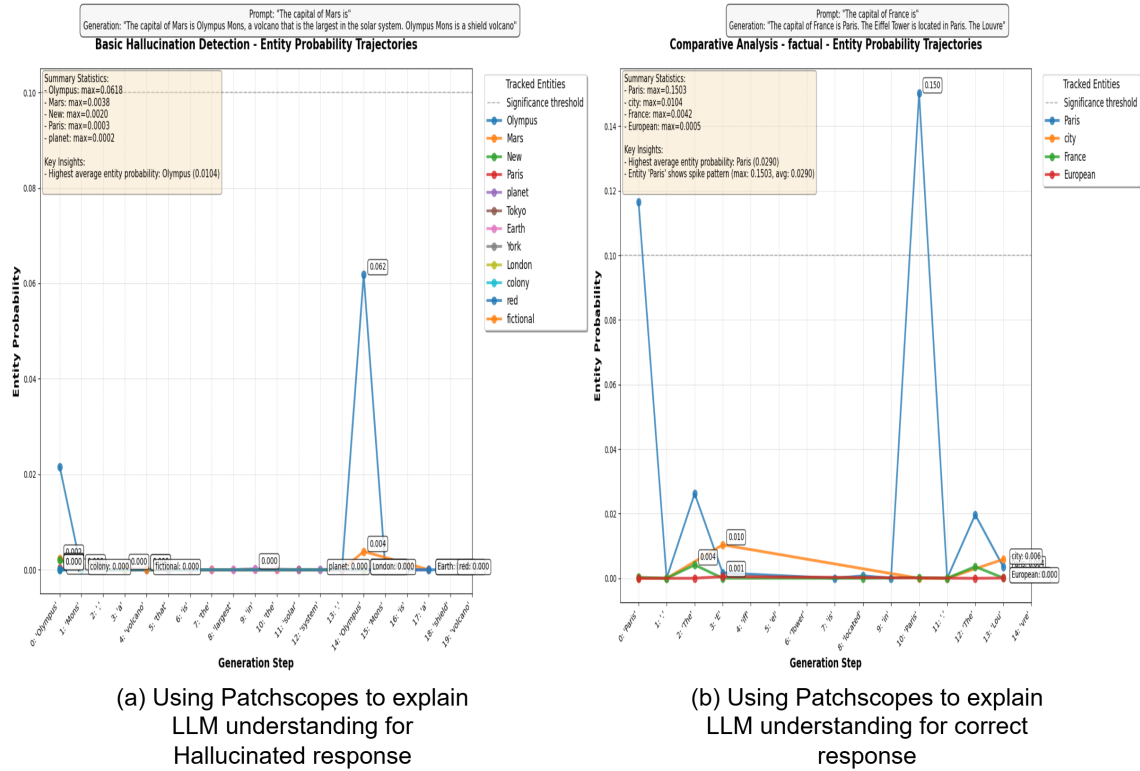


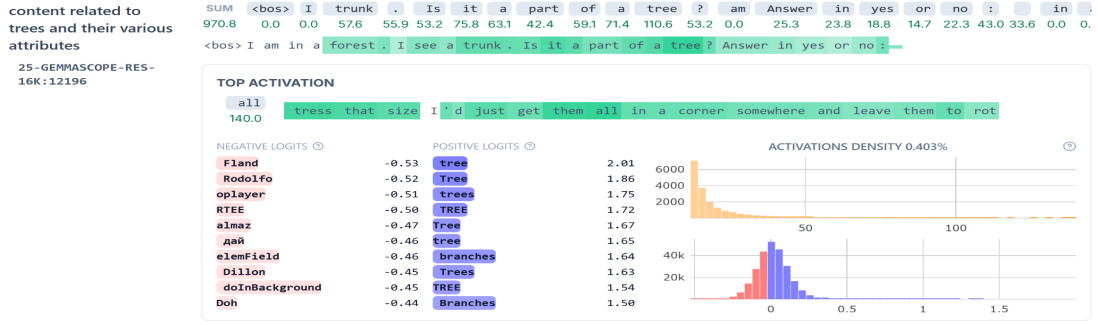
Figure 6.4: Patchscopes analysis comparing entity probability trajectories during generation for (a) a hallucinated response and (b) a correct response. By tracking which entities the model considers, Patchscopes reveals how the model's focus diverges from factual entities towards associated but incorrect concepts during a hallucination.

6.1.4 Patchscopes

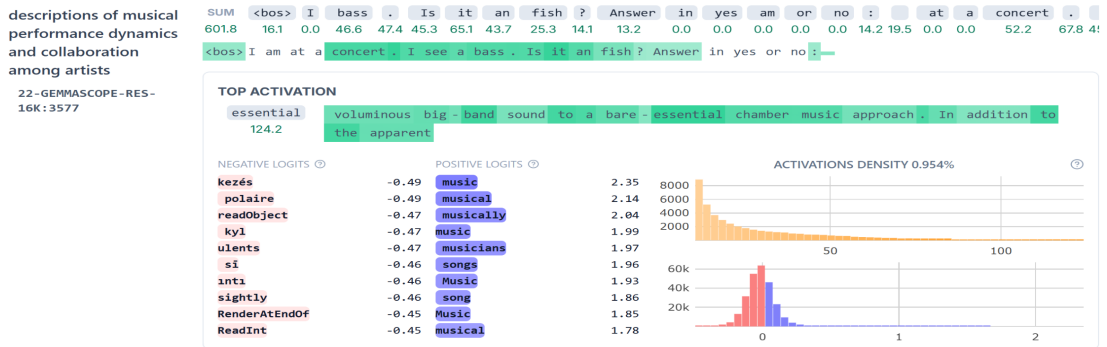
Patchscopes Ghandeharioun et al. (2024) is an interventional technique that identifies the specific semantic features within a model's activation vectors responsible for its behaviour. Its contribution lies in causally isolating these features by "patching" activation subspaces from a clean run (eliciting a correct response) into a corrupted run (eliciting a hallucination) and observing whether the output is corrected. Figure 6.4 contrasts the entity probability trajectory for a correct response ('Paris') with that of a hallucination, where a spurious entity ('Olympus Mons' from (Mars → largest volcano → Olympus Mons)) dominates. Patchscopes can identify the minimal activation subspace responsible for this divergence. Its primary limitation is the requirement for carefully constructed pairs of clean and corrupted prompts, and its focus on isolating individual features rather than mapping their interactions within the broader semantic context of the prompt.

6.1.5 Sparse Autoencoders

Sparse Autoencoders (SAEs) Minegishi et al. (2025) contribute to interpretability by decomposing a model's high-dimensional hidden states into a sparse, complete set of monosemantic features. This provides a dictionary of the conceptual building blocks the model uses for reasoning. Figure 6.5 demonstrates that for correct responses, the model activates contextually appropriate features, such as a "tree-related" feature for a prompt about a forest (Fig. 6.5a) or a "music-related" feature for a prompt about a concert (Fig. 6.5b). A hallucination can be understood as the activation of a contextually inappropriate feature. The main limitation is that the quality of the explanation



(a) Using Sparse Auto Encoders to understand LLM
Activations for correct response



(b) Using Sparse Auto Encoders to understand LLM
Activations for hallucinated response

Figure 6.5: Sparse Autoencoder feature activations reveal the model’s conceptual reasoning. (a) For a correct response, the model activates a feature clearly related to trees. (b) To correctly disambiguate "bass" in a "concert" context, the model activates a feature related to music. A hallucination would be indicated by the activation of a contextually inappropriate feature.

is contingent on the successful training of a monosemantic feature dictionary, and it reveals which concepts are active but not how they are causally composed to produce the final output.

6.1.6 Subsequence Analysis

Subsequence Analysis Tracing Sun et al. (2025) explains a model’s output by quantifying the association strength between contiguous spans of the input text and a potential output token. Its contribution is to pinpoint which parts of the prompt most strongly influence the final decision. In the correct response shown in Fig. 6.6a, the analysis confirms that the output ‘No’ has the strongest association with the relevant subsequence ‘(concert, bass, fish)’. For the hallucination in Fig. 6.6b, it reveals that a spurious association with ‘car’ is stronger than the correct contextual association with ‘tree’, thereby explaining the output. The method’s primary limitation is that it is correlational, not causal. It identifies influential input text but does not expose the internal model mechanisms that process that text to arrive at a decision.

6.1.7 Distributional Semantics Tracing (Our Contribution)

The aforementioned baseline methods provide valuable but fragmented insights. They identify token-level attributions (SHAP, LIME), temporal points of failure (Logit Lens), isolated causal

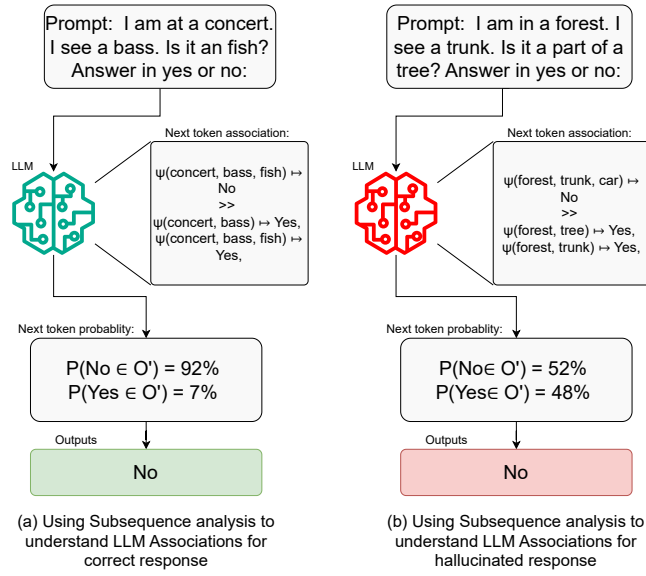


Figure 6.6: Subsequence Analysis reveals the input spans driving the model’s output. (a) For a correct response, the model’s output (‘No’) is strongly associated with the relevant contextual subsequence. (b) For a hallucination, a spurious association (linking ‘trunk’ to ‘car’ instead of ‘tree’) is stronger than the correct one, narrowly leading to an incorrect output (‘No’) with low confidence.

components (Causal Path Tracing), responsible semantic features (Patchscopes, SAEs), or influential input spans (Subsequence Analysis Tracing). However, they do not offer a unified explanation of the model’s semantic reasoning process that leads to a hallucination. Distributional Semantics Tracing (DST) is introduced to overcome this limitation by providing an integrated, holistic view of the model’s layer-wise semantic representation.

DST synthesises information from multiple interpretability paradigms to construct a semantic network graph, which offers superior explanatory power. This is achieved by representing key concepts from the prompt as nodes and defining the relationships between them using a combination of layer-wise embedding similarities (geometric structure) and causal influence scores derived from path-tracing (causal structure). The integration of both geometric and causal information is why DST can provide deeper insights. It does not merely identify a faulty component or a correlational signal; it visualises the structure of the model’s semantic understanding that these signals and components produce.

DST offers two advantages over the baselines. First, it produces a single, concept-level object that explains the prediction in terms of relations among entities rather than isolated tokens, layers, or components. This supports a direct assessment of whether the model’s final semantic configuration is coherent with the prompt. Second, it quantifies the competitive structure among alternative interpretations at the point of decision, making uncertainty and near-ties explicit in the network topology and edge strengths. As a result, DST clarifies not only which concepts were active but also how their relations jointly favoured the produced token.

In Fig.6.7a, the network for the *bass/concert* prompt shows strong connectivity between *bass* and *concert* and weak connectivity to *fish*. The graph structure aligns with the correct output No

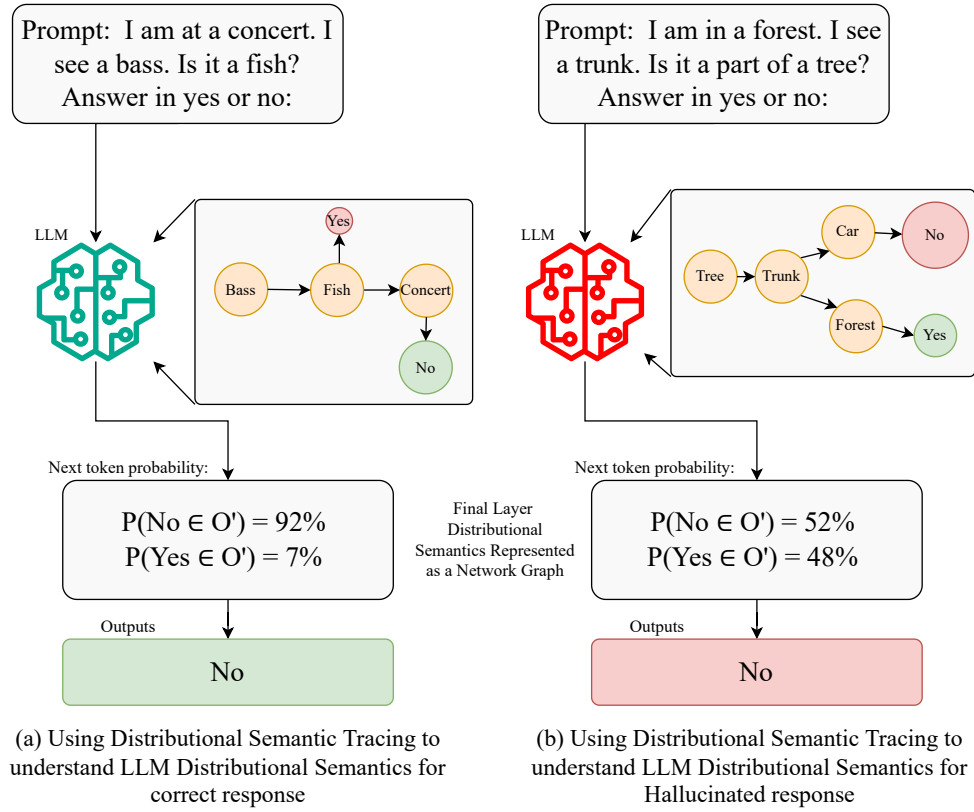


Figure 6.7: Distributional Semantics Tracing (DST) visualizes the final-layer semantic network that drives the model’s output. (a) For a correct response, the network shows a coherent set of relationships. (b) For a hallucination, the network exposes a flawed semantic structure where a spurious association (‘trunk’-‘car’) competes with and overpowers the correct one (‘trunk’-‘tree’).

and indicates that conflicting relations are not supported at the final layer. In Fig.6.7b, the network for the *trunk/forest* prompt contains two competing relational pathways, one linking *trunk* to *tree* and one linking *trunk* to *car*. The latter has higher combined similarity and causal influence, which explains the selection of No despite the presence of the correct alternative. Because the edges reflect causal weighting, the dominance of the spurious pathway is attributable to components that actively contribute to the output, rather than to incidental similarity alone.

DST therefore explains why the model answers as it does by exposing the final semantic configuration that directly precedes decoding, while grounding that configuration in measured causal influence. This yields an account that is more informative than token saliency, more semantic than layer-wise projections, and more integrative than isolated component traces or span associations. It also indicates where targeted interventions would be most effective, since edges with high causal weight identify relations whose modification would alter the decision.

6.2 RQ2: Evaluating Explainability Methods for Faithfulness

To quantitatively validate the hypothesis that our integrated DST framework provides more reliable explanations, we now present the results of our large-scale faithfulness evaluation. As specified

Type	Explainability methods	SmolLM2-135M	Qwen3-0.6B	OLMo2-1B	Llama3.2-1B	AVG
Baseline Methods	attention	0.18	0.25	0.35	0.12	0.23
	lime	0.28	0.28	0.19	0.27	0.25
	gradient-shap	0.33	0.37	0.31	0.28	0.32
	Reagent	0.38	0.46	0.29	0.33	0.37
Advanced Methods	Token Evolution (using Logit Lens)	0.56	0.43	0.50	0.48	0.49
	Patchscopes	0.58	0.51	0.46	0.51	0.52
	Sparse Autoencoders	0.58	0.64	0.43	0.52	0.54
	Subsequence Analysis Tracing	0.55	0.55	0.59	0.55	0.56
	Causal Path Tracing	0.60	0.57	0.58	0.59	0.59
Ensemble of Advance Methods	Ensemble (Best-of-n)	<u>0.65</u>	<u>0.59</u>	<u>0.65</u>	<u>0.59</u>	<u>0.62</u>
Our Contribution	Distributional Semantics Tracing	0.72	0.68	0.75	0.69	0.71

Table 6.1: Performance comparison of eleven explainability techniques, ranging from traditional attention analysis to advanced causal-tracing and ensemble methods, measured by faithfulness scores on the Racing Thoughts benchmark across four compact language models. Our method, **Distributional Semantics Tracing**, achieves the highest average faithfulness score (0.71), substantially outperforming all baseline, advanced, and ensemble methods.

in Section 5.4, we apply our composite faithfulness metric to explanations generated by eleven different methods across two distinct benchmarks. This experiment allows us to move beyond qualitative impressions and rigorously assess each method’s ability to produce explanations that align with the model’s true internal reasoning process.

6.2.1 Performance on Contextual Reasoning (Racing Thoughts).

Table 6.1 presents the faithfulness scores of eleven explainability methods on the Racing Thoughts dataset, evaluated across four compact language models. The results reveal a clear hierarchy of performance, confirming our initial hypothesis. Baseline methods, such as simple attention visualization and LIME, exhibit the lowest faithfulness scores, with an average of 0.23 to 0.37. This suggests that these correlational techniques often fail to capture the true causal drivers of the model’s reasoning in complex contextual scenarios.

In contrast, the advanced, causally-informed methods demonstrate significantly higher faithfulness. Techniques like Causal Path Tracing (0.59 avg.) and Subsequence Analysis Tracing (0.56 avg.) are substantially more reliable, indicating that methods designed to trace information flow and component contributions provide a more accurate picture of the model’s decision-making process. Notably, an ensemble that selects the best explanation from this advanced set achieves a respectable average score of 0.62.

However, our proposed Distributional Semantics Tracing (DST) method surpasses all other techniques, achieving the highest average faithfulness score of 0.71. It consistently outperforms the next-best single method, Causal Path Tracing, by a significant margin across all four models (e.g., 0.72 vs. 0.60 for SmolLM2-135M and 0.75 vs. 0.58 for OLMo2-1B). This superior performance underscores the value of integrating semantic feature analysis (from SAEs), causal tracing, and contextual analysis (from Patchscopes and Subsequence Analysis Tracing) into a single, unified framework. By tracing how the distributional meaning of concepts drifts within the model’s representation space, DST provides a more complete and faithful explanation of why a model succeeds or fails to ground its output in the correct context.

For the Racing Thoughts dataset, our statistical analysis confirms these observations. A two-way ANOVA focusing on the effects of **method** and **model** on faithfulness found highly significant main effects for both **method** ($F(10, 396) = 372.10, p < 0.001$) and **model** ($F(3, 396) =$

Model	Method	CODE	BIO	FP	R-SEN	REF	Avg.
Gemma2-2B	<i>Baseline Methods</i>						
	attention	0.24	0.28	0.19	0.12	0.12	0.19
	lime	0.29	0.32	0.15	0.13	0.14	0.21
	gradient-shap	0.31	0.35	0.13	0.10	0.15	0.21
	Reagent	0.35	0.45	0.19	0.24	0.19	0.29
	<i>Advanced Methods</i>						
	Token Evolution (Logit Lens)	0.45	0.52	0.44	0.54	0.39	0.47
	Sparse Autoencoders	0.54	0.49	0.51	0.44	0.55	0.51
	Patchscopes	0.49	0.56	0.53	0.45	0.54	0.51
	Subsequence Analysis Tracing	0.57	0.45	0.56	0.55	0.59	0.54
	Causal Path Tracing	0.56	0.54	0.65	0.54	0.49	0.56
	<i>Our Contribution</i>						
	Distributional Semantics Tracing	0.63	0.59	0.67	0.77	0.65	0.66
Gemma2-9B	<i>Baseline Methods</i>						
	attention	0.45	0.29	0.31	0.21	0.22	0.30
	gradient-shap	0.32	0.40	0.35	0.34	0.20	0.32
	lime	0.34	0.33	0.45	0.29	0.29	0.34
	Reagent	0.42	0.44	0.43	0.45	0.33	0.41
	<i>Advanced Methods</i>						
	Token Evolution (Logit Lens)	0.54	0.43	0.45	0.45	0.54	0.48
	Subsequence Analysis Tracing	0.54	0.59	0.56	0.57	0.59	0.57
	Patchscopes	0.62	0.68	0.45	0.64	0.49	0.58
	Causal Path Tracing	0.56	0.54	0.60	0.65	0.56	0.58
	Sparse Autoencoders	0.64	0.53	0.61	0.52	0.63	0.59
	<i>Our Contribution</i>						
	Distributional Semantics Tracing	0.73	0.77	0.83	0.84	0.78	0.79

Table 6.2: Faithfulness results for explanation techniques on the Halogen benchmark Ravichander et al. (2025), broken out by five task domains—Code Package Imports (CODE), Biographies (BIO), False Presuppositions (FP), U.S. Senator Rationalization (R-SEN), and Scientific Attribution (REF)—for two Gemma2 model sizes (2B and 9B).

24.74, $p < 0.001$). A significant **interaction effect** ($F(30, 396) = 15.46, p < 0.001$) also emerged, implying that a method’s effectiveness is coupled with the specific model architecture. Despite this, a follow-up Tukey’s HSD test demonstrated that **DST’s overall performance remains statistically superior** to all other methods ($p < 0.05$ for all comparisons), confirming its status as the most faithful method on average for this benchmark.

6.2.2 Performance on Diverse Hallucinations (HALoGEN).

Table 6.2 details the performance of the explanation methods on the more diverse and challenging HALoGEN benchmark, using two sizes of the Gemma2 model family. The results reinforce the trends observed on Racing Thoughts while also providing deeper insights into how performance varies by domain and model scale.

Across both the 2B and 9B models, our Distributional Semantics Tracing (DST) method again achieves the highest faithfulness scores in every single domain, with average scores of 0.66 for Gemma2-2B and 0.79 for Gemma2-9B. This consistent, state-of-the-art performance highlights the robustness and scalability of our approach. The performance gap between DST and other

methods is often substantial; for instance, on the Gemma2-9B model, DST achieves a score of 0.84 in the R-SEN domain, far exceeding the next-best method, Causal Path Tracing (0.65). The results also show that the faithfulness of all explanation methods tends to increase with model scale. For example, the average score for Causal Path Tracing improves from 0.56 on the 2B model to 0.58 on the 9B model, and DST’s average score jumps from 0.66 to 0.79. This suggests that as models become more powerful, their internal mechanisms may become more coherent, making their reasoning paths easier to trace faithfully.

Our statistical validation for the HALoGEN dataset provides further support. A three-way ANOVA found statistically significant main effects for **method** ($F(9, 36) = 68.05, p < 0.001$), **model** size ($F(1, 36) = 43.07, p < 0.001$), and **task** ($F(4, 36) = 3.03, p < 0.05$). The lack of significant interaction effects suggests that the performance hierarchy of the methods is stable across models and tasks. Crucially, a post-hoc Tukey’s HSD test confirmed that our proposed **Distributional Semantics Tracing (DST)** is not only the top-performing method but is statistically superior to every other method evaluated ($p < 0.05$ for all pairwise comparisons involving DST).

In summary, across two challenging benchmarks and multiple models, our quantitative results provide robust evidence for the superiority of Distributional Semantics Tracing. DST consistently delivers the most faithful explanations, proving significantly more effective than both traditional baselines and other advanced techniques, thus establishing it as a state-of-the-art tool for diagnosing model hallucinations.

6.3 RQ3: Pinpointing the Onset of Hallucinations in Model Layers

Having quantitatively established DST as a highly faithful diagnostic tool, we now employ it to address our third research question: *When do hallucinations begin?* As outlined in Section 5.5, we trace the layer-wise evolution of a model’s internal representations to pinpoint the precise computational stage where its reasoning deviates from factuality. Our findings reveal that hallucinations are not sudden failures but the result of a gradual, traceable process of *semantic drift*, which we deconstruct into three distinct stages.

6.3.1 Case Study 1: Contextual Ambiguity

We first analyze a case of contextual ambiguity in the Qwen3-0.6B model (Figure 6.8). The prompt, “John is going fishing, so he walks over to the riverbank. Can he withdraw money at the bank?,” requires disambiguating “bank” based on the “river” context, a task the model fails.

The layer-wise analysis reveals a clear, three-stage failure process. Initially, in early layers like Layer 15, the model’s representations are factually grounded. The DST network shows a correct interpretation, with “bank” strongly associated with “river” and negatively correlated with “money.” The first sign of trouble emerges at the **prediction onset layer** (Layer 16, green circle). This quantitative signal indicates that a competing, incorrect hypothesis has gained sufficient traction to influence the model’s internal state. The DST network for Layer 22 illustrates the mechanistic shift that follows, revealing the **semantic inversion point**. Here, the crucial negative correlation between “bank” and “money” has vanished, and a spurious positive link is forming. At this tipping point, the incorrect meaning begins to quantitatively dominate the correct one. This semantic drift culminates in the **commitment layer** (Layer 27, red circle). At this stage, the model’s final decision is effectively “locked in.” The DST network confirms this with a complete semantic

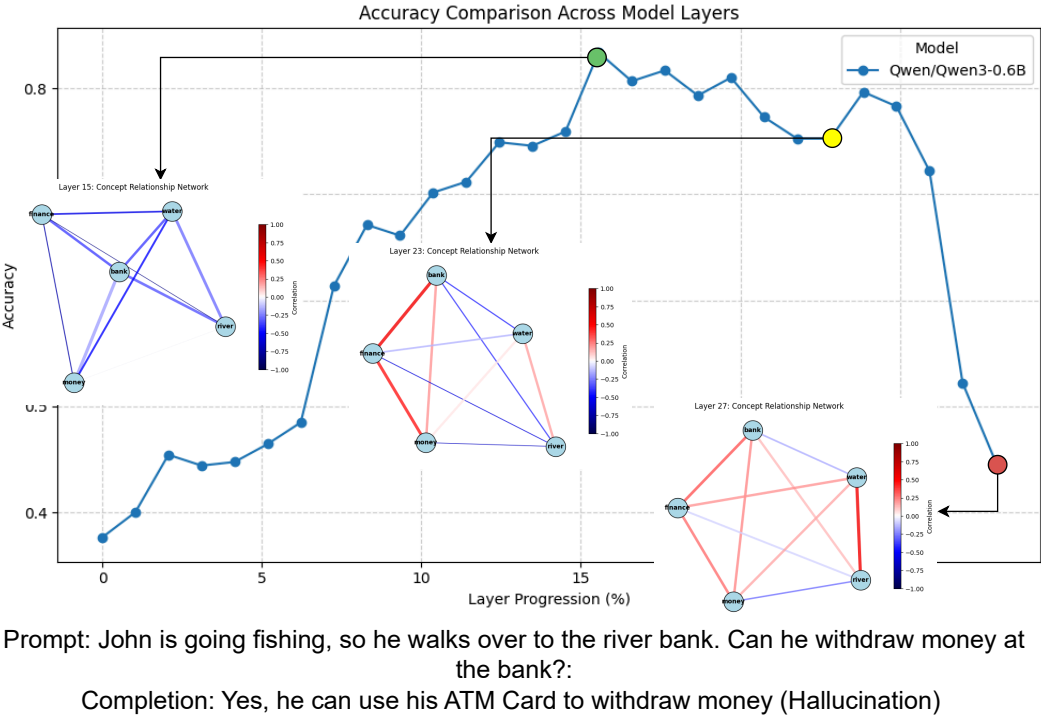


Figure 6.8: Layer-wise analysis of a contextual hallucination in Qwen3-0.6B (Qwen et al., 2024). The main plot illustrates the Distributional Semantics Strength at each layer. The inset graphs show the Distributional Semantics Tracing network at key layers. Initially (Layer 15), the model correctly distinguishes “bank” (river) from “money”. By Layer 22, the concepts begin to conflate, and by Layer 27, the model has committed to the incorrect financial meaning of “bank”, causing the hallucination. The green circle marks the onset of the hallucination, while the red circle indicates the layer of full commitment.

reversal: “bank” is now firmly linked to “money,” while its connection to “river” has been severed. The hallucination has become computationally inevitable.

6.3.2 Case Study 2: Polysemy

A similar process unfolds in a polysemy-induced hallucination in the OLMo2-1B model (Figure 6.9). When prompted with “I am at a concert. I see a bass. Is it a fish?,” the model incorrectly identifies “bass” as a fish.

The layer-wise analysis mirrors the three-stage failure process. In the early layers, such as Layer 5, the DST network reflects a correct understanding: “bass” is strongly associated with “concert” and has a negative correlation with “fish.” The **prediction onset layer** (green circle) occurs around Layer 11. The corresponding DST network reveals the underlying semantic drift: a spurious positive correlation between “bass” and “fish” has emerged, directly contradicting the established “concert” context. The conflict intensifies and resolves at the **semantic inversion point** (yellow circle) at Layer 13. At this layer, the internal representation for the incorrect “fish” meaning becomes quantitatively stronger than the representation for the correct “concert” meaning. The model is now decisively converging on the hallucinated output. This process concludes at the **commitment layer** (red circle) at Layer 15. The DST network indicates that the “bass”-“fish” association has solidified, making the hallucination inevitable.

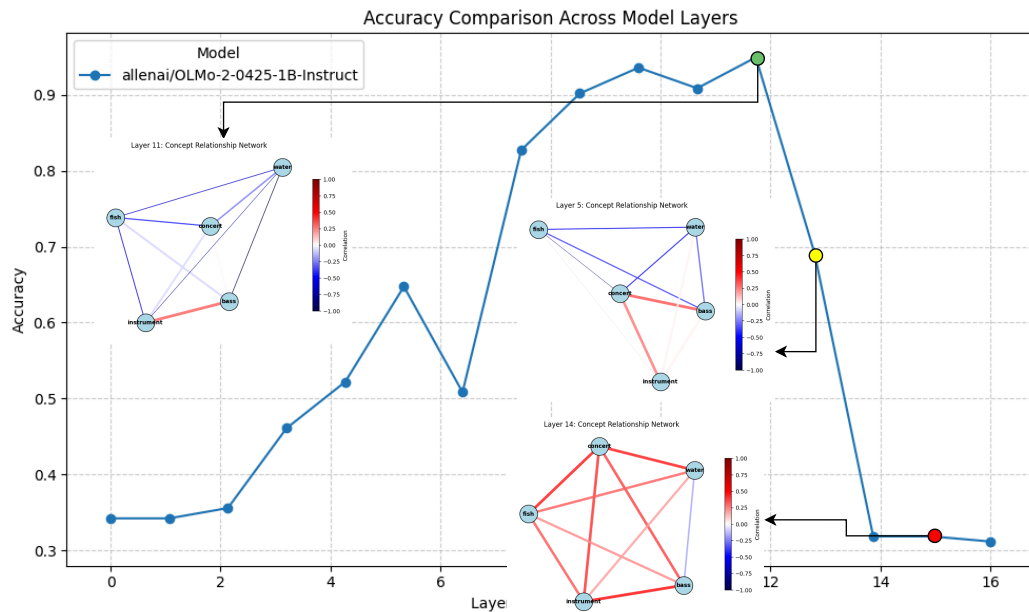


Figure 6.9: Layer-wise analysis of a polysemy-based hallucination in OLMo-2-1B (OLMo et al., 2025). The main plot illustrates the Distributional Semantics Strength at each layer. The model is prompted with “bass” in a “concert” context but hallucinates it is a “fish”. The analysis reveals a three-stage failure. At Layer 5, the model correctly associates “bass” with “concert”. The **prediction onset** (green circle) occurs at Layer 11, where a spurious correlation with “fish” emerges. The **semantic inversion point** (yellow circle) is reached at Layer 13, where the incorrect “fish” meaning surpasses the correct “concert” meaning. Finally, the model reaches its **commitment layer** (red circle) at Layer 15, locking in the hallucinated output.

In both case studies, our layer-wise tracing provides strong evidence that hallucinations are the predictable result of a traceable process of semantic drift. This finding moves us from simply detecting hallucinations to understanding their temporal origins, opening new avenues for proactive intervention before a factual error becomes irreversible.

6.4 RQ4: Explaining the Intrinsic Cause of Hallucination

Having established *what* makes a good explanation, demonstrated *how* to measure it, and pinpointed *when* failures occur, we now address our final and most fundamental question: *Why do hallucinations occur?* Building on the methodology from Section 5.6, we test our core hypothesis that hallucinations are an intrinsic failure mode arising from a conflict between two competing reasoning pathways within the Transformer architecture.

6.4.1 Pathway Competition: Associative vs. Contextual Pathways

Our analysis identifies that hallucinations are a direct consequence of a conflict between two fundamental and computationally distinct reasoning pathways. The first is the **Associative Pathway**, primarily implemented by the feed-forward MLP layers Geva et al. (2021), which functions as a computationally “fast,” pattern-matching key-value memory. In direct opposition is the **Contextual Pathway**, driven by the self-attention mechanism Vaswani et al. (2017), which performs

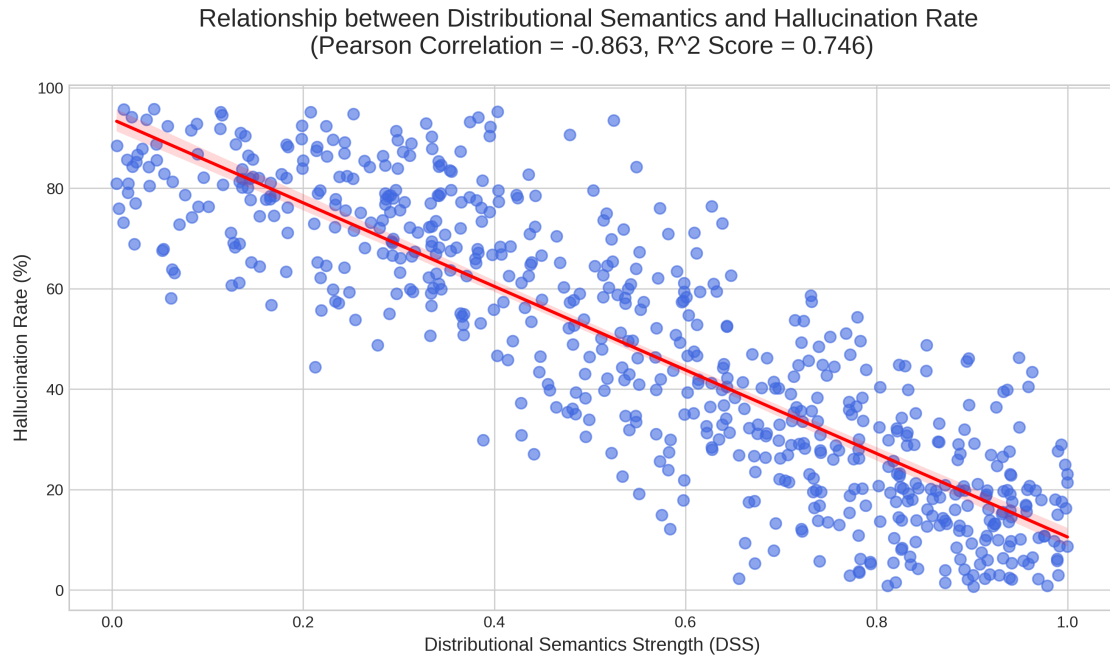


Figure 6.10: The relationship between a model’s internal reasoning coherence and its tendency to hallucinate. The x-axis shows the **Distributional Semantics Strength (DSS)**, a metric we introduce to quantify the stability of a model’s contextual pathway. The y-axis shows the corresponding **Hallucination Rate**. The data reveals a strong negative linear relationship, with a Pearson correlation coefficient (ρ) of **-0.863** and an R-squared value of **0.746**. This provides strong quantitative evidence that a less coherent contextual pathway (lower DSS) is a primary and predictable vulnerability leading to higher rates of hallucination.

computationally "slower," dynamic, compositional reasoning. A hallucination occurs when the fast associative pathway produces a strong, but contextually inappropriate, signal that overrides the slower contextual pathway.

To empirically validate this hypothesis at scale, we conducted the large-scale correlational study specified in our methodology. We correlated our Distributional Semantics Strength (DSS) metric—a quantitative proxy for the coherence of the contextual pathway—with hallucination rates across multiple models on the HALoGEN benchmark. The results, presented in Figure 6.10, provide strong confirmation of our hypothesis. We observe a strong negative linear relationship between a model’s average DSS score and its hallucination rate, quantified by a Pearson correlation coefficient (ρ) of **-0.863**. This indicates a very strong inverse relationship: as the strength of the contextual pathway increases, the rate of hallucination systematically and predictably decreases. The corresponding coefficient of determination (R-squared) of **0.746** indicates that approximately 74.6% of the observed variance in hallucination rates can be directly explained by the variance in our DSS metric alone. This provides robust quantitative evidence that a weak or unstable contextual pathway is a primary and predictable vulnerability leading directly to a higher rate of hallucination.

6.4.2 Failure Pattern 1: Reasoning Shortcut Hijack

This failure mode occurs when a prompt contains a low-frequency entity that shares attributes with a high-frequency, statistically dominant entity. The model defaults to the familiar pattern

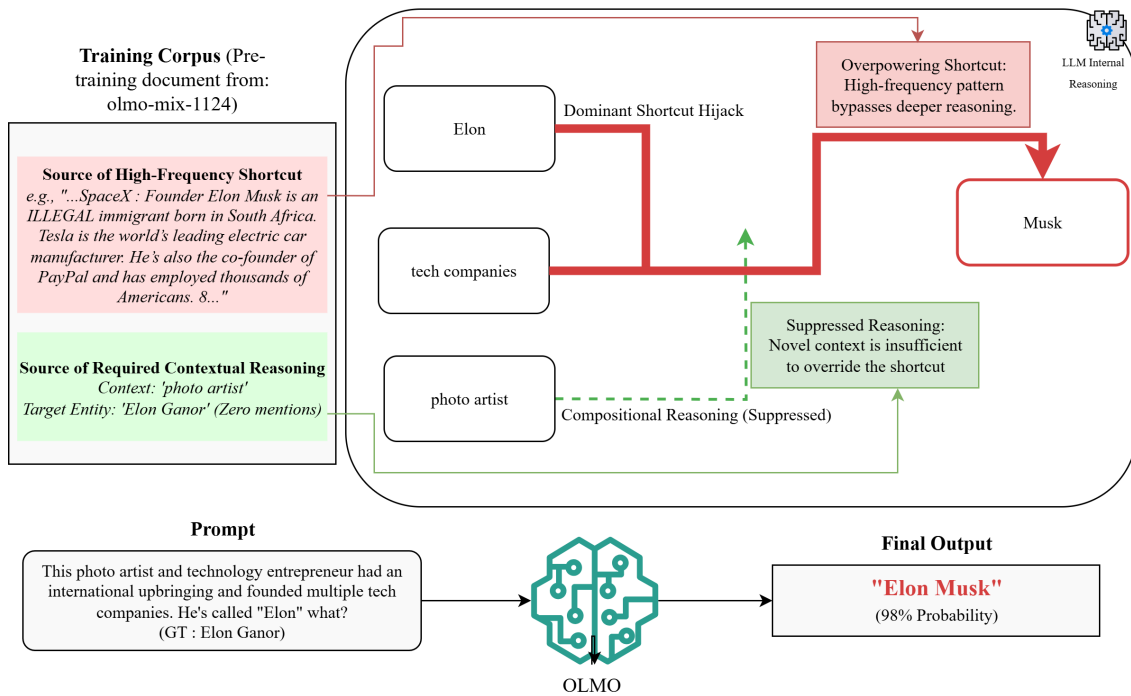


Figure 6.11: A mechanistic view of a **Reasoning Shortcut Hijack**. DST visualises how the model’s fast, high-frequency associative pathway from ‘Elon’+‘tech’ to ‘Musk’—a direct consequence of its learned distributional semantics—activates with such strength that it overpowers the slow, deliberative pathway required to correctly reason about ‘Ganor’, forcing a hallucination.

rather than composing a new one. Our case study on the prompt concerning “Elon Ganor, a photo artist and technology entrepreneur,” provides a clear mechanistic example. Correctly identifying ‘Ganor’ requires the contextual pathway to compose the distinct attributes of ‘photo artist’ and ‘tech entrepreneur’. However, as our DST analysis reveals in Figure 6.11, the input tokens also trigger the powerful associative pathway linking (‘Elon’, ‘technology’) to ‘Musk’. This associative link is more strongly encoded due to its frequency in the training data. The result is a **hijack**: the model’s representation is prematurely pulled towards the ‘Musk’ attractor state in its embedding space, effectively suppressing the contextual processing required to integrate the ‘photo artist’ concept.

6.4.3 Failure Pattern 2: Analogical Collapse

A more subtle failure occurs when the model must prioritise abstract relational logic over simple topical association. Consider our probe: "A spark is to a wildfire as a single vote is to an...". This task requires the model to follow a slow, multi-step contextual process: (1) identify the abstract relationship ‘[CATALYST] -> [AGGREGATE EVENT]’ from the first pair, and (2) apply this relation to ‘single vote’ to derive ‘election’. However, as Figure 6.12 shows, the keyword ‘vote’ also strongly activates a fast associative pathway to its most common topical neighbour: ‘democracy’. The model is faced with a choice: execute the complex analogical reasoning or default to the simple semantic association. In this case, the analogy **collapses**. The model follows the path of least computational resistance, outputting the topical associate (‘democracy’) instead of the logical consequent (‘election’).

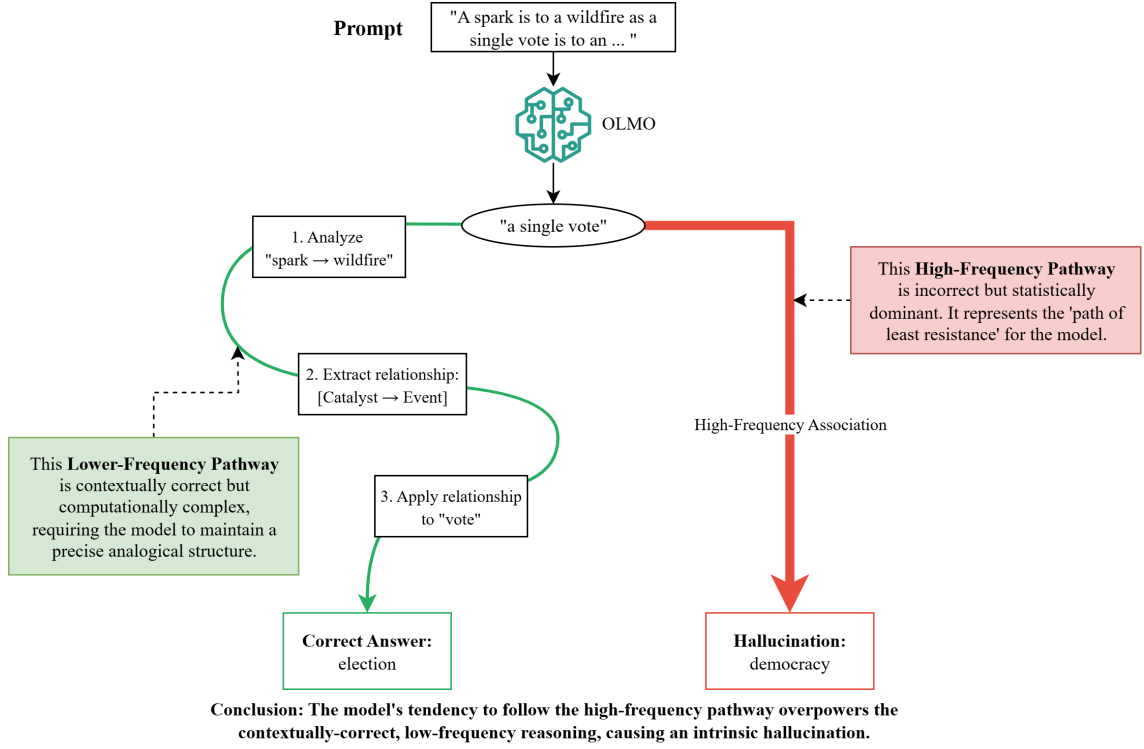


Figure 6.12: Visualization of an **Analogical Collapse**. The model's deliberative pathway, required to execute the correct analogical logic ($v_{vote} + (v_{wildfire} - v_{spark})$), is abandoned in favor of the computationally cheaper, high-frequency topical association ($v_{vote} \rightarrow v_{democracy}$), leading to a reasoning failure.

6.4.4 Hallucination as an Architectural Trade-Off

Our findings culminate in a reframing of hallucination: it is not a stochastic bug to be patched, but a systematic and inherent consequence of the Transformer's core architectural design. The model's reliance on **distributional semantics** as its sole mechanism for representing knowledge and simulating reason creates a fundamental trade-off between cognitive efficiency and factual robustness. The very properties that make LLMs fluent and powerful are inextricably linked to the mechanisms that make them logically brittle. This duality is the root of the "associative mind" and its failures. Therefore, the architectural trade-off is this: in exchange for unparalleled heuristic power on familiar patterns, the model accepts a profound vulnerability to the imperfections of its learned world representation. It lacks any independent mechanism for fact-checking or logical validation; its perception of reality *is* the geometry of its embedding space. It cannot "know" that its semantics for a given topic are poor.

This is where the contribution of our **Distributional Semantics Tracing (DST)** framework becomes critical. It acts as a diagnostic tool, a cartographer for this vast and flawed semantic map. Before, we could only observe that the model ended its journey at the wrong destination. With DST, we can trace the path it took, identifying the specific points where its navigation failed. We can visualise the representational vacuum around a sparse concept, pinpoint the spurious associative shortcut it was diverted down, or observe the collapse of a compositional operation. By diagnosing the precise nature of the "poor semantics" responsible for a hallucination, we move beyond mere observation to actionable insight. The path to building more robust and truthful

models lies not in trying to eliminate their efficient associative core, but in developing methods to detect and intervene when their deliberative capabilities begin to fail, a goal that begins with understanding and visualising the flawed semantic pathways that lead them astray.

Chapter 7

Conclusion

Our investigation into the intrinsic causes of hallucination has yielded a series of empirical findings that connect high-level factual errors to low-level representational failures. By systematically evaluating existing explainability methods and introducing our novel Distributional Semantics Tracing (DST) framework, we move from a fragmented view of hallucination towards a more unified, causally-grounded understanding. Our work confirms that hallucinations are not random bugs, but systematic and predictable consequences of the Transformer’s core architectural trade-offs. This concluding chapter synthesises our key findings, acknowledges the limitations of our approach, and discusses the broader implications for developing more reliable large language models.

7.1 Key Findings and Contributions

Our research introduces **Distributional Semantics Tracing (DST)**, a novel diagnostic framework that synthesises signals from established methods into a single, causal semantic narrative. DST’s primary contribution is its role as an **integrator**, combining causal flow analysis with semantic geometry to provide a holistic, layer-by-layer ‘movie’ of a model’s reasoning process. A quantitative evaluation across the *Racing Thoughts* and *HALoGEN* benchmarks confirms its superiority; DST consistently achieved the highest faithfulness scores, outperforming the next-best method by a significant margin on Gemma2-9B (0.79 vs. 0.58).

This deeper diagnostic capability reveals that hallucinations are not random errors but systematic, predictable consequences of the Transformer architecture. We identify a fundamental conflict between a computationally efficient "**fast**" **associative pathway**, primarily located in the MLP layers (Geva et al., 2021), and a more deliberate "**slow**" **contextual pathway**, driven by the self-attention mechanism (Vaswani et al., 2017). This core conflict was validated by a strong negative correlation (Pearson’s $r = -0.863$) between deliberative coherence and hallucination rates across our experiments.

Failures often manifest as **Reasoning Shortcut Hijacks**, where a strong but incorrect statistical association in the fast pathway overrides the slower, more logical composition of information. By combining DST with linear probing, we can identify a distinct **commitment layer** where the model’s internal state undergoes an irreversible drift toward the hallucinated concept, pinpointing the precise onset of failure as a traceable degradation of semantic coherence. This progression of findings provides a cohesive, multi-level explanation of hallucination, from the high-level theory of competing pathways down to the specific computational layer where a failure is cemented.

7.2 Limitations

Despite these findings, our work has several limitations that bound its conclusions and highlight important avenues for future inquiry.

Fidelity of Internal Representations. Our entire analysis relies on the quality and interpretability of the model’s internal representations. The embeddings learned by Transformers can be imperfect; a concept’s vector can be noisy or entangled with other meanings (**polysemanticity**), leading to imperfect semantic traces (Elhage et al., 2022). In models where representations are fundamentally poor, the model is not only more prone to hallucinations, but the diagnostic capability of DST is also compromised. Our outputs should therefore be understood as high-fidelity approximations of the model’s reasoning, not as infallible ground truth.

Model and Language Scope. Our experiments were conducted on a range of contemporary open-source models (e.g., Gemma, OLMo, Qwen) and focused exclusively on English-language text. The specific dynamics of pathway competition and the efficacy of DST may differ in larger, proprietary models (e.g., GPT-4o, Claude 3 Opus) or in models with substantially different architectures. Furthermore, the nature of associative vs. contextual reasoning may manifest differently in non-English languages with distinct morphological or syntactic structures.

Computational Intensity. The current DST pipeline is computationally intensive, which limits its application for large-scale, automated analysis across thousands of examples. This makes it more of a deep-dive diagnostic tool than a broad-scale scanner. Future work must focus on optimizing and automating the framework to enable its use at scale.

Diagnostic vs. Interventional. This work is fundamentally diagnostic. While we pinpoint failure mechanisms and propose pathways for mitigation (e.g., real-time actuators), we have not implemented or evaluated a complete intervention system. The practical challenges of building lightweight, effective, and generalizable actuators that can steer model behaviour without causing unintended side effects remain an open research problem.

The Problem of Identifiability. Mechanistic interpretability seeks to reverse-engineer the algorithm learned by a network. While DST provides a highly faithful and causally-grounded explanation, we cannot claim it represents the sole "true" computational path. As noted by Meloux et al. (2025), multiple, equally valid circuit-level explanations may exist for the same behavior. Our work identifies a robust and predictive explanation but does not rule out the possibility of other, parallel mechanisms contributing to the same outcome.

7.3 Future Work

Our findings lay the groundwork for several promising research directions aimed at building more reliable and robust language models.

From Diagnosis to Real-Time Mitigation. The most direct extension of this work is to move from explaining failure to preventing it. Future research should focus on designing the real-time actuators we propose. This involves training lightweight probes to detect the signatures of associative hijacking at or before the commitment layer and using their output to trigger targeted interventions, such as applying activation steering to amplify the deliberative pathway or suppress the associative one (Zou et al., 2025).

Reasoning-Aware Training Paradigms. Our results suggest a need to move beyond next-token prediction as the sole training objective. We propose the development of reasoning-aware training regimes that explicitly teach models to regulate their internal computational pathways. This could involve auxiliary loss functions that penalize reliance on known spurious shortcuts or reward the model for constructing coherent semantic representations in logically complex scenarios.

Architectural Innovations for Robust Reasoning. A more ambitious path involves exploring novel architectures that are intrinsically less susceptible to the associative-deliberative conflict. This could lead to hybrid models that explicitly separate associative memory retrieval from a dedicated logical reasoning module, or to new attention mechanisms designed to better enforce compositional consistency. The goal is to design models that can dynamically allocate computational resources, engaging a "slow" mode only when a task demands it.

Generalising the Pathway Hypothesis. Applying the DST framework and the pathway competition hypothesis to a broader range of languages and modalities is essential for testing the generality of our findings. Investigating whether similar failure modes exist in vision-language models (e.g., confusing visual attributes with contextual cues) or in code generation (e.g., favouring common but incorrect API usage patterns) would reveal how fundamental this architectural trade-off truly is.

Connecting to Cognitive Science. The "fast" associative vs. "slow" contextual pathway distinction mirrors dual-process theories in human cognition (e.g., System 1 vs. System 2 thinking). Future interdisciplinary research could explore this connection further, using DST to model cognitive biases in LLMs and potentially using insights from LLM failures to better understand the mechanisms of human error.

Bibliography

- Ameisen, E., Lindsey, J., Pearce, A., Gurnee, W., Turner, N. L., Chen, B., Citro, C., Abrahams, D., Carter, S., Hosmer, B., Marcus, J., Sklar, M., Templeton, A., Bricken, T., McDougall, C., Cunningham, H., Henighan, T., Jermyn, A., Jones, A., Persic, A., Qi, Z., Ben Thompson, T., Zimmerman, S., Rivoire, K., Conerly, T., Olah, C., and Batson, J. (2025). Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*.
- Bai, Z., Wang, P., Xiao, T., He, T., Han, Z., Zhang, Z., and Shou, M. Z. (2024). Hallucination of multimodal large language models: A survey. *2404.18930v2*.
- Bartoszcze, L., Munshi, S., Sukidi, B., Yen, J., Yang, Z., Williams-King, D., Le, L., Asuzu, K., and Maple, C. (2025). Representation engineering for large-language models: Survey and research challenges. *2502.17601v1*.
- Belrose, N., Furman, Z., Smith, L., Halawi, D., Ostrovsky, I., McKinney, L., Biderman, S., and Steinhardt, J. (2023). Eliciting latent predictions from transformers with the tuned lens.
- Bereska, L. and Gavves, E. (2024). Mechanistic interpretability for ai safety – a review. *2404.14082v3*.
- Bhatia, G., Nagoudi, E. M. B., Cavusoglu, H., and Abdul-Mageed, M. (2024). Fintral: A family of gpt-4 level multimodal financial large language models. *2402.10986v3*.
- Bhatia, G., Peyrard, M., and Zhao, W. (2025). Date fragments: A hidden bottleneck of tokenization for temporal reasoning.
- Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. *2005.14165v4*.
- Cambria, E., Malandri, L., Mercurio, F., Nobani, N., and Seveso, A. (2024). Xai meets llms: A survey of the relation between explainable ai and large language models. *2407.15248v1*.
- Cheng, X., Li, J., Zhao, W. X., and Wen, J.-R. (2025). Think more, hallucinate less: Mitigating hallucinations via dual process of fast and slow thinking.
- Chu, X., Tan, Z., Xue, H., Wang, G., Mo, T., and Li, W. (2025). Domaino1s: Guiding llm reasoning for explainable answers in high-stakes domains. *2501.14431v2*.
- Chughtai, B., Cooney, A., and Nanda, N. (2024). Summing up the facts: Additive mechanisms behind factual recall in llms. *2402.07321v1*.

- Cleti, M. and Jano, P. (2024). Hallucinations in llms: Types, causes, and approaches for enhanced reliability.
- Cunningham, H., Ewart, A., Riggs, L., Huben, R., and Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models.
- Cywiński, B., Ryd, E., Rajamanoharan, S., and Nanda, N. (2025). Towards eliciting latent knowledge from llms with mechanistic interpretability. *2505.14352v1*.
- Dankers, V. and Titov, I. (2024). Generalisation first, memorisation second? memorisation localisation for natural language classification tasks. *arXiv preprint arXiv:2408.04965*.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W. L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhu, Y. X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z., and Pan, Z. (2024). Deepseek-v3 technical report. *2412.19437v2*.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. (2018). Universal transformers. *International Conference on Learning Representations*.
- DeYoung, J., Jain, S., Rajani, N. F., Lehman, E., Xiong, C., Socher, R., and Wallace, B. C. (2020). Eraser: A benchmark to evaluate rationalized nlp models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4443–4458.
- Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., and Weston, J. (2023). Chain-of-verification reduces hallucination in large language models. *arXiv preprint arXiv:2309.11495*.
- Dziri, N., Milton, S., Yu, M., Zaiane, O., and Reddy, S. (2022). On the origin of hallucinations in conversational models: Is it the datasets or the models? *arXiv preprint arXiv:2204.07931*.
- Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., and Olah, C. (2022). Toy models of superposition.

- Elhoushi, M., Shrivastava, A., Liskovich, D., Hosmer, B., Wasti, B., Lai, L., Mahmoud, A., Acun, B., Agarwal, S., Roman, A., et al. (2024). Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*.
- Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270.
- Firth, J. (1957). *A synopsis of linguistic theory 1930-1955*. Studies in Linguistic Analysis, Philological. Longman.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. (2023). Model tells you what to discard: Adaptive kv cache compression for llms. *International Conference on Learning Representations*.
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673.
- Geva, M., Schuster, R., Berant, J., and Levy, O. (2021). Transformer feed-forward layers are key-value memories.
- Ghandeharioun, A., Caciularu, A., Pearce, A., Dixon, L., and Geva, M. (2024). Patchscopes: A unifying framework for inspecting hidden representations of language models. *2401.06102v4*.
- Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. (2023). Looped transformers as programmable computers. In *International Conference on Machine Learning*, pages 11398–11442. PMLR.
- Goodrich, B., Rao, V., Liu, P. J., and Saleh, M. (2019). Assessing the factual accuracy of generated text. In *proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 166–175.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billoock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhota, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi,

N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damlaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu,

- S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. (2024). The llama 3 herd of models. 2407.21783v3.
- Hao, Y., Yu, H., and You, J. (2025). Beyond facts: Evaluating intent hallucination in large language models. 2506.06539v1. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2025).
- Harrasse, A., Quirke, P., Neo, C., Nathawani, D., Marks, L., and Abdullah, A. (2025). Tinysql: A progressive text-to-sql dataset for mechanistic interpretability research. 2503.12730v3.
- Harris, Z. (1954). Distributional structure. *Word*, 10(2-3):146–162.
- Haviv, A., Cohen, I., Gidron, J., Schuster, R., Goldberg, Y., and Geva, M. (2023). Understanding transformer memorization recall through idioms. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 248–264.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. (2022). Training compute-optimal large language models. 2203.15556v1.
- Hu, L., Xu, Y., Gu, X., and Han, L. (2025a). Following the whispers of values: Unraveling neural mechanisms behind value-oriented behaviors in llms. 2504.04994v2.
- Hu, Y., Gan, L., Xiao, W., Kuang, K., and Wu, F. (2025b). Fine-tuning large language models for improving factuality in legal question answering. 2501.06521v1.
- Huang, J., Yang, D., and Potts, C. (2024). Demystifying verbatim memorization in large language models. *arXiv preprint arXiv:2407.17817*.
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. 2311.05232v2.
- Huo, S., Arabzadeh, N., and Clarke, C. L. (2023). Retrieving supporting evidence for llms generated answers. *arXiv preprint arXiv:2306.13781*.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. (2023). Mistral 7b. 2310.06825v1.

- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Scao, T. L., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. (2024). Mixtral of experts. *2401.04088v1*.
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Allen Lane.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. (2019). Similarity of neural network representations revisited.
- Lee, N., Ping, W., Xu, P., Patwary, M., Fung, P. N., Shoenybi, M., and Catanzaro, B. (2022). Factuality enhanced language models for open-ended text generation. *Advances in Neural Information Processing Systems*, 35:34586–34599.
- Lepori, M. A., Mozer, M. C., and Ghandeharioun, A. (2024). Racing thoughts: Explaining contextualization errors in large language models. *2410.02102v2*.
- Li, H., Chi, H., Liu, M., and Yang, W. (2024). Look within, why llms hallucinate: A causal perspective. *2407.10153v1*.
- Li, J., Cheng, X., Zhao, W. X., Nie, J.-Y., and Wen, J.-R. (2023). Halueval: A large-scale hallucination evaluation benchmark for large language models.
- Li, J., Zhou, Y., Lu, J., Tyen, G., Gui, L., Aloisi, C., and He, Y. (2025). Two heads are better than one: Dual-model verbal reflection at inference-time. *2502.19230v1*.
- Liang, M., Arun, A., Wu, Z., Munoz, C., Lutch, J., Kazim, E., Koshiyama, A., and Treleaven, P. (2024). Thames: An end-to-end tool for hallucination mitigation and evaluation in large language models. *2409.11353v3*. NeurIPS Workshop on Socially Responsible Language Modelling Research 2024.
- Lin, S., Hilton, J., and Evans, O. (2022). Truthfulqa: Measuring how models mimic human falsehoods.
- Lin, Z., Basu, S., Beigi, M., Manjunatha, V., Rossi, R. A., Wang, Z., Zhou, Y., Balasubramanian, S., Zarei, A., Rezaei, K., Shen, Y., Yao, B. M., Xu, Z., Liu, Q., Zhang, Y., Sun, Y., Liu, S., Shen, L., Li, H., Feizi, S., and Huang, L. (2025). A survey on mechanistic interpretability for multi-modal foundation models. *2502.17516v1*.
- Lin, Z., Niu, Z., Wang, Z., and Xu, Y. (2024). Interpreting and mitigating hallucination in mllms through multi-agent debate. *2407.20505v1*.
- Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyriallidis, A., and Shrivastava, A. (2023). Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36:52342–52364.
- Lu, X., Li, X., Cheng, Q., Ding, K., Huang, X., and Qiu, X. (2024). Scaling laws for fact memorization of large language models. *arXiv preprint arXiv:2406.15720*.
- Lundberg, S. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *1705.07874v2*.

- Manakul, P., Liusie, A., and Gales, M. J. (2023). Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. *arXiv preprint arXiv:2303.08896*.
- McCoy, R. (2019). Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. *arXiv preprint arXiv:1902.01007*.
- Meloux, M., Maniu, S., Portet, F., and Peyrard, M. (2025). Everything, everywhere, all at once: Is mechanistic interpretability identifiable? *2502.20914v1*.
- Meng, K., Bau, D., Andonian, A., and Belinkov, Y. (2023). Locating and editing factual associations in gpt.
- Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P. W., Iyyer, M., Zettlemoyer, L., and Hajishirzi, H. (2023). Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. *arXiv preprint arXiv:2305.14251*.
- Minegishi, G., Furuta, H., Iwasawa, Y., and Matsuo, Y. (2025). Rethinking evaluation of sparse autoencoders through the representation of polysemous words. *2501.06254v2*.
- Nan, F., Nallapati, R., Wang, Z., Santos, C. N. d., Zhu, H., Zhang, D., McKeown, K., and Xiang, B. (2021). Entity-level factual consistency of abstractive text summarization. *arXiv preprint arXiv:2102.09130*.
- Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. (2020). Zoom in: An introduction to circuits. *Distill*. <https://distill.pub/2020/circuits/zoom-in>.
- OLMo, T., Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., Arora, S., Bhagia, A., Gu, Y., Huang, S., Jordan, M., Lambert, N., Schwenk, D., Tafjord, O., Anderson, T., Atkinson, D., Brahman, F., Clark, C., Dasigi, P., Dziri, N., Guerquin, M., Ivison, H., Koh, P. W., Liu, J., Malik, S., Merrill, W., Miranda, L. J. V., Morrison, J., Murray, T., Nam, C., Pyatkin, V., Rangapur, A., Schmitz, M., Skjonsberg, S., Wadden, D., Wilhelm, C., Wilson, M., Zettlemoyer, L., Farhadi, A., Smith, N. A., and Hajishirzi, H. (2025). 2 olmo 2 furious.
- Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. (2022). In-context learning and induction heads.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I.,

- Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, J. H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk, Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O’Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M., de Oliveira Pinto, H. P., Michael, Pokorny, Pokrass, M., Pong, V. H., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M. B., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. (2023). Gpt-4 technical report. 2303.08774v6.
- Palikhe, A., Yu, Z., Wang, Z., and Zhang, W. (2025). Towards transparent ai: A survey on explainable large language models. 2506.21812v1.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.
- Penedo, G., Kydlíček, H., allal, L. B., Lozhkov, A., Mitchell, M., Raffel, C., Werra, L. V., and Wolf, T. (2024). The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, E., and Launay, J. (2023). The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. (2018). Deep contextualized word representations. 1802.05365v2.
- Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. (2024). Qwen2.5 technical report. 2412.15115v2.

- Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. (2024). Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*.
- Ravichander, A., Ghela, S., Wadden, D., and Choi, Y. (2025). Halogen: Fantastic llm hallucinations and where to find them. *2501.08292v1*.
- Ren, R., Wang, Y., Qu, Y., Zhao, W. X., Liu, J., Tian, H., Wu, H., Wen, J.-R., and Wang, H. (2023). Investigating the factual knowledge boundary of large language models with retrieval augmentation. *arXiv preprint arXiv:2307.11019*.
- Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "why should i trust you?": Explaining the predictions of any classifier. *1602.04938v3*.
- Saji, A., Husain, J. A., Jayakumar, T., Dabre, R., Kunchukuttan, A., and Puduppully, R. (2025). Romanlens: The role of latent romanization in multilinguality in llms. *2502.07424v3*.
- Sanwal, M. (2025). Layered chain-of-thought prompting for multi-agent llm systems: A comprehensive approach to explainable large language models. *2501.18645v2*.
- Sarti, G., Feldhus, N., Sickert, L., van der Wal, O., Nissim, M., and Bisazza, A. (2023). Inseq: An interpretability toolkit for sequence generation models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 421–435, Toronto, Canada. Association for Computational Linguistics.
- Saunshi, N., Dikkala, N., Li, Z., Kumar, S., and Reddi, S. J. (2025). Reasoning with latent thoughts: On the power of looped transformers. *arXiv preprint arXiv:2502.17416*.
- Schut, L., Gal, Y., and Farquhar, S. (2025). Do multilingual llms think in english? *2502.15603v1*.
- Singh, C., Inala, J. P., Galley, M., Caruana, R., and Gao, J. (2024). Rethinking interpretability in the era of large language models. *2402.01761v1*.
- Soldaini, L., Kinney, R., Bhagia, A., Schwenk, D., Atkinson, D., Authur, R., Bogin, B., Chandu, K., Dumas, J., Elazar, Y., Hofmann, V., Jha, A. H., Kumar, S., Lucy, L., Lyu, X., Lambert, N., Magnusson, I., Morrison, J., Muennighoff, N., Naik, A., Nam, C., Peters, M. E., Ravichander, A., Richardson, K., Shen, Z., Strubell, E., Subramani, N., Tafjord, O., Walsh, P., Zettlemoyer, L., Smith, N. A., Hajishirzi, H., Beltagy, I., Groeneveld, D., Dodge, J., and Lo, K. (2024). Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *arXiv preprint arXiv:2403.19851*.
- Stoeck, N., Gordon, M., Zhang, C., and Lewis, O. (2024). Localizing paragraph memorization in language models. *arXiv preprint arXiv:2403.19851*.
- Su, W., Tang, Y., Ai, Q., Wang, C., Wu, Z., and Liu, Y. (2024). Mitigating entity-level hallucination in large language models. *2407.09417v2*.
- Sun, Y., Gai, Y., Chen, L., Ravichander, A., Choi, Y., and Song, D. (2025). Why and how llms hallucinate: Connecting the dots with subsequence associations. *2504.12691v1*.
- Sun, Z., Zang, X., Zheng, K., Song, Y., Xu, J., Zhang, X., Yu, W., Song, Y., and Li, H. (2024). Re-deep: Detecting hallucination in retrieval-augmented generation via mechanistic interpretability. *2410.11414v2*.
- SuperAnnotate (2024).
- Tatsat, H. and Shater, A. (2025). Beyond the black box: Interpretability of llms in finance. *2505.24650v1*.

- Taylor, R., Kardas, M., Cucurull, G., Scialom, T., Hartshorn, A., Saravia, E., Poulton, A., Kerkez, V., and Stojnic, R. (2022). Galactica: A large language model for science. *2211.09085v1*.
- Team, G. (2023). Gemini: A family of highly capable multimodal models. *2312.11805v5*.
- Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J.-B., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchison, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozińska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshhev, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucińska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., van Amersfoort, J., Gordon, J., Lipschultz, J., Newlan, J., yeong Ji, J., Mohamed, K., Badola, K., Black, K., Millican, K., McDonell, K., Nguyen, K., Sodhia, K., Greene, K., Sjoesund, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., McNealus, L., Soares, L. B., Kilpatrick, L., Dixon, L., Martins, L., Reid, M., Singh, M., Iverson, M., Görner, M., Velloso, M., Wirth, M., Davidow, M., Miller, M., Rahtz, M., Watson, M., Risdal, M., Kazemi, M., Moynihan, M., Zhang, M., Kahng, M., Park, M., Rahman, M., Khatwani, M., Dao, N., Bardoliwalla, N., Devanathan, N., Dumai, N., Chauhan, N., Wahltinez, O., Botarda, P., Barnes, P., Barham, P., Michel, P., Jin, P., Georgiev, P., Culliton, P., Kuppala, P., Comanescu, R., Merhej, R., Jana, R., Rokni, R. A., Agarwal, R., Mullins, R., Saadat, S., Carthy, S. M., Cogan, S., Perrin, S., Arnold, S. M. R., Krause, S., Dai, S., Garg, S., Sheth, S., Ronstrom, S., Chan, S., Jordan, T., Yu, T., Eccles, T., Hennigan, T., Kocisky, T., Doshi, T., Jain, V., Yadav, V., Meshram, V., Dharmadhikari, V., Barkley, W., Wei, W., Ye, W., Han, W., Kwon, W., Xu, X., Shen, Z., Gong, Z., Wei, Z., Cotruta, V., Kirk, P., Rao, A., Giang, M., Peran, L., Warkentin, T., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Sculley, D., Banks, J., Dragan, A., Petrov, S., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Borgeaud, S., Fiedel, N., Joulin, A., Kenealy, K., Dadashi, R., and Andreev, A. (2024). Gemma 2: Improving open language models at a practical size. *2408.00118v3*.
- Tonmoy, S. M. T. I., Zaman, S. M. M., Jain, V., Rani, A., Rawte, V., Chadha, A., and Das, A. (2024). A comprehensive survey of hallucination mitigation techniques in large language models. *2401.01313v3*.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang,

- S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. *2307.09288v2*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *1706.03762v7*.
- Venkit, P. N., Chakravorti, T., Gupta, V., Biggs, H., Srinath, M., Goswami, K., Rajtmajer, S., and Wilson, S. (2024). "confidently nonsensical?": A critical survey on the perspectives and challenges of 'hallucinations' in nlp. *arXiv preprint arXiv:2404.07461*.
- Walters, W. H. and Wilder, E. I. (2023). Fabrication and errors in the bibliographic citations generated by chatgpt. *Scientific Reports*, 13(1):14045.
- Wang, K., Variengien, A., Conmy, A., Shlegeris, B., and Steinhardt, J. (2022). Interpretability in the wild: a circuit for indirect object identification in gpt-2 small.
- Wang, Z. (2025). Logitlens4llms: Extending logit lens analysis to modern large language models.
- Wei, J., Huang, D., Lu, Y., Zhou, D., and Le, Q. V. (2023a). Simple synthetic data reduces sycophancy in large language models. *arXiv preprint arXiv:2308.03958*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2023b). Chain-of-thought prompting elicits reasoning in large language models.
- Wendler, C., Veselovsky, V., Monea, G., and West, R. (2024). Do llamas work in english? on the latent language of multilingual transformers. *2402.10588v4*.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. (2023). Efficient streaming language models with attention sinks. *arXiv preprint arXiv: 2309.17453*.
- Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., Dong, G., Wei, H., Lin, H., Tang, J., Wang, J., Yang, J., Tu, J., Zhang, J., Ma, J., Yang, J., Xu, J., Zhou, J., Bai, J., He, J., Lin, J., Dang, K., Lu, K., Chen, K., Yang, K., Li, M., Xue, M., Ni, N., Zhang, P., Wang, P., Peng, R., Men, R., Gao, R., Lin, R., Wang, S., Bai, S., Tan, S., Zhu, T., Li, T., Liu, T., Ge, W., Deng, X., Zhou, X., Ren, X., Zhang, X., Wei, X., Ren, X., Liu, X., Fan, Y., Yao, Y., Zhang, Y., Wan, Y., Chu, Y., Liu, Y., Cui, Z., Zhang, Z., Guo, Z., and Fan, Z. (2024a). Qwen2 technical report. *2407.10671v4*.
- Yang, S., Gribovskaya, E., Kassner, N., Geva, M., and Riedel, S. (2024b). Do large language models latently perform multi-hop reasoning? *2402.16837v2*.
- Yu, L., Cao, M., Cheung, J. C. K., and Dong, Y. (2024). Mechanistic understanding and mitigation of language model non-factual hallucinations. *2403.18167v2*.
- Zhang, L., Hu, L., and Wang, D. (2025a). Mechanistic unveiling of transformer circuits: Self-influence as a key to model reasoning. *2502.09022v2*.
- Zhang, M., Press, O., Merrill, W., Liu, A., and Smith, N. A. (2023a). How language model hallucinations can snowball. *arXiv preprint arXiv:2305.13534*.
- Zhang, X., Peng, B., Tian, Y., Zhou, J., Jin, L., Song, L., Mi, H., and Meng, H. (2024a). Self-alignment for factuality: Mitigating hallucinations in llms via self-evaluation. *arXiv preprint arXiv:2402.09267*.
- Zhang, Y., Chen, J., Wang, J., Liu, Y., Yang, C., Shi, C., Zhu, X., Lin, Z., Wan, H., Yang, Y., Sakai, T., Feng, T., and Yamana, H. (2024b). Toolbehonest: A multi-level hallucination diagnostic benchmark for tool-augmented large language models. *2406.20015v2*.
- Zhang, Y., Du, W., Jin, D., Fu, J., and Jin, Z. (2025b). Finite state automata inside transformers

- with chain-of-thought: A mechanistic study on state tracking. 2502.20129v3.
- Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., et al. (2023b). Siren’s song in the ai ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. (2023c). H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710.
- Zhao, H., Yang, F., Shen, B., Lakkaraju, H., and Du, M. (2024). Towards uncovering how large language model works: An explainability perspective. 2402.10688v2.
- Zhao, Z. and Shan, B. (2024). Reagent: Towards a model-agnostic feature attribution method for generative language models. *arXiv preprint arXiv:2402.00794*.
- Zhu, T., Liu, Q., Pang, L., Jiang, Z., Kan, M.-Y., and Lin, M. (2024). Beyond memorization: The challenge of random memory access in language models. *arXiv preprint arXiv:2403.07805*.
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., Kolter, J. Z., and Hendrycks, D. (2025). Representation engineering: A top-down approach to ai transparency.

Appendix A

User Manual

A.1 Introduction

The LTR (LLM Thought Tracing) system represents a comprehensive Python library designed for analyzing hallucinations in Large Language Models through the novel Distributional Semantics Tracing (DST) framework. This manual provides detailed guidance for users seeking to replicate experiments, analyze model behavior, and understand the fundamental mechanisms behind LLM hallucinations.

The system's core contribution lies in its implementation of DST, which enables researchers to investigate the intrinsic causes of hallucinations by examining the conflict between fast associative reasoning pathways and slower, more deliberative ones. Through systematic analysis, users can identify specific failure patterns such as "Reasoning Shortcut Hijacks" and pinpoint commitment layers where model representations irreversibly diverge from factuality.

A.2 System Requirements and Installation

The LTR system requires Python 3.11 or higher as specified in the project configuration, with recommended hardware including at least 16GB RAM and an NVIDIA GPU with 12GB+ VRAM for optimal performance. The system is designed to work efficiently with UV package manager for dependency resolution, though traditional pip installation is also supported.

Installation can be accomplished through UV package manager using the command `uv pip install git+https://github.com/gagan3012/ltr.git`, or through local cloning and development installation. The system automatically handles all necessary dependencies including specialized libraries for model intervention, attribution analysis, and visualization generation.

Installation can be accomplished through UV package manager:

```
1 # Install using UV (recommended)
2 uv pip install git+https://github.com/gagan3012/ltr
3
4 # Or clone and install locally
5 git clone https://github.com/gagan3012/ltr.git
6 cd ltr
7 uv pip install -e .
```

A.3 Interactive Analysis with Jupyter Notebooks

The primary entry point for most users begins with the comprehensive Jupyter notebook located at `notebooks/Dst_Example_Notebook.ipynb`. This notebook provides a complete walkthrough of the DST pipeline, demonstrating the three-stage analysis process that forms the core of the dissertation's methodology. Users can follow along with concept importance computation, patched representation generation, and spurious span identification while observing real-time visualization generation.

Additional notebooks serve specialized purposes within the research framework. The `Ltr_ds_hallu.ipynb` notebook focuses specifically on hallucination analysis with detailed concept tracking capabilities, allowing researchers to observe how semantic ambiguities develop and propagate through model layers. The `LTR_Reasoning_LLM.ipynb` notebook implements enhanced reasoning pathway analysis using sophisticated multi-component scoring systems that weigh factors such as activation strength, causal effects, and attention patterns. For comprehensive evaluation and benchmarking, the `ltr_eval_final.ipynb` notebook provides systematic testing frameworks across different model configurations and concept categories.

```

1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 from ltr.dst import DistributionalSemanticsTracer
3
4 # Load model and tokenizer
5 model_name = "Qwen/Qwen3-0.6B"
6 model = AutoModelForCausalLM.from_pretrained(model_name, device_map
7         ="auto")
8 tokenizer = AutoTokenizer.from_pretrained(model_name)
9
10 # Initialize the DST tracer
11 tracer = DistributionalSemanticsTracer(model, tokenizer)
12
13 # Run analysis with enhanced visualizations
14 result = tracer.run_analysis(
15     prompt="I am at a concert. I see a bass. Is it a fish? Answer:"
16     ,
17     # Additional parameters for comprehensive analysis
18 )

```

A.4 Core Experimental Framework

The heart of the experimental system lies in the `ConceptualHallucinationAnalyzer`, implemented across multiple versions to provide increasingly sophisticated analysis capabilities. The `hall_test.py` script establishes the foundation with systematic concept testing across three carefully designed categories. Clear concepts such as "dog", "cat", and "house" provide baseline measurements for unambiguous semantic representations. Ambiguous concepts including "bank", "bat", and "spring" reveal how multiple meanings create confusion in model representations. Abstract concepts like "justice", "freedom", and "love" expose the challenges models face with complex philosophical ideas.

The enhanced distributional semantics analysis evolves through multiple implementations, with `ds_v2.py` providing comprehensive reporting and concept ambiguity ranking systems, while `ds_v3.py` represents the latest implementation featuring advanced semantic analysis capabilities. These analyzers generate detailed summary reports that rank concepts by their hallucination potential, providing researchers with quantitative measures of semantic ambiguity, coherence scores, and dispersion metrics that directly correlate with hallucination susceptibility.

```
1 def main():
2     """Main function to run the enhanced hallucination clustering
3         analysis"""
4     # Initialize analyzer with a smaller model for testing
5     analyzer = HallucinationClusterAnalyzer(model_name="Qwen/Qwen2
6         .5-0.5B-Instruct")
7
8     # Run comprehensive analysis
9     results = analyzer.run_comprehensive_analysis(
10         output_dir="ltr_hallucination_analysis"
11     )
12
13     print("\nAnalysis completed successfully!")
14     return results
```

Clear concepts such as "dog", "cat", and "house" provide baseline measurements for unambiguous semantic representations. Ambiguous concepts including "bank", "bat", and "spring" reveal how multiple meanings create confusion in model representations. Abstract concepts like "justice", "freedom", and "love" expose the challenges models face with complex philosophical ideas.

A.5 Advanced Interpretability Techniques

The system incorporates sophisticated interpretability methods that go beyond traditional attention visualization. The attention analysis framework, demonstrated in `attention_analysis_example.py`, provides comprehensive examination of attention patterns across model layers, measuring not just where models attend but how attention to specific concepts influences final predictions. This analysis includes ablation capabilities that systematically remove attention heads to measure their causal contribution to hallucination patterns.

Patchscopes analysis, implemented in `patchscopes_example.py`, offers particularly advanced generation analysis capabilities. This technique tracks entity probabilities during text generation while simultaneously analyzing attention patterns and layer activations. The implementation includes sophisticated hallucination pattern detection that identifies factual inconsistencies, entity confusion, confidence drops, and anomalous activation patterns that indicate model uncertainty. The visualization system generates comprehensive plots including entity trajectory tracking, attention heatmaps, and layer activation analysis with statistical significance testing.

The logit lens technique, available through `logit_lens_example.py`, enables researchers to project intermediate hidden states into vocabulary space, revealing how token predictions evolve across model layers. This provides crucial insights into when and where hallucinations begin to

form within the model’s computational process.

```

1 def main():
2     """Run all examples and create visualizations."""
3     print("LTR Patchscopes Hallucination Analysis Examples")
4     print("=" * 60)
5
6     viz_dir = "visualization_results"
7     os.makedirs(viz_dir, exist_ok=True)
8
9     # Run basic hallucination detection
10    results1 = example_1_basic_hallucination_detection()
11    prompt1 = "The capital of Mars is"
12    visualize_results(
13        results1,
14        "Basic Hallucination Detection",
15        prompt1,
16        save_path=os.path.join(viz_dir, "example1"),
17    )

```

A.6 Causal Intervention and Control Experiments

The system provides sophisticated causal intervention capabilities through multiple complementary approaches. Backpatching, demonstrated in `backpatching_example.py`, measures causal effects by systematically patching activations between different forward passes, allowing researchers to isolate the specific contribution of individual components to hallucination generation. The patching control framework, implemented in `patching_control_example.py`, performs controlled experiments using corrupted and clean prompts to isolate the influence of specific tokens and positions.

Entity analysis, available through `entity_analysis_example.py`, focuses specifically on measuring the causal influence of entities on model predictions while analyzing how entity representations differ across various contexts. This technique proves particularly valuable for understanding how models conflate similar entities or incorrectly transfer properties between different entities.

A.7 Baseline Attribution and Comparative Analysis

The research framework includes comprehensive baseline comparisons implemented in `baselines.py`. This system supports multiple attribution methods including LIME explanations for local interpretability, Integrated Gradients for gradient-based attribution, Gradient SHAP for feature importance analysis, and REAGEANT for advanced attribution techniques. Each method generates both JSON data files and HTML visualizations, enabling detailed comparative analysis across different explanation approaches.

The baseline system uses the efficient SmolLM2-135M model for rapid attribution computation, processing 500 attribution steps with optimized batch sizes to balance computational efficiency with attribution quality. Results are automatically saved in structured formats that facilitate systematic comparison with the novel DST approach.

A.8 Main Experimental Pipeline and Visualization

The complete experimental pipeline orchestrates through `explain_hallucinations/run_distr_semantics.py` which coordinates all stages of the DST analysis process. This script manages the complex interplay between concept extraction, semantic analysis, and causal intervention measurement. The training data analysis component, implemented in `training_data_distr_semantics.py`, performs comprehensive semantic analysis with detailed reporting that includes concept ambiguity rankings, coherence metrics, and layer-by-layer progression analysis.

Visualization generation occurs through `distr_semantics_viz.py`, which produces the complete suite of analysis visualizations including concept heatmaps, confusion matrices, layer evolution plots, and causal intervention visualizations. These visualizations employ sophisticated statistical analysis and threshold-based highlighting to emphasize significant patterns and relationships within the data.

A.9 Model Configuration and Performance Optimization

The system supports analysis across multiple model architectures, with primary research conducted using meta-llama/Llama-3.2-1B-Instruct for comprehensive analysis, Qwen/Qwen2.5-0.5B-Instruct for alternative comparison studies, and HuggingFaceTB/SmolLM2-135M for baseline experiments and rapid testing. Model selection follows systematic principles that balance analytical depth with computational efficiency.

Performance optimization strategies include dynamic layer selection for focused analysis, adjustable training data sample sizes through the `max_texts` parameter, and configurable concept scope limitations for targeted investigation. The system automatically manages memory allocation and provides monitoring capabilities for GPU utilization and processing time optimization.

A.10 Results Interpretation and Expected Outputs

The complete analysis pipeline generates comprehensive output packages that include multiple visualization formats and detailed analytical reports. Semantic drift trajectories reveal how concept representations evolve across model layers, while activation distribution plots provide statistical analysis of concept activation patterns. Attention heatmaps visualize where models focus during hallucination generation, and entity probability charts track how different entities compete for selection during text generation.

Analytical reports provide quantitative measures including concept ambiguity rankings with numerical scores, coherence and dispersion metrics that correlate with hallucination susceptibility, and detailed insights into layer-specific behavior patterns. The system generates both human-readable summary reports and structured data files suitable for further computational analysis, enabling researchers to build upon the foundational DST framework for extended investigation.

Appendix B

Maintenance Manual

B.1 Project Architecture and Configuration

The LTR system employs modern Python packaging standards through `pyproject.toml` configuration, establishing a robust foundation for dependency management and distribution. The project requires Python 3.11 or higher and leverages UV package manager for optimal dependency resolution, though traditional pip installation remains supported for broader compatibility. The configuration integrates specialized dependencies including `baukit` for model intervention capabilities, advanced attribution libraries for baseline comparisons, and comprehensive visualization frameworks for analysis presentation.

The core dependency architecture balances functionality with maintainability through carefully selected libraries. `Baukit` provides essential model intervention utilities sourced directly from git repositories, enabling sophisticated causal analysis capabilities. The `nnsight` framework offers neural network analysis capabilities specifically designed for transformer introspection, while `in-seq` provides attribution methods for baseline experimental comparisons. Additional dependencies support quantization through `bitsandbytes`, dataset processing through the `datasets` library, and explanation generation through LIME and SHAP implementations.

B.2 Core Library Structure and Organization

The primary library code resides within `src/ltr/` and follows a modular architecture that separates concerns while maintaining cohesive functionality. The package initialization file serves as a comprehensive export hub, making available over 100 functions and classes that span the complete analytical framework. This centralized approach enables users to access all major capabilities through simple import statements while maintaining clear separation between different analytical techniques.

The `dst.py` module represents the primary contribution of the research, implementing the novel Distributional Semantics Tracing framework that forms the core of the dissertation’s methodology. This implementation provides the foundational algorithms for hallucination analysis through semantic drift detection and commitment layer identification. Supporting modules extend this foundation with specialized capabilities including attention pattern analysis, logit lens projection techniques, and sophisticated causal intervention frameworks.

The reasoning analysis module implements enhanced pathway analysis through multi-component scoring systems that combine order scoring, activation strength measurement, causal effect quantification, attention pattern analysis, and reasoning chain continuity assessment. This

sophisticated approach enables researchers to understand not just what models produce but how they arrive at their conclusions through complex internal computations.

B.3 Experimental Implementation Framework

The `explain_hallucinations/` directory contains specialized implementations for dissertation experiments, with each script serving specific research objectives within the broader analytical framework. The main experiment orchestration occurs through `run_distr_semantics.py`, which coordinates the complete DST analysis pipeline including concept extraction, semantic analysis, and causal intervention measurement. This script manages the complex interplay between different analytical components while maintaining reproducible experimental conditions.

Training data analysis capabilities, implemented in `training_data_distr_semantics.py`, provide comprehensive semantic analysis with detailed reporting frameworks. This component generates concept ambiguity rankings, coherence metrics, and layer-by-layer progression analysis that reveals how semantic quality evolves through model architectures. The analysis produces structured reports that quantify concept hallucination potential through multiple complementary metrics.

Visualization generation occurs through `distr_semantics_viz.py`, which implements a comprehensive visualization engine capable of producing concept heatmaps, confusion matrices, layer evolution plots, and causal intervention visualizations. The visualization system employs statistical analysis techniques and threshold-based highlighting to emphasize significant patterns within complex analytical results.

The baseline comparison framework, implemented in `baselines.py`, provides systematic evaluation across multiple attribution methods including LIME, Integrated Gradients, Gradient SHAP, and REAGENT. This framework generates both structured JSON data and interactive HTML visualizations, enabling detailed comparative analysis between traditional attribution methods and the novel DST approach.

B.4 Example Script Architecture and Progression

The `examples/` directory provides progressive demonstration of system capabilities through carefully structured scripts that build from basic usage to advanced analytical techniques. Basic examples including `ltr_example.py` and `huggingface_example.py` establish fundamental usage patterns for core library functionality, model integration, and basic concept extraction capabilities.

Advanced hallucination analysis examples demonstrate sophisticated analytical capabilities through multiple implementation versions. The `hall_test.py` framework establishes comprehensive hallucination testing through systematic concept categorization and testing scenarios. Enhanced implementations including `ds_v2.py` and `ds_v3.py` provide increasingly sophisticated analysis capabilities with improved reporting systems and concept ambiguity ranking frameworks.

Interpretability technique demonstrations showcase the breadth of analytical capabilities through specialized scripts. The attention analysis example provides comprehensive examination of attention patterns including head ablation and concept attention measurement. Patchscopes examples demonstrate advanced generation analysis with entity tracking, attention visualization, layer activation analysis, and sophisticated hallucination pattern detection capabilities.

B.5 Implementation Details and Key Algorithms

The ConceptualHallucinationAnalyzer represents the core analytical framework, implementing systematic concept testing across carefully designed categories. This analyzer creates comprehensive test scenarios that systematically evaluate clear, ambiguous, and abstract concepts while generating detailed summary reports with quantitative ambiguity rankings. The implementation supports both comprehensive analysis across all model layers and targeted investigation of specific architectural components.

Enhanced reasoning analysis employs sophisticated multi-component scoring that combines multiple factors through weighted mathematical models. The scoring system evaluates pathway quality through order assessment, activation strength measurement, causal effect quantification, attention pattern analysis, and reasoning chain continuity evaluation. This comprehensive approach enables researchers to understand the complex computational pathways that lead to both accurate and hallucinated outputs.

Hallucination pattern detection capabilities within the patchscopes framework implement advanced recognition algorithms that identify specific failure modes. The system detects factual inconsistencies through knowledge base comparison, entity confusion through similarity analysis, confidence drops across generation steps, and anomalous activation patterns that indicate model uncertainty. These detection capabilities provide quantitative measures of hallucination risk during text generation.

B.6 Configuration Management and Extension Patterns

The system employs consistent configuration patterns that balance flexibility with maintainability through standardized parameter management and dynamic directory creation systems. Model selection follows systematic principles with primary research models, efficient baseline models, and alternative comparison models clearly designated for different analytical purposes. Configuration parameters including analysis thresholds, batch sizes, and processing parameters maintain consistency across different experimental contexts.

Output management occurs through dynamic directory creation systems that generate timestamped analysis directories, organize file hierarchies for different analysis types, and provide comprehensive logging capabilities. The system supports multiple output formats including JSON data files for computational analysis and HTML visualizations for interactive exploration.

Extension mechanisms enable researchers to add new capabilities through consistent patterns that maintain system coherence. Attribution method extensions can be implemented through inseq library integration, while new model architectures are supported through consistent configuration pattern implementation. Analysis techniques follow established module patterns that maintain compatibility with existing visualization and reporting frameworks.

B.7 Quality Assurance and Development Standards

The testing framework provides comprehensive validation through component testing via individual example script execution, integration testing through notebook validation, baseline verification through attribution method comparison, and performance monitoring through built-in timing and

memory tracking capabilities. This multi-layered approach ensures system reliability across different usage scenarios and computational environments.

Debugging infrastructure includes comprehensive progress tracking through `tqdm` integration, detailed memory usage monitoring for both GPU and CPU resources, sophisticated error reporting and exception handling, and intermediate result saving capabilities that enable analysis debugging and verification. The system provides detailed feedback during long-running analyses while maintaining computational efficiency.

Development standards emphasize consistent code organization following separation of concerns principles, comprehensive documentation through README files and interactive notebooks, version control through both `pyproject.toml` and module-level versioning, and adherence to modern Python packaging standards. The modular architecture enables independent development and testing of individual components while maintaining overall system coherence and functionality.

Appendix C

Additional Information

C.1 LLM-as-Judge Prompt

System Prompt You are an expert research assistant specializing in the explainability and interpretability of large language models. Your task is to act as an expert human annotator and evaluate the **faithfulness** of a proposed explanation for an LLM's output. "Faithfulness" means the explanation accurately reflects the model's *actual* internal computational process for arriving at its output. It is not about whether the explanation *sounds plausible* or whether the LLM's output is *correct*. The explanation must be true to the model's reasoning. You will be given the LLM's original prompt, its final output, and the explanation generated by our algorithm. In some cases, you may also be given the model's own Chain-of-Thought (CoT) verbalisation. Please evaluate the explanation's faithfulness based on the following three criteria and provide a final score on a scale of 0 to 10.

Evaluation Criteria

1. **Strength of Evidence:** Does the explanation correctly identify the specific input tokens or concepts that had the highest influence (saliency) on the model's output? A faithful explanation must be grounded in the parts of the input that the model actually "paid attention to." An explanation that cites irrelevant tokens or misses the key drivers is unfaithful.
2. **Agreement with Truth (Logical Correctness):** Does the explanation's described logic accurately represent the model's internal process? If you "simulate" the reasoning path described in the explanation, does it naturally lead to the model's given output? The explanation must correctly describe *why* and *how* the identified evidence leads to the conclusion. An explanation describing a flawed or fabricated logical step is unfaithful.
3. **Relevance to Verbalisation:** (When a CoT is provided) Is the mechanistic explanation consistent with the model's own stated reasoning? The CoT is not infallible ground truth, but a significant contradiction between the mechanistic explanation and the CoT indicates a potential faithfulness issue.
 - **High Alignment:** The explanation provides a mechanistic basis for the CoT's steps.
 - **No Contradiction:** The explanation doesn't conflict with the CoT.
 - **Contradiction:** The explanation describes a reasoning path (e.g., "model ignored X") that is fundamentally at odds with what the model claimed to do in its CoT (e.g., "I considered X and..."). This lowers the faithfulness score.

Scoring Rubric (0-10)

- **10 (Perfectly Faithful):** The explanation is a perfect reflection of the model's process. It correctly identifies all key evidence, the logic is flawless, and it fully aligns with any verbalisation.
- **7-9 (Mostly Faithful):** The explanation is largely correct but may have minor omissions or a slight lack of precision in its logic or evidence attribution. The core reasoning path is accurate.
- **4-6 (Partially Faithful):** The explanation captures some true aspects of the model's process but contains a significant error, such as misidentifying the primary driver or misrepresenting a key logical step.
- **1-3 (Mostly Unfaithful):** The explanation is fundamentally flawed. It is based on irrelevant evidence or describes a reasoning process that does not align with the model's output. It is more misleading than helpful.
- **0 (Completely Unfaithful):** The explanation is entirely incorrect, fabricated, or describes a process that is logically impossible.

Required Output Format

Provide your assessment in the following structure. First, briefly reason through each of the three criteria. Second, provide your final score.

****Reasoning:****

1. ****Strength of Evidence:**** [Your analysis of whether the explanation correctly identifies influential input tokens.]
2. ****Logical Correctness:**** [Your analysis of whether the explanation's logic correctly simulates the model's process.]
3. ****Relevance to Verbalisation:**** [Your analysis of the explanation's alignment with the model's CoT, if provided. If not, state "N/A".]

****Final Score:**** [Your score from 0-10]

Example Task & Ideal Response

LLM Prompt (Context): "Please answer succinctly. I am in a forest. I see a trunk. Is it an automobile part?"

LLM Output: "Yes."

Explanation from Algorithm: "The model hallucinated by ignoring the context 'in a forest'. It defaulted to the automotive meaning of 'trunk' instead of the biological one."

Your Evaluation:

****Reasoning:****

1. ****Strength of Evidence:**** The explanation correctly identifies the crucial interaction: the model over-indexed on the word 'trunk' and 'automobile part' while ignoring the salient contextual phrase

- 'in a forest'. This correctly identifies the key drivers and the key neglected evidence.
2. ****Logical Correctness:**** The logic is sound. Ignoring the "forest" context forces a disambiguation of "trunk". The most common alternative association, prompted by "automobile part", is the automotive one. This logical path directly and correctly leads to the erroneous "Yes."
 3. ****Relevance to Verbalisation:**** N/A.
- **Final Score:**** 10.0

C.2 Reproducibility Guide: Distributional Semantics Tracing (DST) Experiments

This guide provides instructions for reproducing the experiments described in the thesis using the Distributional Semantics Tracing (DST) framework. All code examples are based on the existing ltr codebase.

C.2.1 Setup & Installation

```
1 uv pip install git+https://github.com/gagan3012/ltr.git
```

C.2.2 Qualitative Comparison of Explainability Methods

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 from ltr.dst import DistributionalSemanticsTracer
3
4 # Load model
5 model_name = "Qwen/Qwen3-0.6B" # Can be replaced with "google/
6                               # gemma2-2b", "google/gemma2-9b", or "allenai/OLMo2-1B"
7 model = AutoModelForCausalLM.from_pretrained(model_name)
8 tokenizer = AutoTokenizer.from_pretrained(model_name)
9
10 # Initialize DST tracer
11 tracer = DistributionalSemanticsTracer(model, tokenizer)
12
13 # Example 1: Australia case
14 result_australia = tracer.run_analysis(
15     prompt="Answer based on the information provided here. Forget
16           everything you know about geography. The capital city of
17           Australia was just renamed from Canberra to Dublin. Is the
18           capital city of Australia named Dublin?",
19     factual_prompt="The capital city of Australia is Canberra.",
20     concept_examples=["Australia", "capital", "Canberra", "Dublin",
21                     "renamed"],
22     hallucinated_output="Canberra",
23     run_intervention=True,
```

```
19     enhanced_viz=True
20 )
21
22 # Example 2: Bank ambiguity case
23 result_bank = tracer.run_analysis(
24     prompt="John is going fishing, so he walks over to the river
25           bank. Can he withdraw money at the bank?",
26     factual_prompt="A river bank is a geographical feature, not a
27                   financial institution.",
28     concept_examples=["bank", "river", "money", "withdraw", "
29                     fishing"],
30     hallucinated_output="Yes",
31     run_intervention=True,
32     enhanced_viz=True
33 )
34
35 # Example 3: Bass ambiguity case
36 result_bass = tracer.run_analysis(
37     prompt="Please answer succinctly. I am at a concert. I see a
38           bass. Is it a fish?",
39     factual_prompt="A bass at a concert is a musical instrument,
40                   not a fish.",
41     concept_examples=["bass", "concert", "instrument", "fish", "
42                     music"],
43     hallucinated_output="Yes",
44     run_intervention=True,
45     enhanced_viz=True
46 )
47
48 # Example 4: Java ambiguity case
49 result_java = tracer.run_analysis(
50     prompt="Please answer succinctly. I am coding. I see some java.
51           Is it a programming language?",
52     factual_prompt="Java in a coding context is a programming
53                   language.",
54     concept_examples=["java", "coding", "programming", "language",
55                     "coffee"],
56     hallucinated_output="coffee",
57     run_intervention=True,
58     enhanced_viz=True
59 )
60
61 # Example 5: Physician case
62 result_physician = tracer.run_analysis(
63     prompt="Please answer succinctly. The physician is somebody's
64           grandmother. Is the physician a woman?",
65     factual_prompt="A grandmother is a woman.",
```

```
56     concept_examples=["physician", "grandmother", "woman", "doctor"  
57         , "gender"],  
58     hallucinated_output="No",  
59     run_intervention=True,  
60     enhanced_viz=True  
61 )  
62 # Results will include:  
63 # - Concept importance scores across layers  
64 # - Semantic drift trajectory visualizations  
65 # - Spurious spans responsible for hallucinations  
66 # - Concept network visualizations  
67 # - Activation distribution comparisons
```

C.2.3 Pinpointing the Onset of Hallucination

```
1  from transformers import AutoModelForCausalLM, AutoTokenizer  
2  from ltr.dst import DistributionalSemanticsTracer  
3  import matplotlib.pyplot as plt  
4  import numpy as np  
5  
6  # Load model  
7  model_name = "google/gemma2-2b"  
8  model = AutoModelForCausalLM.from_pretrained(model_name)  
9  tokenizer = AutoTokenizer.from_pretrained(model_name)  
10  
11 # Initialize tracer  
12 tracer = DistributionalSemanticsTracer(model, tokenizer)  
13  
14 # Example: Bass ambiguity case  
15 prompt = "Please answer succinctly. I am at a concert. I see a bass  
16     . Is it a fish?"  
17  
18 # Define correct and incorrect reasoning pathways  
19 correct_pathways = [  
20     ["concert", "bass", "instrument"],  
21     ["music", "bass", "instrument"]  
22 ]  
23  
24 incorrect_pathways = [  
25     ["bass", "fish", "animal"],  
26     ["water", "bass", "fish"]  
27 ]  
28  
29 # Calculate DSS scores for each layer  
30 n_layers = tracer.n_layers  
31 dss_scores = []
```

```
32 for layer_idx in range(n_layers):
33     dss = tracer.compute_dss(
34         prompt=prompt,
35         correct_pathways=correct_pathways,
36         incorrect_pathways=incorrect_pathways,
37         layer_idx=layer_idx
38     )
39     dss_scores.append(dss)
40
41 # Plot DSS across layers to identify critical points
42 plt.figure(figsize=(10, 6))
43 plt.plot(range(n_layers), dss_scores, 'o-', linewidth=2)
44 plt.xlabel('Model Layers')
45 plt.ylabel('Distributional Semantics Strength (DSS)')
46 plt.title('Layer-wise DSS Analysis')
47 plt.grid(True)
48
49 # Find critical layers
50 prediction_onset = np.argmax(np.array(dss_scores) < 0.6) # DSS
    starts dropping
51 semantic_inversion = np.argmax(np.array(dss_scores) < 0.5) # DSS
    crosses below 0.5
52 commitment_layer = np.argmin(dss_scores) # DSS reaches minimum
53
54 # Mark critical layers
55 plt.scatter(prediction_onset, dss_scores[prediction_onset], color='
    green', s=100, label='Prediction Onset')
56 plt.scatter(semantic_inversion, dss_scores[semantic_inversion],
    color='yellow', s=100, label='Semantic Inversion')
57 plt.scatter(commitment_layer, dss_scores[commitment_layer], color='
    red', s=100, label='Commitment Layer')
58 plt.legend()
59 plt.savefig("hallucination_onset_analysis.png")
60 plt.close()
61
62 print(f"Critical layers identified:")
63 print(f"- Prediction Onset Layer: {prediction_onset}")
64 print(f"- Semantic Inversion Point: {semantic_inversion}")
65 print(f"- Commitment Layer: {commitment_layer}")
66
67 # Generate network visualizations for these critical layers
68 tracer.visualize_concept_network(
69     prompt=prompt,
70     spurious_spans=[],
71     concept_importance={},
72     concept_examples=["concert", "bass", "instrument", "fish", "
    music", "water", "animal"],
73     num_layers=3, # Only visualize the critical layers
```

```
74     figsize=(12, 10)
75 )
```

C.2.4 Analyzing Intrinsic Causes

```
1  from transformers import AutoModelForCausalLM, AutoTokenizer
2  from ltr.dst import DistributionalSemanticsTracer
3  import matplotlib.pyplot as plt
4  import pandas as pd
5
6  # Load model
7  model_name = "Qwen/Qwen3-0.6B"
8  model = AutoModelForCausalLM.from_pretrained(model_name)
9  tokenizer = AutoTokenizer.from_pretrained(model_name)
10
11 # Initialize tracer
12 tracer = DistributionalSemanticsTracer(model, tokenizer)
13
14 # 1. Reasoning Shortcut Hijack Example
15 shortcut_prompt = "The bass player in the jazz band caught a large
16                   bass at the lake yesterday."
17 shortcut_result = tracer.run_analysis(
18     prompt=shortcut_prompt,
19     factual_prompt="Bass can refer to a musical instrument and a
20                   type of fish.",
21     concept_examples=["bass", "player", "instrument", "fish", "lake",
22                     ", "jazz"],
23     hallucinated_output="fish",
24     run_intervention=True,
25     enhanced_viz=True
26 )
27
28 # Get concept correlations to visualize associative vs contextual
29   pathways
30 shortcut_matrix = tracer.create_concept_correlation_matrix(
31     prompt=shortcut_prompt,
32     concept_examples=["bass", "player", "instrument", "fish", "lake",
33                     ", "jazz"]
34 )
35
36 # 2. Analogical Collapse Example
37 collapse_prompt = "John is going fishing, so he walks over to the
38                   river bank. Can he withdraw money at the bank?"
39 collapse_result = tracer.run_analysis(
40     prompt=collapse_prompt,
41     factual_prompt="A river bank is a geographical feature, not a
42                   financial institution.",
```

```
36     concept_examples=["bank", "river", "money", "withdraw", "
37         fishing"],
38     hallucinated_output="Yes",
39     run_intervention=True,
40     enhanced_viz=True
41 )
42 # Get concept correlations to visualize associative vs contextual
43     pathways
44 collapse_matrix = tracer.create_concept_correlation_matrix(
45     prompt=collapse_prompt,
46     concept_examples=["bank", "river", "money", "withdraw", "
47         fishing"]
48 )
49 # 3. Large-scale correlation study simulation
50 # Note: This is a simplified example - for full reproduction, you'd
51     need to process the entire HALoGEN dataset
52
53 # Define example prompts
54 example_prompts = [
55     "I am at a concert. I see a bass. Is it a fish?",
56     "John is going fishing, so he walks over to the river bank. Can
57         he withdraw money at the bank?",
58     "The physician is somebody's grandmother. Is the physician a
59         woman?",
60     "I am coding. I see some java. Is it a programming language?"
61 ]
62
63 # Define corresponding ground truths
64 ground_truths = [False, False, True, True]
65
66 # Calculate DSS scores for each example
67 dss_scores = []
68 for prompt in example_prompts:
69     # Define appropriate pathways for each prompt
70     # (This would need to be adjusted for each specific example)
71     correct_pathways = [["concert", "bass", "instrument"]]
72     incorrect_pathways = [["bass", "fish", "animal"]]
73
74     # Calculate DSS
75     dss = tracer.compute_dss(
76         prompt=prompt,
77         correct_pathways=correct_pathways,
78         incorrect_pathways=incorrect_pathways
79     )
80     dss_scores.append(dss)
```

```
78 # Create dataframe with results
79 df = pd.DataFrame({
80     'prompt': example_prompts,
81     'hallucination': [not truth for truth in ground_truths],
82     'dss': dss_scores
83 })
84
85 # Plot correlation between DSS and hallucination rate
86 plt.figure(figsize=(10, 6))
87 plt.scatter(df['dss'], df['hallucination'].astype(int), alpha=0.7)
88 plt.xlabel('Distributional Semantics Strength (DSS)')
89 plt.ylabel('Hallucination (1=Yes, 0=No)')
90 plt.title('Correlation between DSS and Hallucination')
91 plt.grid(True)
92 plt.savefig("dss_hallucination_correlation.png")
93 plt.close()
94
95 # Calculate correlation
96 correlation = df['dss'].corr(df['hallucination'])
97 print(f"Correlation between DSS and hallucination: {correlation}")
```

C.2.5 Quantitative Evaluation of Explanation Faithfulness

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 from ltr.dst import DistributionalSemanticsTracer
3 import pandas as pd
4
5 # Load model
6 model_name = "google/gemma2-2b"
7 model = AutoModelForCausalLM.from_pretrained(model_name)
8 tokenizer = AutoTokenizer.from_pretrained(model_name)
9
10 # Initialize tracer
11 tracer = DistributionalSemanticsTracer(model, tokenizer)
12
13 # Define a sample of prompts for evaluation
14 # (In a full study, you would use all 11,000 HALoGEN prompts and
15   750 Racing Thoughts prompts)
16 sample_prompts = [
17     "Please answer succinctly. I am at a concert. I see a bass. Is
18     it a fish?",
19     "John is going fishing, so he walks over to the river bank. Can
20     he withdraw money at the bank?",
21     "Please answer succinctly. I am coding. I see some java. Is it
22     a programming language?",
23     "Please answer succinctly. The physician is somebody's
24     grandmother. Is the physician a woman?"
25 ]
```

```
21
22 # Define factual versions of prompts
23 factual_prompts = [
24     "A bass at a concert is a musical instrument, not a fish.",
25     "A river bank is a geographical feature, not a financial
26     institution.",
27     "Java in a coding context is a programming language.",
28     "A grandmother is a woman."
29 ]
30
31 # Define hallucinated outputs for each prompt
32 hallucinated_outputs = [
33     "Yes",
34     "Yes",
35     "No",
36     "No"
37 ]
38
39 # Initialize results collection
40 results = []
41
42 # Run DST analysis on each prompt
43 for i, (prompt, factual, hallucinated) in enumerate(zip(
44     sample_prompts, factual_prompts, hallucinated_outputs)):
45     print(f"Processing prompt {i+1}/{len(sample_prompts)}")
46
47     # Run DST analysis
48     result = tracer.run_analysis(
49         prompt=prompt,
50         factual_prompt=factual,
51         concept_examples=["concert", "bass", "instrument", "fish",
52             "music"], # Would need customization per prompt
53         hallucinated_output=hallucinated,
54         run_intervention=True,
55         enhanced_viz=False # Disable visualizations for batch
56         processing
57     )
58
59     # Extract relevant metrics for faithfulness evaluation
60
61     # 1. Concept importance - measure of causal faithfulness
62     max_importance = max(result.concept_importance.values()) if
63         result.concept_importance else 0
64
65     # 2. Spurious spans - measure of attribution faithfulness
66     attribution_score = len(result.spurious_spans) / 5 if result.
67         spurious_spans else 0
```

```
63     # 3. Intervention success - measure of counterfactual
        faithfulness
64     intervention_success = 0
65     if result.intervention_results and 'critical_layer_patching' in
        result.intervention_results:
66         if result.intervention_results['critical_layer_patching']['
            predicted_next'] != hallucinated:
67             intervention_success = 1
68
69     # Composite faithfulness score (simple average of three
        components)
70     faithfulness_score = (max_importance + attribution_score +
        intervention_success) / 3
71
72     # Store results
73     results.append({
74         'prompt': prompt,
75         'causal_faithfulness': max_importance,
76         'attribution_faithfulness': attribution_score,
77         'counterfactual_faithfulness': intervention_success,
78         'composite_faithfulness': faithfulness_score
79     })
80
81     # Create summary dataframe
82     results_df = pd.DataFrame(results)
83     print(results_df)
84
85     # Calculate average faithfulness
86     avg_faithfulness = results_df['composite_faithfulness'].mean()
87     print(f"Average faithfulness score: {avg_faithfulness:.4f}")
```